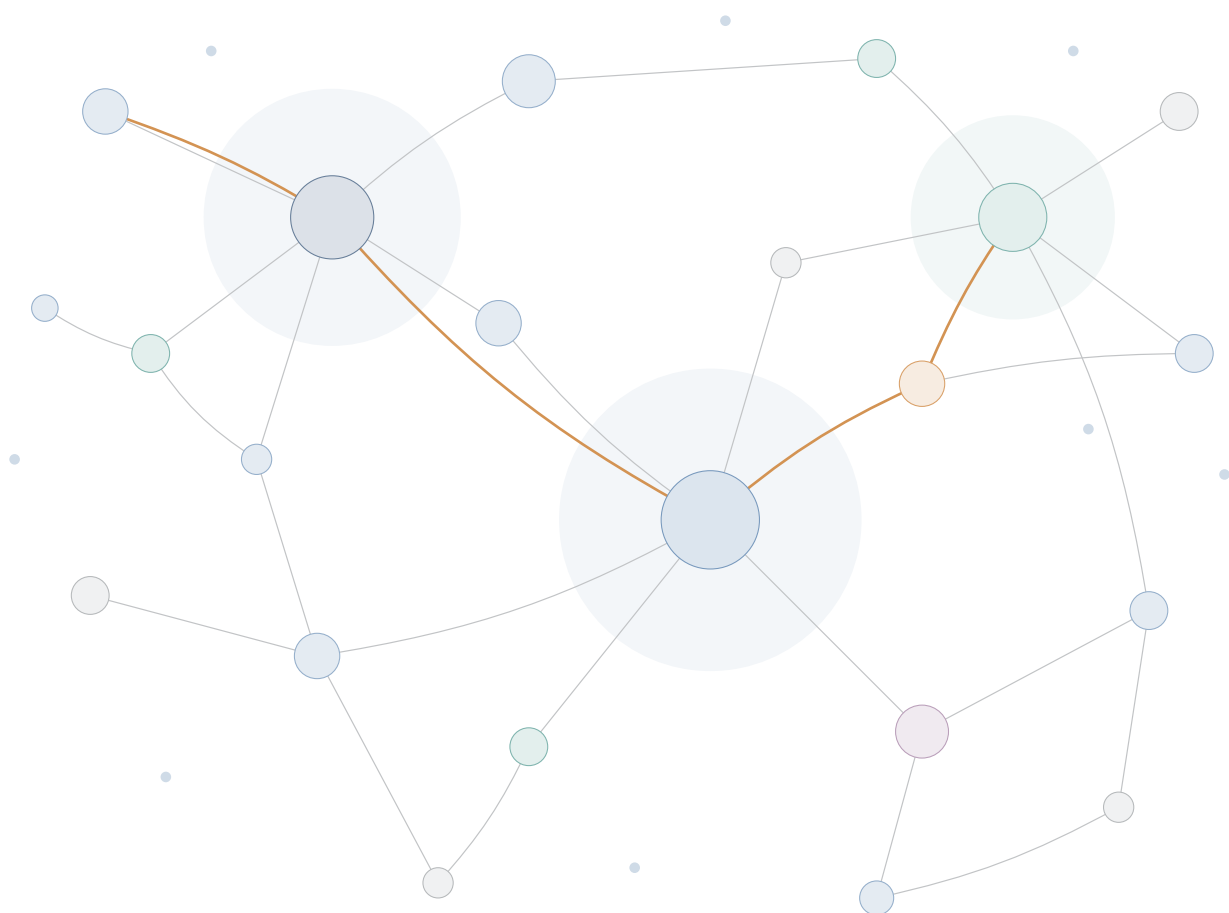


アーカイブズ学を学ぶ人のためのテキスト

Records in Contexts 入門

アーカイブズ記述の新しい国際標準を理解する

第 1.3 版 2026 年 8 月



久保山哲二

学習院大学大学院人文科学研究科アーカイブズ学専攻/計算機センター

本書について

本書は、国際文書館評議会 (ICA: International Council on Archives) が策定したアーカイブズ記述の新しい国際標準 Records in Contexts(RiC) を、アーカイブズ学を学ぶ人に向けて解説するテキストである。以下の一次資料の内容に基づいて執筆した。

- *Records in Contexts—Foundations of Archival Description* (RiC-FAD) Version 1.0, ICA EGAD, 2023 年 11 月
- *Records in Contexts—Conceptual Model* (RiC-CM) Version 1.0, ICA EGAD, 2023 年 11 月
- *Records in Contexts—Ontology* (RiC-O) Version 1.1, ICA EGAD, 2025 年 5 月 (OWL ソースファイル、公式ドキュメント、公式サンプルデータを含む)
- *Records in Contexts—Application Guidelines* (RiC-AG) Version 0.1(草案), ICA EGAD, 2025 年 10 月 (本文、公式クロスワーク、ダイアグラム類を含む)

一次資料はいずれも英語で書かれ、しかもその中心である RiC-CM と RiC-O は「モデルの仕様書」という性質上、初学者への配慮が十分とは言えない。定義は抽象的で、具体的な作業手順はほとんど示されない。「どう使えばよいのか」という問いに答える第 4 部 RiC-AG は、2025 年 10 月によりやく最初の草案 (0.1) が公開されたところであり、それ自身「段階的な手順書を提供することは不可能」と明言する、事例と考え方の集成である。本書は、仕様書と実務の間になお残る距離を埋めることを目的に、4 部すべての内容を踏まえて、原文を忠実に紹介しつつ、背景となる考え方、他分野の技術との関係、実践への道筋を補って解説する。

■**本書の構成** 第 1 章で RiC 標準の全体像と一次資料の構成を示す。第 2 章で従来の標準 (ISAD(G) など) と、参照 (記述同士をつなぐ仕組み) がどう扱われてきたかの経緯を確認し、第 3 章で RiC の新しさを論じる。第 4 章は RiC-CM(概念モデル)、第 5 章は RiC-O(オントロジー) の解説である。第 6 章では推論という、RiC-O の利点が最も分かりやすく現れる仕組みを具体例で示す。第 7 章は実践の手順、第 8 章は必要となるコンピュータ科学の素養を扱う。巻末付録に実体・属性の一覧と用語集を収めた。

■**凡例** 原語を併記すべき用語は「実体 (entity)」のように示す。RiC-O の語彙は `rico:Record` のように接頭辞付きの等幅書体で示す。原文の該当箇所は「(RiC-CM §1.6)」のように節番号で示すので、必ず原文と往復しながら読んでほしい。本書は地図であり、領土は一次資料の側にある。

目次

本書について	ii
第 1 章 RiC とは何か — 標準の全体像と一次資料	1
1.1 RiC の輪郭	1
1.2 標準の構成 — 4 つの部分	1
1.3 一次資料の入手先と構成	3
1.4 開発の経緯	4
1.5 この標準は誰のためのものか	5
第 2 章 ISAD(G) の世界から RiC へ	6
2.1 ICA の 4 つの記述標準	6
2.2 従来の記述を支えていた前提	7
2.3 記述と通信技術	8
2.4 参照と識別子 — 記述をつなぐ仕組み	10
2.5 ISO 23081 との関係	14
2.6 隣接標準との関係 — GLAM の隣人たち	15
2.7 移行はどう構想されているか	16
第 3 章 RiC がもたらした変化	17
3.1 変化 1: 検索手段のモデルから、世界のモデルへ	17
3.2 変化 2: 多段階記述から多次元記述へ	19
3.3 変化 3: 出所概念の拡張	19
3.4 変化 4: 「記述単位」の解体	22
3.5 変化 5: 記録と具現化 (Instantiation) の分離	23
3.6 記述の自由度と、そのトレードオフ	24
第 4 章 RiC-CM を読む — 実体・属性・関係	27
4.1 準備: 「実体」という言葉	27
4.2 実体の階層	29
4.3 属性	33
4.4 関係	34
4.5 記述を記述する	38

第 5 章	RiC-O — オントロジーという実装	42
5.1	RDF 入門 — 3 語の文でできた世界	42
5.2	オントロジーとは何か	49
5.3	なぜ同じ土俵に載るのか — RDF・OWL・SKOS の層構造	57
5.4	RiC-O の全体像	58
5.5	RiC-CM から RiC-O へ — 何がどう対応するか	59
5.6	実例を読む	61
第 6 章	推論 — グラフだからできること	64
6.1	推論とは何か	64
6.2	場面 1: 階層をまたぐ検索 — 推移性	65
6.3	場面 2: どちら向きに書いても見つかる — 逆プロパティ	67
6.4	場面 3: 詳しく書いた人の損にならない — 下位プロパティのロールアップ	67
6.5	場面 4: 近道の自動生成 — プロパティ連鎖	68
6.6	場面 5: 組織の変遷をたどる — アーカイブズらしい総合問題	68
6.7	場面 6: 自分の規則は足せるか — 拡張性と、OWL では書けない規則	70
6.8	処理系の中身 — 書いた記述を誰が実行するか	72
6.9	推論は魔法ではない — 注意点	75
第 7 章	実践の手順	76
7.1	3 つのシナリオ	76
7.2	シナリオ A の標準的な工程	77
7.3	通し例: 目録と典拠レコードの連携は、RiC でどうなるか	79
7.4	ドメイン知識をどこに置くか — 3 つの置き場所	86
7.5	共有の基盤は誰が用意するのか — 識別子と拡張のインフラ問題	91
7.6	日本語データの実務 — 和暦・字体・多言語	92
7.7	個人情報とアクセス制限 — どこまで公開するか	94
7.8	シナリオ B: 新規記述のワークフロー	94
7.9	工具箱	95
7.10	最初の一步の例 — 演習として	97
第 8 章	どこまで技術を学ぶ必要があるか	99
8.1	役割別の必要水準	99
8.2	要素技術の地図	100
8.3	リレーショナルデータベースの経験はどう活かすか	101
8.4	学習ロードマップ	102
付録 A	付録	104
A.1	RiC-CM 1.0 実体一覧	104
A.2	関係の 13 の型 (RiC-CM §5.2)	105
A.3	RiC-CM 1.0 関係一覧 — 原文との対照	105

A.4	よく使う RiC-O 語彙の抜粋	108
A.5	確認問題のヒント	108
A.6	用語集 — 原語との対照、分野で意味がずれる言葉	109
A.7	一次資料と参考リソース	111
索引		113

第 1 章

RiC とは何か — 標準の全体像と一次資料

1.1 RiC の輪郭

Records in Contexts(RiC) は、国際文書館評議会 (ICA) の記述専門家グループ EGAD(Expert Group on Archival Description) が策定した、アーカイブズ記述のための国際標準である。その名の通り「文脈の中の記録 (records *in contexts*)」を記述することを目的とする。従来の ICA 標準である ISAD(G)、ISAAR(CPF)、ISDF、ISDIAH の 4 つを統合して置き換える、単一の包括的標準である (RiC-CM §1.2)。

RiC の最大の特徴は、記述の**成果物** (検索手段=ファインディングエイド) の様式を定めるのではなく、記述の**対象となる世界そのもの**—記録、それを作り使った人々、人々の活動、そしてそれらの間の関係—をモデル化する点にある。従来の階層的な「フォンド単位の記述」も表現できるが、それにとどまらず、記録をめぐる関係のネットワーク (グラフ) 全体を記述できる。この転換が何を意味するかは第 3 章で詳しく論じる。

1.2 標準の構成 — 4 つの部分

RiC は単一の文書ではなく、性格の異なる 4 つの部分から構成される (RiC-CM §1.1)。この構成を頭に入れておくことが、一次資料を読むときの最初の助けになる。

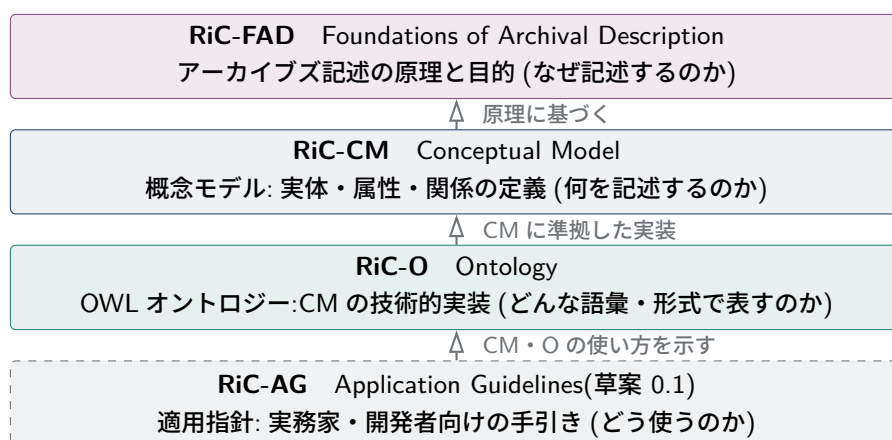


図 1.1 RiC 標準の 4 部構成。上から下へ、抽象 (原理) から具体 (実務) への層をなす。

1.2.1 RiC-FAD — 原理

RiC-FAD(Foundations of Archival Description) は、わずか8ページほどの短い文書で、アーカイブズ記述の基礎にある原理と目的を述べる。記録管理の歴史(メソポタミアの粘土トークンにまで遡る)、出所原則(Principle of Provenance)の成立とその拡張、アーキビストの役割、記述の3つの目的(記録の管理・保存・利用)を扱う。RiC-CMが「なぜこうモデル化するのか」を正当化する理論的な土台であり、CM本文でも繰り返し参照される。もともとRiC-CM 0.1の一部だったものが独立の文書になった経緯があり、CMと併せて読むことを前提としている。

1.2.2 RiC-CM — 概念モデル

RiC-CM(Conceptual Model) が標準の中心文書である(約130ページ)。実体関連モデル(ERM: Entity-Relationship Model)の枠組みを用いて、記述の対象を「実体(entity)」「属性(attribute)」「関係(relation)」として定義する。RiC-CM 1.0は19の実体、42の属性、85の関係(多くは逆向きの関係と対で定義される)を定める。

重要なのは、RiC-CMが「高水準の(high-level)概念モデル」だと自己規定していることである(RiC-CM §1.2)。RiC-CMは次のいずれでもないと言明される。

- 記述内容の書き方(データ内容規則)を定める標準ではない
- システムの実装仕様ではない
- 記録の物理的管理のモデルではない
- データ交換フォーマットではない

つまり、EAD(Encoded Archival Description。検索手段をコンピュータで交換できる形式に符号化するための標準)とも、各国の目録規則(たとえばDACSやRAD)とも役割が異なる。RiC-CMは「世界をどんな概念で切り分けるか」だけを定め、その表現方法や運用は実装側に委ねる。ここが、後述する「自由度の高さ」と「ガイドラインの不足」の両方の源泉になる。

1.2.3 RiC-O — オントロジー

RiC-O(Ontology)は、RiC-CMをWebオントロジー言語OWL 2(ウェブ技術の標準化団体W3Cが定める形式言語)で形式化した「特定の一実装(a specific implementation)」である。RiC-CMが紙の上の概念図だとすれば、RiC-Oは機械が読める語彙集であり、アーカイブズ記述をLinked Open Data(LOD。ウェブ上で相互にリンクされた公開データ。第5章)として公開し、SPARQL(そのデータのための検索言語。第6章)で検索し、論理推論を行うための道具立てを提供する。RiC-O 1.1は107のクラス、75のデータタイププロパティ、480のオブジェクトプロパティを定義しており、高水準のRiC-CMよりはるかに詳細である。CMに存在しない構成要素(Proxyクラスなど)も多く含む。第5章で詳しく扱う。

なお「CMの実装はRiC-Oだけとは限らない」ことにも注意したい。RiC-Oの公式ドキュメントは、他の技術的実装(たとえば将来のEAD/EAC-CPFの改訂版)がありうると明記している。CMとOは「概念とその一実装」という関係である。

1.2.4 RiC-AG — 適用指針 (草案)

RiC-AG(Application Guidelines) は、実務家・開発者・管理者向けに RiC の理解と実装を助ける第 4 部である。最初の草案 0.1 が 2025 年 10 月 29 日、バルセロナの ICA 大会で公開された。EGAD はこれを「生きた文書 (living document)」と位置づけ、コミュニティからのフィードバックを募りながら育てていくとしている。

内容は大きく 2 系統に分かれる。第一に RiC 全体を主題とする部分で、RiC の効用 (利用者・実務家・管理者それぞれにとって)、実装戦略、他標準との併用、そして既存 4 標準から RiC-CM への公式クロスウォーク (対応表) と EAD 2002 から RiC-O 1.1 へのクロスウォークを含む。第二に実践の指針で、既存の ISAD(G) 型目録を RiC に写す実例 (英国国立公文書館の目録を素材にした Getting Started)、FAQ 群 (記録か記録集合か、具現化、順序、活動の記述、語彙の使い方、拡張の方法など)、先住民コミュニティに関するケーススタディ、セルバンテスと『ドン・キホーテ』出版を素材にした拡張例が並ぶ。

注意すべきはその性格である。RiC-AG は冒頭で「利用場面は多様すぎるため、段階的な手順書 (step-by-step guide) を提供することは不可能」と明言し、代わりに「検討しうる行動のリスト」と豊富な事例・ダイアグラムを提供する。つまり第 4 部が出て、RiC は「これに従えばよい」型の標準にはならなかった。判断の支援はするが、判断そのものは実装者に残る—この設計思想は第 3 章・第 7 章で改めて論じる。

コラム: 仕様書とガイドライン — 期待できることの違い

RiC-CM を初めて読んだ人の多くが「結局、何をどう書けばいいのか分からない」という感想を持つ。これは文書の性格から来る自然な反応である。RiC-CM は語彙の定義集であり、そこに書かれているのは「世界の切り分け方」である。ISAD(G) は 26 の記述要素と適用規則をコンパクトに示していたので、記入すべき項目の一覧としてそのまま使えた。RiC-CM にはそれに当たる様式がない。2025 年に公開された RiC-AG 草案はこの落差を埋めるための文書で、事例を通じて判断の仕方を示すという形をとる (AG 自身、記録か記録集合かの判断について「決定的な正解・不正解はない」と書く)。手順を求めるなら AG と本書第 7 章、語彙の意味を確かめるなら CM、と役割を分けて読むのが实际的である。

1.3 一次資料の入手先と構成

RiC の一次資料はすべて無償で公開されており、Creative Commons Attribution 4.0 ライセンスで利用できる。GitHub リポジトリが開発と公開の場を兼ねているため、仕様書本体に加えて、版の履歴、コミュニティからのコメント (Issues)、公式サンプルデータまでが同じ場所に揃う「生きた一次資料」になっている。

RiC-O のリポジトリ構成は一次資料としてとりわけ有用なので、少し立ち入って紹介する。

- `ontology/current-version/RiC-O_1-1.rdf` — オントロジー本体 (RDF/XML 形式の OWL ファイル)。すべてのクラスとプロパティに英語・フランス語・スペイン語 (1.1 からドイツ語も) のラベルと定義コメントが付き、RiC-CM のどの構成要素に対応するかの注記 (`rico:`

表 1.1 RiC 一次資料の所在 (2026 年 7 月現在)

文書	最新版	主な入手先・内容
RiC-FAD	1.0(2023 年 11 月)	GitHub ICA-EGAD/RiC-FAD。PDF 本体と旧版 (RiC-IAD 0.2 ほか)
RiC-CM	1.0(2023 年 11 月)	GitHub ICA-EGAD/RiC-CM および ICA ウェブサイト。PDF 本体、旧版 (0.1、0.2、1.0-beta)、公開コメントへの回答記録
RiC-O	1.1(2025 年 5 月)	GitHub ICA-EGAD/RiC-O。OWL ソース (RiC-O_1-1.rdf)、HTML 版ドキュメント、クラス・プロパティの CSV 一覧、公式サンプル (仏国立公文書館ほか)、ダイアグラム集
RiC-AG	0.1 草案 (2025 年 10 月)	GitHub ICA-EGAD/RiC-AG およびウェブ版。本文 (Markdown/HTML)、公式クロスワーク (xlsx)、ダイアグラム (draw.io 形式)
関連	—	RiC-O 解説サイト (ica-egad.github.io/RiC-O)、プロジェクト一覧 (RiC-ResourceList)、利用者メーリングリスト (Records in Contexts users, Google Group)

RiCCMCorrespondingComponent) もある。事実上、これ自体が詳細版のリファレンスマニュアルである。

- `CSV_lists_of_components/` — クラス・プロパティの一覧表 (CSV)。オントロジーエディタを使わずに全体を見渡すのに便利で、EGAD 自身が「RDF や OWL に不慣れな人向け」と位置づけている。
- `examples/` — 実データの例。スイスの docuteam 社によるデジタル保存パッケージの例 (PREMIS との併用)、フランス国立公文書館の目録・典拠レコードの変換例、ストラスクライド大学文書館の例が含まれる。「実際のトリプルがどう書かれるか」を知る最良の教材である。
- `diagrams/` — 概念図 (draw.io 形式と PNG)。

1.4 開発の経緯

開発の経緯をたどると、どの版が何に対応しているか (たとえば RiC-O 0.2 は RiC-CM 0.2 対応、RiC-O 1.1 は RiC-CM 1.0 対応) が見えてくる。ウェブ上の解説記事やツールがどの版を前提にしているかは、この対応関係を手がかりに判別できる。

経緯の要点は次のとおりである (RiC-CM §1.7、RiC-O 履歴ノートによる)。

- 2012 年、ICA が EGAD を設置し、既存 4 標準を「調停し、統合し、その上に築く」単一標準の開発を委任した。メンバーは 15 か国から延べ 31 名。
- 2016 年に最初の草案 RiC-CM 0.1 が公開され、19 か国から 62 件 (数百ページ分) のコメントが

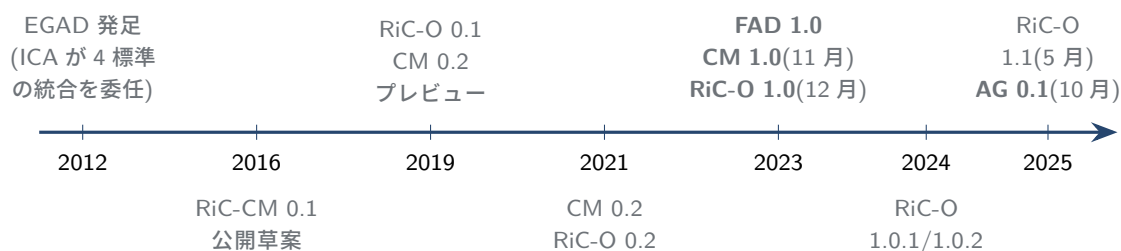


図 1.2 RiC 開発の年表。2023 年の 1.0 公開が「最初の安定した完全版」とされる。

寄せられた。このフィードバックを受けて構成が全面的に再編された。

- 2019 年 12 月に RiC-O 0.1 が公開。RiC-O はフランス国立公文書館の実証実験 (PIAAF) など、早くから実地に試されてきた。
- 2023 年 11～12 月に FAD 1.0、CM 1.0、RiC-O 1.0 が揃って公開され、「最初の安定した完全版」となった。
- その後も RiC-O は改訂が続き、2025 年 5 月の 1.1 が最新である (CM 1.0 に準拠)。クラス数 107、オブジェクトプロパティ 480 と、0.2 の頃 (423) より語彙は増えている。
- 2025 年 10 月、バルセロナの ICA 大会で RiC-AG の最初の草案 0.1 が公開され、4 部がすべて (草案を含め) 出揃った。

EGAD 自身が認めるとおり、開発への参加はヨーロッパ・北米・オセアニアに偏っており、アジアや東欧の伝統は十分に代表されていない (RiC-CM §1.7)。RiC が日本のアーカイブズ実務に合うかどうかは、これから日本のコミュニティが検証しフィードバックしていくべき課題として残されている。

1.5 この標準は誰のためのものか

RiC-CM §1.3 は想定読者を列挙している。順に、アーカイブズの専門家 (第一の読者)、レコードマネージャー、隣接する文化遺産コミュニティ (図書館・博物館)、システム開発者、そして研究者である。初学者にとって示唆的なのは、システム開発者に向けた次の一節である。「RiC-CM は高水準とはいえ詳細で複雑である。しかしシステムの開発者は、複雑さを覆い隠すユーザインタフェースを設計・実装することで、データ作成と維持の知的・技術的・経済的負担を軽減できる」(RiC-CM §1.3)。つまり EGAD は、実務者が RiC の複雑さと生のまま格闘することを想定しておらず、道具による媒介を前提に標準を設計している。RiC を学ぶことは、生の RDF を手書きする技能を身につけることではなく、道具が扱う概念の意味を理解することである。この視点は第 8 章で再論する。

確認問題 (ヒントは付録 A.5)

- 問 1.1 RiC の 4 つの部分 (FAD・CM・O・AG) の役割を、それぞれ一文で説明せよ。
- 問 1.2 RiC-CM のバージョンが 1.0 のまま、RiC-O だけが 1.1 に進んでいる。概念モデルとオントロジーが別々に改版されるのはなぜか、両者の役割の違いから説明せよ。
- 問 1.3 自分の目的 (概念の理解/実装/実務の指針) を 1 つ選び、4 部のどれをどの順で読むのが近道か、本章の内容から組み立てよ。

第 2 章

ISAD(G) の世界から RiC へ

RiC を理解する早道は、それが置き換えようとする従来標準を正確に理解することである。本章では ICA の既存 4 標準を確認し、それらがどんな世界観に立っていたか、どこに限界があったかを、RiC-CM 自身の分析 (§1.5–1.6) に沿って整理する。

2.1 ICA の 4 つの記述標準

表 2.1 ICA の既存 4 標準と RiC による統合

標準	初版/2 版	記述の対象
ISAD(G)	1994/2000	記録 (フォンドとその構成部分)。26 の記述要素と多段階記述の規則
ISAAR(CPF)	1996/2004	記録の作成者など (団体・個人・家族) の典拠レコード
ISDF	2007	機能 (組織の活動・業務)
ISDIAH	2008	所蔵機関

ISAD(G)(General International Standard Archival Description) は 1994 年に初版が出た、最も広く普及した標準である。フォンドを頂点とする多段階記述 (multilevel description) を定式化し、日本を含む世界の目録実務とシステム (および符号化標準 EAD) の土台になってきた。

ここで用語を 1 つ整理しておく。本書にはこの先「検索手段」と「目録」の両方が登場する。両者は、同じものを広さの違う言葉で呼び分けたものである。**検索手段** (finding aid) は、利用者が記録にたどり着くための道具の**総称**で、英語圏の標準文書はこの語を使う (訳語はアーカイブズ学で定着したもの)。目録、ガイド、索引などはその種類であり、日本の実務で圧倒的になじみ深い検索手段が**目録**である。だから ISAD(G) は「検索手段 (の中身となる記述) の標準」であり、その成果物は日本ではふつう目録として現れる。本書では、標準や符号化 (EAD) の話では原語に合わせて検索手段と書き、具体物としてのなじみを優先する場面では目録と書く。

残る 3 標準は、記述の主要な構成部分を**分離**する構想の産物である。ISAAR(CPF) は「作成者の記述」を記録の記述から切り離して独立に維持することを可能にし (その符号化標準が EAC-CPF)、ISDF は「機能」を、ISDIAH は「所蔵機関」を独立の記述対象とした。分離の狙いは、同じ作成者情報を複数のフォンドの記述で使い回せるようにし、記述の経済性と一貫性を高め、さらに新しい切り口からのアクセスを可能にすることだった。

しかし RiC-CM §1.6.1 は、この 4 標準体制の問題を率直に指摘する。4 標準は 14 年にわたりばらばらに開発され、「分離された構成部分をどう関係づけて全体の記述を形づくるか」についての一貫した構想を欠いていた。その結果、4 標準は「首尾一貫した単一の記述モデル」を構成していない。普及の実態にも差があり、ISAD(G) は広く使われ、ISAAR(CPF) はある程度使われ、ISDF と ISDIAH はほとんど使われていない。ICA が EGAD に委任したのは、この寄せ集めを、最初から一貫した単一の多実体モデルとして設計し直すことだった。

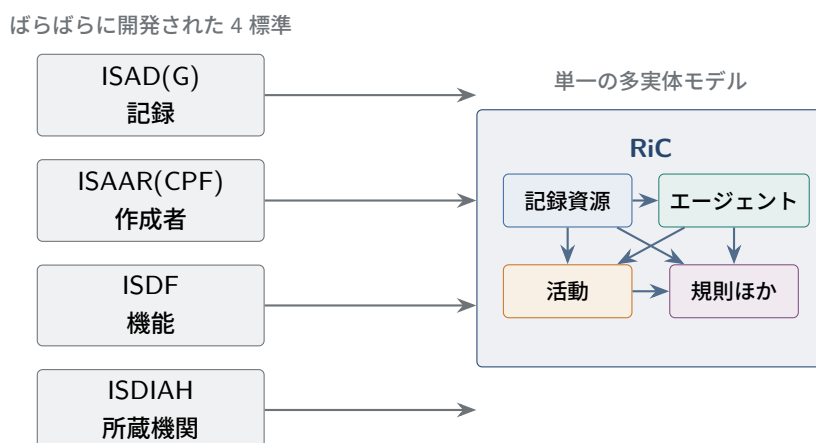


図 2.1 4 標準から RiC へ。分離されていた記述対象が、関係で結ばれた実体として 1 つのモデルに統合される。

2.2 従来の記述を支えていた前提

RiC-CM §1.5–1.6 の分析によれば、ISAD(G) 的な記述は次の前提の上に成り立っていた。

■(1) 伝統的な出所原則 記述は出所原則 (Principle of Provenance) の伝統的理解、すなわちフォンド尊重 (respect des fonds: ある個人・団体が生活や業務の中で作成・蓄積・使用した記録はひとまとまりに保ち、他と混ぜない) と原秩序尊重 (respect for original order: 蓄積・使用の文脈で形成された記録間の相互関係を保存する) に基づく。

この 2 つは役割が違う。フォンド尊重が定めるのは記録群の**外側の境界**——どこまでを 1 つのまとまりとして扱うか——であり、原秩序尊重が定めるのは**その内側の並び**——まとまりの中をどう保つか——である。境界は記述者が引くのではなく、蓄積した主体の存在によってあらかじめ与えられる、という点が重要で、だからこそ第 3 章で見る RiC の「1 つの記録が複数の記録集合に属せる」という設計が大きな変更になる。

なお、この 2 つはもともと一体だったわけではない。19 世紀フランスの respect des fonds は内部の秩序までは要求しておらず、内部の秩序は respect de l'ordre intérieur として区別されていた。両者を 1 つの原則として結びつけたのは、プロイセンの Registraturprinzip と、1898 年の**ダッチマニュアル**^{*1} である。今日ふつう「出所原則には 2 つの側面がある」と説明されるのは、この結合の結果である。RiC

^{*1} オランダの 3 人のアーキビスト Muller・Feith・Fruin による Handleiding voor het ordenen en beschrijven van archieven(英訳 Manual for the Arrangement and Description of Archives)。オランダ・アーキビスト協会のために書かれ、近代アーカイブズ学の最初の体系的な成文化とされる。

もこの2原則自体は「方法論として今も健全」と再確認している (RiC-CM §1.5.3)。問題は、伝統的理解が出所を「フォンドを蓄積した単一の主体」に切り詰めてきたことにある (第3章)。

■(2) 単一フォンドの自己完結的・階層的記述 記述はフォンドの記述から始まり、その構成部分、さらにその構成部分へと降りていく。1つのフォンドの記述は内向きに閉じ、外の世界との関係をほとんど持たない。

■(3) 入力と出力の同一視 記述システムへの入力 (データ) と利用者への提示 (出力) が同じものだと暗黙に想定されてきた。ISAD(G) は建前上これを否定している (I.6 節) が、要素の線形な並びは紙の検索手段をなぞっており、実装も「印刷物に似た画面」を作りがちだった。

■(4) アクセスポイントによる代用 記録の記述の中に、人物・団体・場所・主題などは「アクセスポイント」という形で名前だけ埋め込まれ、それ自体は記述されない。ISAAR(CPF) 以降はこの限界への対応だったが、体系的な統合には至らなかった。

コラム: RiC から見た ISAD(G) の評価

RiC-CM は ISAD(G) を高く評価している。「ISAD(G) に従った記述に本質的に誤ったところは何もない」と明言し (§1.6.2)、フォンド単位の階層的記述が当面は支配的であり続けるだろうとも述べている。理由も具体的で、伝統的出所原則に対応していること、コミュニティに深く理解されていること、既存のシステムと方法が揃っていること、そして複雑で労働集約的な課題への比較的経済的な解であることが挙げられている。RiC が着目するのは ISAD(G) の表現力の上限である。現在の通信技術で可能になったこと、利用者が期待するようになったことに応えるには、より広い枠組みが要る—これが RiC の出発点である。

2.3 記述と通信技術

RiC-CM §1.5.2 は、アーカイブズ記述が利用可能な通信技術に依存して形を変えてきたことを強調する。この節は本書の後半 (第5章・第8章) への伏線になるので、丁寧に追っておく。

20世紀末の20年間に、2つの技術がアーカイブズ記述を紙からコンピュータへ移行させた。1つはマークアップ技術—文書の中に「タグ」を埋め込んで構造を示す方式で、その代表が XML、アーカイブズでの応用が EAD—であり、もう1つはリレーショナルデータベース (表の形でデータを管理する方式。操作言語が SQL) である。RiC-CM の観察が面白いのは、「どちらの技術も支配的になりきれなかった」事実を、フォンド記述そのものの性質から説明する点である。フォンド単位の記述は、文書のような部分 (来歴の散文など) と表のような部分 (統制された要素値など) が混在する「どっちつかず (betwixt and between)」の情報であり、木構造の文書として扱う XML にも、表の集まりとして扱う RDB にも、部分的にしか適合しない。だから実装は両者を使い分け、あるいは慎重に組み合わせてきた。

XML は本書の各所に顔を出すので、文法の基礎をここでまとめておく。

コラム: XML の文法 — これだけ知れば読める

XML (Extensible Markup Language) は、文書に構造の印を書き込むための W3C 標準である。印の付け方だけを定める規格で、どんなタグを使うかは応用ごとに決める。EAD はアーカイブズ

記述のためのタグの決め方、RDF/XML は RDF のためのタグの決め方である。

要素: 構造の単位を**要素** (element) と呼び、開始タグ `<unittitle>` と終了タグ `</unittitle>` で内容をはさむ。内容が空なら `<dao/>` と 1 つで書ける。

属性: 要素に付ける補助的な値を**属性** (attribute) と呼び、開始タグの中に **名前="値"** の形で書く。値は必ず引用符で囲む。

入れ子: 要素の中に要素を入れられる。開始と終了は交差してはならず、`<a><b></a></b>` は誤りである。この規則があるので、XML 文書の構造は必ず木になる。最も外側の要素はちょうど 1 つ (ルート要素) である。

```
<c level="series" id="ser02">
  <did>
    <unitid>A-2</unitid>
    <unittitle>衛生行政文書</unittitle>
    <unitdate normal="1933/1945">昭和8-20年</unitdate>
  </did>
  <c level="file" id="fil07">
    <did><unittitle>伝染病予防関係綴</unittitle></did>
  </c>
</c>
```

この断片は EAD の一部で、シリーズ要素 `<c>` の中にファイル要素 `<c>` が入れ子になっている。記述の階層が、タグの入れ子でそのまま表される。

空白と改行: タグの外側の字下げや改行は、読みやすさのために入れるのが普通で、多くの応用では意味を持たない。ただし XML の規格上は要素の内容の一部として保存されるため、「無視してよい空白かどうか」は応用ごとの取り決めになる。この点は Turtle(第 5 章) と違って割り切れていない。

整形式と妥当: タグの入れ子が正しく、引用符が閉じているなど、文法規則を満たす文書を**整形式** (well-formed) という。さらに「どのタグをどこに置いてよいか」を定めたスキーマ (EAD なら DTD や XML Schema) に適合する文書を**妥当** (valid) という。整形式でなければそもそも読み込めず、妥当でなければ応用の要求を満たさない。

その他の約束: 文書の先頭には、文字コードなどを宣言する行を置く。

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- ここはコメント。データとしては読まれない -->
```

コメントは `<!--` から `-->` までである。`<` と `&` は構文に使う文字なので、内容に書きたいときは `<` と `&` に置き換える。要素名と属性名は大文字小文字を区別する。

これに対して RiC-CM は、第 3 の技術としてグラフ技術を挙げる。W3C の RDF(Resource Description Framework) に代表されるグラフ技術は、データをノード (実体) とアーク (関係) の網として表現し、関係をたどる問い合わせを可能にする。

ここでの**グラフ** (graph) は、棒グラフや折れ線グラフのことではない。数学でいうグラフ、すなわち**点と、点をつなぐ線からなる構造**を指す。点を**ノード** (node) または頂点、線を**アーク** (arc) または辺と呼ぶ。路線図が駅 (点) と線路 (線) からなり、家系図が人 (点) と血縁 (線) からなるのと同じ構造である。アーカイブズ記述に当てはめれば、記録・人・活動が点、それらの間の関係が線になる。図 2.2 の

右側がその形である。木(階層)はグラフの特殊な場合で、点が1本の親線しか持たないものを指す。だから階層記述はグラフの中にそのまま収まる。

ここで一点、正確を期しておく。「XMLは木しか表現できない」のではない。XMLの**構文**が直接与える構造が木(タグの入れ子)なのであって、要素にIDを振って参照し合う規約を載せればグラフも表現できる—現にRDFのXML形式(第5章)も、グラフ記述言語 GraphML も XML である。ただしその場合、グラフの構造は構文ではなく**規約の側**にあり、入れ子という構文の後押しを受けられるのは木の部分だけである。EADで階層だけが入れ子で自然に書け、それ以外の関係がID参照の記載にとどまるのは(第7章7.3節)、この非対称の現れである。これに対しRDFは、**データモデルそのものがグラフ**であり、どの関係も同じ資格のトリプルとして書ける。

「現実の世界は、空間と時間の中に位置づけられた人と物の、動的に相互関連する広大なネットワークとして理解できる」のだから、グラフのほうが現実在即した表現形式だ、というのがRiCの技術選択の論理である。RiC-CMはグラフとしての記述の**概念的土台**を与え、RiC-OがそれをRDFグラフとして実装する。

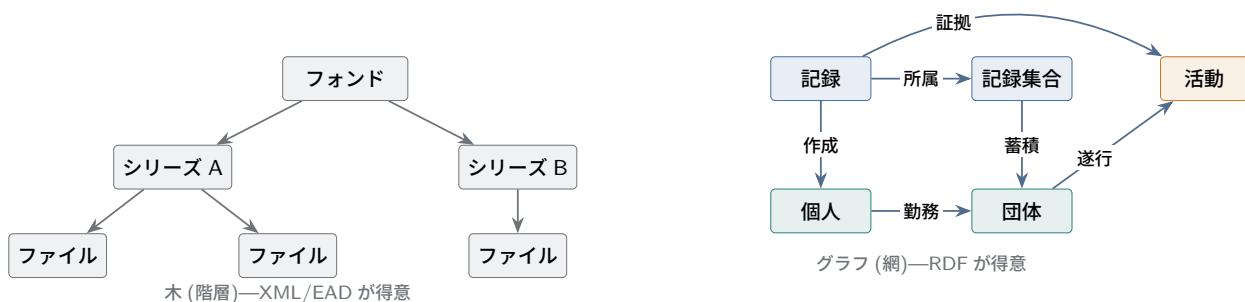


図 2.2 木からグラフへ。階層は「親子」1種類の関係しか表せないが、グラフでは関係の種類も向きも自由に増やせる。なお木はグラフの特殊な場合なので、階層記述はグラフの中にそのまま収まる。

2.4 参照と識別子 — 記述をつなぐ仕組み

RiC-Oのデータが目指す姿は Linked Data、すなわち「つながったデータ」である。この *Linked* の中身、つまり**参照**(ある記述の中から別のものを名指すこと)は、本書全体を貫く土台である。ところが参照は、実務では整理番号や典拠IDという地味な姿で現れるため、「管理の都合」程度に受け取られがちで、その役割の大きさに見合う扱いを受けてこなかった。本節では、参照が何のためにあるのか、参照には何が必要なのか、参照にはなぜ向きがあるのか、という基本から積み上げ、最後に参照手段の歴史を追う。ここを押さえておくと、第5章(RDF)と第7章(変換の実際)の意味がよく見えるようになる。

2.4.1 参照は何のためにあるか

記述とは、世界を切り分けて書いた断片の集まりである。フォンドの記述、その中のシリーズの記述、作成者の記述—これらの断片は、ばらばらに置かれていては役に立たない。断片同士をつなぐ仕組みが参照である。

参照の第一の効用は、**同じことを二度書かなくてよくなる**ことである。ある県庁が作成したフォンド

が 30 件あるとする。参照がなければ、県庁の沿革を 30 件の記述それぞれに書き写すことになり、組織改編が起きれば 30 箇所を直して回ることになる（そして必ず直し漏れる）。参照があれば、県庁の記述は 1 つだけ作り、30 件の記述はそれを**指す**。事実上 1 箇所にだけ書かれ、更新も 1 箇所で済む。データベース設計で**正規化** (normalization) と呼ばれる原則が守ろうとしているのも、これと同じことである。

コラム: 正規化とは何をすることが

正規化は、1 枚の表に混ざっている複数の対象を、対象ごとの表に分ける作業である。目録を 1 枚の表で管理している場面を考える。

資料 ID	資料名	作成者名	設置年	廃止年	上位機関
A-1	衛生行政文書	某県衛生部	1878	1947	某県庁
A-2	伝染病統計	某県衛生部	1878	1947	某県庁
A-3	検疫記録	某県衛生部	1878	1947	某県庁

作成者に関する 3 列が、資料の件数だけ繰り返されている。これが引き起こす不都合は 3 つある。第一に、設置年の誤りが見つかったとき、直す場所が資料の件数だけある (**更新時の不整合**)。1 箇所でも直し漏れれば、同じ組織に 2 つの設置年が記録されることになる。第二に、まだ 1 件も資料を受け入れていない組織を登録できない (**挿入時の不都合**)。作成者の情報を書く場所が資料の行しかないからである。第三に、ある組織の資料をすべて廃棄して行を消すと、その組織自体の情報も消える (**削除時の不都合**)。

原因ははっきりしている。「資料」と「作成者」という**別々の対象**についての事実が、1 枚の表に同居していることである。そこで表を分け、資料の側からは ID で参照する。

資料表		
資料 ID	資料名	作成者 ID
A-1	衛生行政文書	P-01
A-2	伝染病統計	P-01
A-3	検疫記録	P-01

作成者表				
作成者 ID	名称	設置年	廃止年	上位機関
P-01	某県衛生部	1878	1947	某県庁

設置年は表の中に 1 度しか現れないので、直す場所は 1 箇所である。まだ資料のない組織も作成者表に登録できる。資料をすべて消しても組織の情報は残る。3 つの不都合が同時に解消した。

ただし、これで扱えるのは「1 件の資料に作成者は 1 人」という場合だけである。共同作成のように多対多になると、作成者 ID の列に複数を書き込むわけにいかない。そこで**関係そのものを表にする**。

資料-作成者表		
資料 ID	作成者 ID	役割
A-1	P-01	起案
A-1	P-02	決裁
A-3	P-01	起案

この第3の表を**結合表**と呼ぶ。行1つが「どの資料と、どの作成者が、どういう関係にあるか」を表す。役割の列に注目してほしい。これは資料の属性でも作成者の属性でもなく、**両者を結ぶ関係の属性**である。関係を独立の行に立てたからこそ書ける。

以上の3段階は、本書がこれから追う道筋とそのまま重なる。作成者を別の表に分けることが ISAAR(CPF) の典拠レコードに当たり (本章冒頭)、結合表を立てることが RiC の関係の実体化に当たる (第4章 4.4 節、第5章)。RDB を設計した経験があれば、RiC-CM の実体・属性・関係はこの延長として読める (第8章 8.3 節)。

第二の効用は、より根本的である。そもそも**検索手段それ自体が、現物への参照装置**なのである。記述の価値は、それを読んだ利用者が (あるいはシステムが) **現物にたどり着ける**ことにあり、そのたどり着く手段がレファレンスコード (請求記号) である。つまりアーカイブズ記述は、記述→現物、記述→記述 (上位・下位・関連)、記述→典拠という参照の網としてはじめて機能する。参照は記述の存在意義そのものを担っている。

2.4.2 参照には識別子が必要である

参照が成り立つためには、指す先を**間違いなく特定できる名前**が要る。これが**識別子 (identifier)**である。「山田花子」のような名前の文字列は識別子になれない。同名の別人がいるし (一意でない)、表記がゆれるし (同定できない)、改姓・改称で変わってしまう (永続しない)。使い物になる識別子の条件は、この裏返しで3つある。

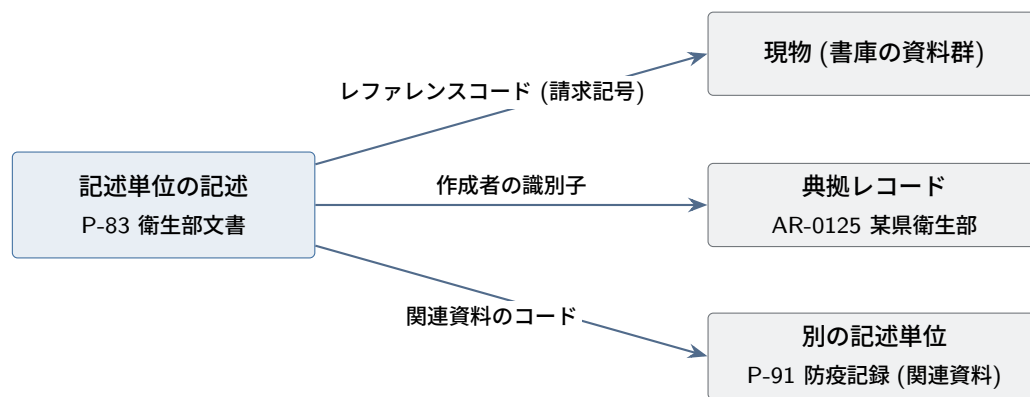
1. **一意である**: 通用する範囲の中で、1つの識別子は1つのものだけを指す。
2. **永続する**: 一度与えた識別子を変えない。変えれば、それを指していたすべての参照が壊れる。
3. **意味に依存しない**: 識別子に意味 (組織名や分類) を埋め込むと、実態が変わったとき識別子ごと変えたくなり、条件2と衝突する。「衛生部-1948-005」という番号は、衛生部が改組された瞬間に嘘になる。

ここまで来ると、ISAD(G) の設計の見え方が変わるはずである。ISAD(G) は26の記述要素のうち、国際交換に不可欠な必須要素を6つだけ挙げるが、その**筆頭**がレファレンスコード (3.1.1) である。タイトルよりも日付よりも先に、識別子が来る。しかも国コード (ISO 3166) + 機関コード + 個別番号という構造まで指定して、機関の外でも一意性が保たれるように設計されている。この配置には理由がある。**識別子を持つこと—参照の受け手になれること—が、記述単位が記述単位であるための条件**なのであり、整理番号はその条件を満たす装置である。多段階記述は上位と下位の記述が互いを指せてはじめて成立するし、他機関との交換も、利用者による出典表示も、すべて識別子を前提にしている。

2.4.3 参照には向きがある

参照は「A が B を指す」という**非対称**の行為である。指す側と指される側があり、記載は指す側に置かれる。図 2.3 は、1つの記述単位から出る3種類の参照を示す。

記載をどちら側に置くかについて、文書の世界には原則がある。「**多数の側が、1つの側を指す**」である。30件のフォンド記述が1つの典拠レコードを指すのであって、典拠レコードに30件のフォンドを



3 本とも「記述の側が指す」向き。逆向きの一覧 (この典拠を指す記述はどれか) は、
文書の世界では、別に索引を作らない限り得られない

図 2.3 1 つの記述から出る 3 種類の参照。検索手段は現物・典拠・他の記述への参照の網として機能する。参照の記載は指す側に置かれ、向きを持つ。

列挙するのではない。指す側に置けば、記載は各記述に 1 行で済み、新しいフォンドが増えても典拠側を触らなくてよいからである。

しかしこの原則には、避けがたい代償がある。逆向きにたどれないのである。「この典拠レコードを指しているフォンドをすべて挙げよ」という、利用者にとってごく自然な問い (この人の記録はほかにどこにあるか) に、文書の集まりは直接答えられない。全文書を総なめするか、あらかじめ逆引きの索引を別に作って維持するしかない。紙の世界の相互参照 (「→～も見よ」) や、データベースの索引は、この代償を人手と手間で払う装置である。—そして、参照を双方向に、しかも自動でたどれるようにすることこそ、のちに見るグラフ技術 (前節) と推論 (第 6 章の逆プロパティ) が解く問題である。参照の向きの原則と限界をここで押さえておくと、後の章の議論がすべてこの延長線上にあることが分かる。

2.4.4 参照手段の歴史 ― 名前の文字列から IRI へ

以上の道具立てで、アーカイブズ記述の歴史を「参照手段の強化の歴史」として読み直してみよう。

■第 1 期: 紙の目録 ― 人・団体への参照は文字列、解決は人間 紙の検索手段にも、識別子による参照はすでにあった。記述から現物への参照 (請求記号) がそれである。文字列のままだったのは、人物・団体・主題への言及である。同一人物の別表記を束ね、別人の同名を区別する装置として名称統制 (典拠形の統一) が図書館の世界で発達し、「→～を見よ」という相互参照も整備されたが、これらはすべて人間の読者に向けた装置である。名指された相手に実際にたどり着く作業—参照の解決 (resolution) と呼ぼう—は、人間が目録室で行うものだった。

■第 2 期: ISAD(G) とアクセスポイント ― 統制された文字列 ISAD(G) は記述の中に人物・団体・場所・主題をアクセスポイントとして、できるだけ統制された形で埋め込むことを求めた。検索の入口としては前進だが、参照の実体は依然として文字列であり、名指された人物・団体それ自体は記述されない。同名異人・改称・組織改編に構造的に弱い。誤解のないように言えば、ISAD(G) が識別子を軽んじていたわけではない。前項で見たとおりレファレンスコードは必須要素の筆頭であり、指す相手が記録であれば識別子で参照できた (関連する記述単位を指す要素 3.5.3 にはレファレンスコードを書ける)。

文字列にとどまったのは、指す相手が**人・団体・主題**の場合である。「記録へは識別子で、行為者へは名前」 という非対称がこの段階の実像であり、その後半分を解いたのが次の ISAAR(CPF) である。

■第3期:ISAAR(CPF) と典拠レコード — 文字列からレコード ID へ ISAAR(CPF) は作成者の記述を独立の**典拠レコード**に切り出し、記録の記述からは**識別子**で指す形を確立した。参照が「名前の文字列」から「レコード ID」へ格上げされた、記述史上の画期である。ただし ID の通用範囲は機関 (せいぜい国の典拠システム) にとどまり、ID から相手のレコードを機械的に取得する手続きは、各システムの内部事情に任された。

■第4期:EAD と EAC-CPF — 参照の機械可読化 符号化標準は参照を機械が読める属性にした。EAD の authfilenumber、EAC-CPF の関係要素が持つリンク属性などである。しかし3つの限界が残った。第一に、ID や URL が将来も有効である保証は運用次第である。第二に、参照の**意味** (なぜ指すのか) は要素名による粗い分類にとどまる。第三に、参照は各文書の中の記載なので、双方向の整合は保証されない。この第三の点は第7章 7.3 節で図とともに詳しく扱う。

■第5期:RiC と LOD — 世界規模の名前と、参照でできたデータモデル RiC-O が乗る RDF は、この系譜の到達点に当たる。第一に、名前が **IRI**(第5章) になり、一意性が機関ローカルでなくウェブの仕組みによって世界規模で保証される。第二に、参照の意味がプロパティという標準化された語彙で与えられ、「作成した」のか「蓄積した」のか「所蔵している」のかが機械可読に区別される。第三に—これが最も深い変化だが—RDF では**データモデルそのものが参照でできている**。トリプルの目的語に IRI を書くこと、それ自体が参照であり、参照の集まりがそのままグラフである。参照はここで記述の本体そのものになる。

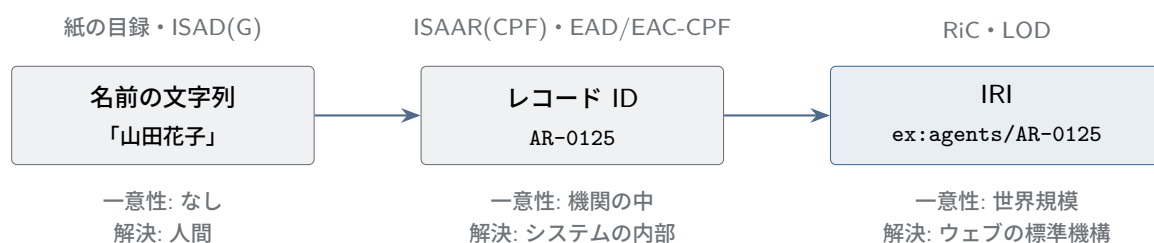


図 2.4 参照の形の変遷。名指しの一意性が通用する範囲と、名指された相手にたどり着く手段 (解決) が段階的に強化されてきた。

この節の見取り図は、以後の議論で繰り返し使う。矢印と参照の向きの読み方は第5章 5.1.2 節、文書間参照・外部キー・グラフの辺という参照アーキテクチャの比較は第7章図 7.2、IRI をだれがどう発行し維持するかというインフラの問題は第7章 7.5 節で扱う。

2.5 ISO 23081 との関係

アーカイブズ標準の外にも近縁者がいる。ISO 23081(記録管理のメタデータ) は、記録を作成の時点から証拠として管理するためのメタデータ枠組みであり、RiC-CM と「かなりの共通目的」を持つ (RiC-CM §1.6.3)。RiC-CM の多実体モデル自体、ISO 23081 の枠組み (記録・エージェント・業務・規則などの実体を関係づける) から強い影響を受けたと明記されている。実務にとっての意味は、レコー

ドマネジメントの段階で作られたメタデータをアーカイブズの記述に再利用する流れが、今後いっそう重要になるということである。両者の実体モデルにはなお差異が残るが、収斂させていくことが目標として掲げられている。

2.6 隣接標準との関係 — GLAM の隣人たち

文化資源を記述する仕事は、隣接する分野と分け合っている。美術館・図書館・文書館・博物館は、文化資源を収集・保存し記述するという共通の仕事を持つことから、英語の頭文字を取って **GLAM**(Galleries, Libraries, Archives, Museums) と総称される。それぞれのコミュニティが、記述の標準を並行して発展させてきた。RiC の位置は、これらの隣人と並べたときによく見える—そして実務では、これらと**組み合わせて**使うことになる。

表 2.2 GLAM 領域の隣接標準と RiC の接点

標準	コミュニティ	記述の中心単位	RiC との関係
CIDOC CRM (ISO 21127)	博物館	出来事 (制作・取得・移動…)	関係中心・グラフ前提という思想が最も近い。意味的マッピングの研究がある
RDA / IFLA LRM、 BIBFRAME	図書館	著作—体現形—個別資料の階層	内容と担体の分離という発想を共有。名称典拠の蓄積と識別子を借りられる
IIIF	画像・AV 配信	マニフェスト (閲覧単位)	メタデータ標準ではなく配信 API。具現化から接続し、閲覧体験を分業
PREMIS	長期保存	保存イベント・フォーマット	知的記述 (RiC) と保存管理 (PREMIS) の分業。具現化が接点
ISO 23081	レコードマネジ メント	記録と業務文脈	前節参照。多実体モデルの直接の先行者

■**CIDOC CRM — 思想上の最近傍** 博物館の CIDOC CRM は、文化財の情報を「モノ」の属性の束ではなく**出来事の連鎖** (誰がいつ制作し、誰が取得し、どこへ移動したか) として記述するオントロジーで、ISO 21127 として標準化されている。関係を第一級に据え、グラフを前提とし、来歴を出来事で追う—RiC が 2010 年代に選んだ道を、博物館界は一足早く歩いていた。違いは焦点である。CRM は文化財一般の汎用出来事モデルで抽象度が高く、RiC は記録と出所というアーカイブズ学の関心に特化する。両者の意味的マッピングは研究として行われており (RiC-CM から CIDOC CRM への対応付け)、博物館所蔵の文書資料や、文書館所蔵の美術資料のような境界事例で将来役に立つ。

■**図書館 — 内容と担体の分離、そして典拠** 図書館の概念モデル IFLA LRM(目録規則 RDA の土台) は、著作 (内容)—体現形 (版)—個別資料 (現物) という階層で書誌世界を切り分ける。米国議会図書館の BIBFRAME はその RDF 実装系である。「同じ内容の複製が多数ある」世界だからこそ発達したこの内容/担体の分離は、RiC の記録資源/具現化の区別 (第 3 章) と同じ発想である—一点物中心のアーカイブズでは 2 層で足りる、という違いはあるが。実務上より重要なのは典拠である。人物・団体・主題の名称統制は図書館が最も厚い蓄積を持ち、VIAF や各国の典拠 (日本なら Web NDL Authorities)

は IRI で参照できる。RiC の Agent 実体をこれらに owl:sameAs で結べば、図書館界のインフラをそのまま借りられる (第7章 7.5 節)。

■IIIF — 閲覧との分業 IIIF(International Image Interoperability Framework) はメタデータ標準ではなく、画像・視聴覚資料の**配信と表示**の API 標準である (画像 API と、閲覧単位をまとめるマニフェスト=JSON-LD)。RiC との接点は具現化にある。デジタル化画像を RiC 側では具現化の実体として記述し、そこから IIIF マニフェストへつなげば、「何であるか」(RiC のグラフ)と「どう見せるか」(IIIF ビューア)を分業できる。

```
ex:r1 rico:hasOrHadDigitalInstantiation ex:r1scan .
ex:r1scan a rico:Instantiation ;
    rico:identifier "https://archives.example.jp/iiif/r1/manifest" .
```

■PREMIS — 保存との分業 PREMIS はデジタル長期保存のメタデータ辞書で (OWL 版もある)、フォーマット、固定性検査、マイグレーションといった**保存イベント**を記録する。RiC は知的・文脈的な記述を、PREMIS は保存管理を受け持ち、同じ具現化を接点に併用する、というのが実際の整理である。

まとめれば、RiC は孤立した発明ではなく、GLAM 各界で同時進行してきた「属性の束からグラフへ」という転換の、アーカイブズ版である。RiC を学ぶことは、この共通の動きへの入口でもある。

2.7 移行はどう構想されているか

RiC-CM §1.6.4 は移行 (transition) について現実的な見通しを述べる。要点は2つある。第一に、RiC-CM は ISAD(G) で符号化された既存の記述実務を**包含**する (accommodate) ので、既存の記述は捨てずに RiC の枠組みの中に置き直せる。第二に、移行は漸進的であり、資源のある機関が先行して実装し、経験を共有して「道を舗装する」ことが期待されている。すべての機関が一斉に移行することは想定されていない。

この見通しは楽観的に過ぎる面もあるが、少なくともこれから学ぶ側にとって言えることは明確である。ISAD(G) と EAD の知識は無駄にならない。それどころか、既存データからの変換 (第7章) を設計するには、変換元である ISAD(G)/EAD の構造を正確に理解していることが前提になる。RiC の学習は従来標準の学習を置き換えるのではなく、その上に積み上がる。

確認問題 (ヒントは付録 A.5)

問 2.1 ISAD(G) が国際交換に不可欠とする 6 要素の筆頭はレファレンスコードである。その理由を「参照」と「識別子」という言葉を使って説明せよ。

問 2.2 本章の EAD/EAC-CPF の例 (または手元の目録) から、文字列による参照と識別子による参照を1つずつ探し出せ。

問 2.3 EAC-CPF の resourceRelation による参照は、EAD の authfilenumber による参照と向きが逆である。それぞれ、どの文書がどの相手を指しているか述べよ。

問 2.4 図書館の典拠 (VIAF や Web NDL Authorities) がアーカイブズにとっても重要なのはなぜか、本章 2.6 節の観点から説明せよ。

第3章

RiC がもたらした変化

本章では、RiC が従来標準に対して持ち込んだ変化を5つに絞って説明し、最後に「自由度が上がった代わりに何が大変になったのか」というトレードオフを正面から論じる。

3.1 変化 1: 検索手段のモデルから、世界のモデルへ

最も根本的な変化は一文で言える。「既存の ICA 標準は**記述**を、すなわち検索手段 (finding aid) をモデル化していた。RiC-CM は**実体そのもの**を、特定の成果物を予定せずにモデル化する」(RiC-CM §1.2)。

ISAD(G) の 26 要素は、突き詰めれば「よい検索手段が備えるべき項目」の一覧だった。だから要素の並び順があり、記述の単位 (unit of description) があり、階層があった。それは紙の検索手段という**出力**の形を写し取ったものである。RiC-CM はこの発想を逆転させる。まず世界のほう (記録・人・活動・規則・日付・場所) を実体と関係のネットワークとして記述し (=システムへの**入力**を標準化し)、検索手段・展示・データ公開などの**出力**は、そのネットワークから必要に応じて生成すればよい。出力の形式は標準では縛らない。「革新と実験の妨げにならないように、出力とユーザインタフェースは無制約のままにしておく」(RiC-CM §1.6.2)。

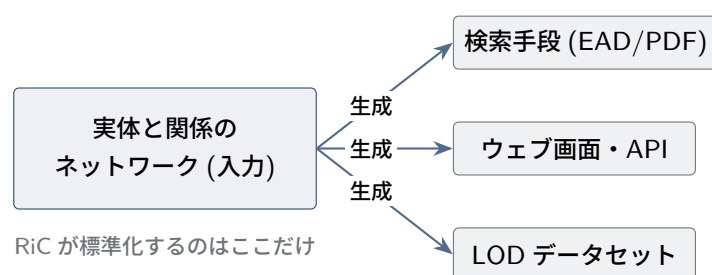


図 3.1 入力と出力の分離。RiC は入力 (記述データ) の概念構造だけを定め、出力は実装に委ねる。

この分離は現代のシステム設計では常識的な考え方であり、リレーショナルデータベースの世界で「データと表示の分離」として実践されてきたものと同じ方向を向いている。アーカイブズ記述標準がようやくそこに追いついた、と見ることもできる。

コラム: 「世界のモデル」という構想 — 知識表現の歴史との距離

「世界そのものをモデル化する」という言い方に、既視感を覚える読者がいるかもしれない。1970～80年代の第二次人工知能ブームは、文字どおり「世界の知識を形式的に書き下し、機械に推論させる」構想(エキスパートシステム、フレーム、意味ネットワーク)を追求し、そして挫折した。挫折の主因ははっきりしている。知識を手作業で形式化する費用が膨大で割に合わないこと(知識獲得のボトルネック)、書いた知識の保守が続かないこと、そして常識的な推論が形式論理では手に負えないことである。人類の常識を書き尽くそうとした Cyc プロジェクトの苦闘は、その象徴としてよく引かれる。2001年に構想されたセマンティックウェブも、当初の壮大な絵のとおりには実現しなかった。RiC が同じ轍を踏まないと言えるかどうかは、検討に値する。

まず正直に言えば、RiC の一次資料がこの歴史を明示的に論じている箇所はない。策定文書に「AI ブームの教訓を踏まえて」という記述は見当たらない。しかし設計の実質を見ると、RiC は挫折した路線とは別の、意図的に切り詰められた道を選んでいる。次の4点である。

第一に、対象領域の限定。RiC がモデル化する「世界」とは、記録・作成者・活動という**記録管理という職業が責任を持つ範囲**だけである。RiC-CM は自分が「～ではない」ものの列挙から始まり (§1.2)、主題となりうる万般の事物は Thing という受け皿で**名指すだけ**にとどめ、その記述は各分野のコミュニティに委ねる。百科事典を作る気は最初からない。

第二に、推論の弱さは意図的である。第6章で見るとおり、RiC-O の推論は分類の階層・逆関係・推移性・経路の近道という、構造的で必ず計算が終わる種類のものに限られている。常識推論はしないし、目指してもいない。セマンティックウェブ研究が挫折から学んだ「少量の意味論を広く薄く使うほうが遠くまで届く」という路線の、領域限定版と読める。

この「常識推論をしない」という選択には、技術的な裏側がある。常識推論の難所の1つは**否定の扱い**である。「この記録には作成者が書かれていない。ゆえに作成者はいない」と結論するには、手元のデータがその点について完全だという前提が要る。この前提を**閉世界仮説**(closed world assumption、CWA)と呼ぶ。第二次 AI ブームの知識表現も、リレーショナルデータベースも、Prolog の「失敗による否定」も、この仮説の上に立っている。閉世界仮説があつてはじめて「ふつう鳥は飛ぶ。ただしペンギンは飛ばない」のような、例外を後から足せる規則(非単調推論)が書ける。常識をうまく扱う道具立ては、この仮説なしには組み立てられなかった。

ところが閉世界仮説は前提が強い。データが完全でない領域では、書かれていないだけのことを「偽」と断じてしまう。アーカイブズ記述はまさにその領域である。作成者が不明の記録も、まだ調査が及んでいない記録も、日常的に存在する。そこで OWL は逆の側、**開世界仮説**(open world assumption)を採る。「書かれていないことは偽ではなく不明」とする立場である(第5章のコラム、第6章6.9節)。機関を越えてデータを寄せ集めるウェブでは、どのデータセットも部分的でしかありえないので、この選択は必然でもある。常識推論をあきらめる代わりに、寄せ集めても壊れない推論だけを残す—RiC-O の推論が地味に見えるのは、この割り切りの結果である。実務では、「作成者が記録されていない記録の一覧」のような欠如の問い合わせを、論理の否定ではなくデータの検査として書き分けることになる(第6章)。

第三に、知識獲得のボトルネックに対する答えを持っている。RiC が形式化しようとする知識は、新たに書き起こすものではなく、アーキビストが1世紀以上にわたり検索手段という形で**すで**

に生産してきた専門知識である。RiC はその再形式化であって、百科事典的な新規記入の要求ではない。知識を書く人と維持する動機を持つ職業集団が、標準の成立以前から存在している—これが第二次ブームの多くのプロジェクトとの決定的な違いである。

第四に、判断を機械に移さない。記録か記録集合か、同じ記録か新しい記録か—RiC は判断の表現形式を与えるだけで、判断そのものは一貫して人間に残す (RiC-AG の「決定的な正解・不正解はない」という態度)。機械に世界を理解させる構想ではなく、専門家の理解を機械と共有可能な形式に写す構想である。

つまり本節の見出しの「世界のモデル」は、「世界知識の網羅」ではなく「記述対象を (検索手段という出力越しにではなく) 直接モデル化する」という意味に切り詰めて読むべきであり、その限定こそが RiC の生存戦略である。ただし、教訓のすべてを免れているわけではない。形式化と保守の費用、共有基盤の不在 (§3.6、第 7 章 7.5 節) は、知識獲得ボトルネックの領域固有版にほかならない。RiC が第二次ブームの二の舞を避けられるかどうかは、語彙の設計ではなく、この費用を誰がどう負担し続けるかで決まる。

3.2 変化 2: 多段階記述から多次元記述へ

ISAD(G) の多段階記述 (multilevel description) は、フォンドを頂点とする 1 本の自己完結した階層だった。RiC-CM はこれを多次元記述 (multidimensional description) に置き換える (§1.6.2.2)。記述はグラフ (ネットワーク) の形をとり、その中に従来型の階層もそのまま収まるが、階層に閉じ込められない関係も表現できる。

原文が挙げる例が分かりやすい。ある活動が複数の組織によって順番に遂行され、1 つの記録シリーズがその活動の全期間を文書化している場合を考える。従来の記述では、このシリーズをどれか 1 つの組織のフォンドに押し込むしかなかった。多次元記述では、シリーズを活動に関係づけ、同時に歴代の各組織のフォンドの中に位置づけることができる。1 つの記録・記録集合が複数の記録集合に同時に所属できる (RiC-E03 スコープノート) ことは、研究者が作る主題別のコレクション、バーチャル展示、機関横断のプロジェクトなど、「二次的な集合」の記述にも道を開く。

注意すべきは、RiC が階層記述を否定していないことである。記録集合は記録集合をメンバーに持てる (シリーズ→サブシリーズ→ファイルという入れ子)。RiC-CM §1.6.2.2 はさらに、記録集合の記述と、そこに含まれる記録の記述の関係を精密化し、「記述の継承 (inheritance of description)」を機械処理可能にすることを目指すと述べる。集合レベルに書かれた情報のうち、メンバー全員が共有する属性・関係 (たとえば「全記録が同一活動を文書化する」) は各メンバーの記述として正当に継承できるが、要約的な情報 (たとえば年代範囲) はメンバー個々の属性ではなく「文脈」としてのみ継承される。この区別は第 4 章の属性論 (§4.3) で再登場する。

3.3 変化 3: 出所概念の拡張

「Records in Contexts」という標準名の複数形 contexts に、この変化が凝縮されている。伝統的な出所原則は、フォンドを蓄積した単一の個人・団体を特権化してきた。これに対して近年のアーカイブズ理論は 2 方向から批判を加えてきた (RiC-CM §1.5.3、RiC-FAD §6)。

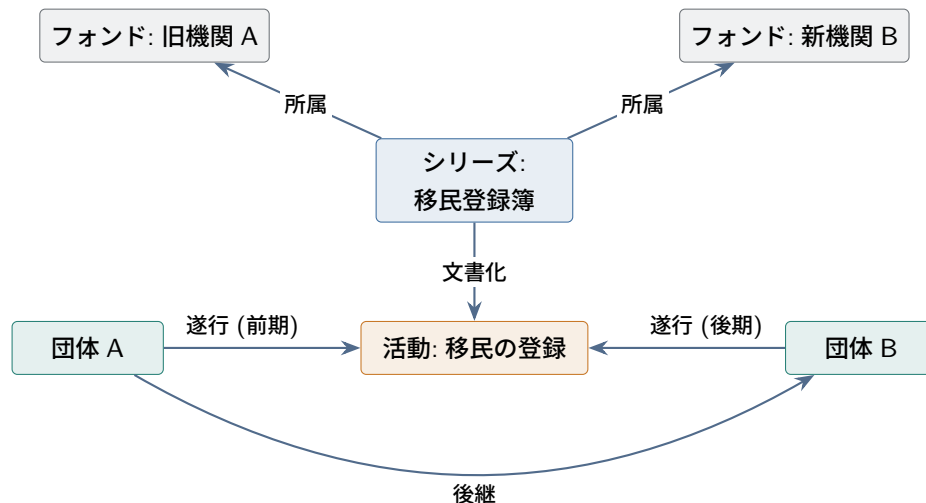


図 3.2 多次元記述の例。活動を引き継いだ2つの組織のファンド双方に、同じシリーズが位置づけられる。木構造ではこの状況を正確に表せない。

知的批判はこうである。記録の起源と歴史には、蓄積者だけでなく、作成や利用に直接関わった他の人々・団体、そして記録に関わる諸活動が含まれる。記録は複数の主体によって共同で、あるいは継起的に作られることが珍しくなく、デジタル環境の共同編集はこの複雑さをさらに増している。倫理的批判はこうである。蓄積者だけを特権化する記述は、記録の作成・利用に参加した他の人々、そして**記録の主題とされた人々**（たとえば植民地行政の記録に記載された先住民、収容所の記録に記載された被収容者）を、記述から見えなくしてしまう。

RiC の立場は「両方とも受け入れる」である。伝統的な出所原則の方法論的な健全さを再確認しつつ、出所を「記録が他の記録・活動・人・団体との動的な関係の網の中で生まれ、存在し続けること」として拡張的に理解する。多実体モデルと関係のネットワークは、この拡張された出所理解を記述として実現するための装置である。誰を記述の中心に置くかをモデルが強制しないので、同じ記録の網に対して複数の視点 (perspective) からの記述と複数のアクセス経路が共存できる。

コラム: 出所と伝来 — 何が違うのか

出所は、隣にある**伝来**という概念とまぎれやすい。整理しておく。

出所 (provenance) が指すのは**生成の文脈**である。RiC-FAD §5 の言い方では「ある個人・集団が生活や業務の中で作成・蓄積・使用した」という関係で、記録がどの主体の、どんな営みの産物かを問う。米国アーキビスト協会の用語辞典^aも、第一義を「もの (記録) を作成または受領した個人・家族・組織」としている。

伝来が指すのは、記録が作成者の手を離れた**あと**の変遷である。ISAD(G) の記述要素 3.2.3 がこれにあたり、国立公文書館による日本語訳の見出しが「伝来」である。目的は訳文で「真正性、完全性、及び解釈に際して重要な意味をもつ、記述単位の履歴についての情報を提供すること」と述べられている。

英語の名称には注意が要る。ISAD(G) 3.2.3 の英語見出しは archival history だが、米国の DACS で同じ位置にある要素 (5.1) は custodial history と呼ばれる。英語では2語あり、日本語ではどちらも伝来と訳される。

2つの標準で範囲は少し違う。ISAD(G)は「所有権、責任、管理の変遷」に加えて「編成の履歴、現在の検索手段の作成、他の目的による記録の再利用、ソフトウェアのマイグレーション」まで挙げるので、所蔵機関自身の整理作業も伝来に入る。DACS 5.1は「作成者の手を離れてから所蔵機関が受け入れるまで」と期間で切り、機関の作業には触れない。始点の扱いは共通で、ISAD(G)も「作成者から記述単位を直接入手した場合は、伝来を記録せず、収集による入手先として記録する」(3.2.4 へ回す)としている。

出所と伝来を分けているのは2つの軸である。第一に**時間**。出所は記録が生まれる場面を指し、伝来はそれ以後に記録がたどった経路を指す。第二に**問いの向き**。出所は「この記録は何の証拠か」に答え、伝来は「この記録は本物か、途中で何が起きたか」に答える。

ややこしいのは、出所という語自体に広い語義があることである。同じ用語辞典は第二義として「もの(記録)の**起源・保管・所有**に関する情報」を挙げており、この意味では伝来も出所に含まれる。実際に両者が衝突した例も報告されている。ある写真群を寄贈者(=保管の連鎖の一点)の出所として整理した結果、本来の撮影者という出所が見えなくなった、という事例である。用語辞典が「出所原則の適用が、起源にもとづく出所を覆い隠した」と要約するとおりで、広い語義で使うときは、どちらの意味かを文脈で示す必要がある。

RiCはこの区別を**関係の型**として持ち込んだ。出所関係(R026 出所を持つとその下位)の値域はエージェントで、定義は「作成・集積・受信・送信する主体」に限られる。保管はここに入らず、管理関係(R038 管理者である、R039 所持者である)として別の型に置かれる。散文で書くときはHistory属性(RiC-A21)が両方を受け止め、その仕様書きは「起源、責任、所有、保管、統制、配列、記述、管理」の履歴を含みうると列挙する。つまりRiCは、散文としてはISAD(G)の伝来を1つの属性に畳み、構造化して書くなら型の違う関係に振り分ける、という設計になっている。

隣接分野では意味がずれる。美術館や古書の世界でprovenanceといえば、作品が作られてから現在の所蔵先に至るまでの**所有の履歴**を指すのがふつうで、真正性の証明と、略奪美術品の返還請求の根拠に使われる。アーカイブズという伝来にほぼ重なり、出所とは重心が違う。一方、データの世界のprovenance(W3CのPROV標準)は「あるデータやものを生み出すのに関与した実体・活動・人についての情報」と定義され、こちらはアーカイブズの出所に近い。RiC-Oがrico:hasOrganicOrFunctionalProvenanceで出所の値域を「主体または活動」に広げているのは、PROVの発想と重なる位置にある。同じ語が分野をまたぐときは、まず「起源の話か、所有の連鎖の話か」を確かめるとよい。

^a Dictionary of Archives Terminology(SAA)。用語辞典は出発点として有用だが、そのまま信用してよいわけではない。同じ辞典のdigital signatureの項は、注記で「メッセージダイジェストを**公開鍵(public key)**で符号化したもの」が電子署名であり、受信者は「対応する**私有鍵(private key)**」で送信者の同一性を確かめられる、と述べている(2026年7月確認)。公開鍵暗号ではこれが逆で、署名は署名者の私有鍵で作成、受信者は署名者の公開鍵で検証する(NIST SP 800-63-4「私有鍵でデータに署名し、公開鍵で署名を検証する非対称鍵の操作」)。そもそも受信者が送信者の私有鍵を持てるなら、署名は成り立たない。分野の外へ踏み出した記述では、この種の取り違えが直らずつに残ることがある。定義は複数の典拠に当たり、可能ならその分野の規格で確かめたい。なおprivate keyには「秘密鍵」という訳も広く行われているが、本書は「私有鍵」を採用。NISTの用語ではsecret keyが対称鍵暗号で当事者間に共有される鍵(共通鍵)を指し、非対称鍵暗号のprivate keyとは別物だからである(NIST SP 800-175B Rev. 1はsecret keyを「対称鍵アルゴリズムで使う単一の鍵。対称鍵ともいう」と定義し、「公開鍵(非対称鍵)アルゴリズムで使うprivate keyと対比せよ」と注記する)。

コラム: 原秩序への問い

RiC-FAD §6 は原秩序尊重にも率直な疑問を向けている。「原 (original)」とはいつの状態か。蓄積の途中、記録の秩序は動的で流動的であり、何度も並べ替えられる。「秩序 (order)」とは物理的配列か知的関係か。合意はない。まったく秩序が判別できないまま移管される記録群も現実には多い。RiC が秩序を「関係」として—つまり物理配列から切り離された知的な情報として—記述する枠組みを持つのは、この反省の帰結である。RiC-O で順序の表現を担うのが第5章で扱う Proxy と順序プロパティ群である。

3.4 変化 4: 「記述単位」の解体

ISAD(G) の「記述単位 (unit of description)」は、フォンドも、シリーズも、1 通の書簡も、同じ 26 要素で記述できる便利な抽象だった。RiC-CM はこれを 3 つの異なる実体に解体する (§1.6.2.1、§2.2.2)。

記録 (Record) 生活・業務の中で形成され、担体に記録された、ひとまとまりの情報内容。

記録集合 (Record Set) 1 つ以上の記録を、共有する属性や関係に基づいてある主体がまとめたもの。

フォンド、シリーズ、ファイル、コレクションはすべて記録集合の種別 (Record Set Type) である。

記録部分 (Record Part) 記録の知的完結性に寄与する、独立の情報内容を持つ構成部分 (書簡の添付、勅許状の印章など)。

名前について先に注意しておく。Record Set という名前から「記録の集まりそのもの」を思い浮かべると誤解しやすいが、記録と記録集合の間に上下 (一方が他方の一種だという) 関係はなく、両者は記録資源の下に並ぶ別種の実体である。記録が記録集合に「属する」ことは、実体の分類とは別の層で、関係として記述される。この区別は第4章のコラムで詳しく扱う。

両者を分ける理由は、記録と記録集合が**別の活動の産物**だという点にある。個々の記録を作る行為と、記録をまとめて集合を作る行為 (分類、編成、蓄積) は、しばしば別の主体により、別の時点で、別の目的で行われる。フォンドの蓄積者と、フォンド内の個々の記録の作成者は一般に一致しない (フォンド内の記録は「混合出所」であるのが普通である)。同一視すれば記述は曖昧になる。記録集合それ自体の属性 (いつ誰が何のためにまとめたか) と、メンバーの要約 (年代範囲、言語の分布) を区別することで、記述の意味が明確になる。

もう 1 つ重要なのは、記録集合が**知的な集まり**であって物理的な集まりではないと明言されたことである (RiC-E03)。メンバーが物理的に同じ場所にある必要はない。また「何を記録と数え、何を部分と数えるか」は文脈と判断に依存することも明記されている (RiC-E02 スコープノート)。メール添付の写真は記録か記録部分か—どちらの指定も RiC は支持し、記録と記録部分がほぼ同じ属性・関係を共有するように設計することで、判断の違いが致命傷にならないようにしている。この態度 (規範ではなく判断の支援) は RiC 全体を貫いている。RiC-AG はこの種の判断に FAQ を 1 本割いており、指針も実際のである。同一性がそれ自身に由来するなら記録、メンバーに由来するなら記録集合という原則に加え、「必要な属性・関係が一方にしかなければ、それが答えの手がかりになる」こと、そして**同じ対象への判断が時間とともに変わってよいこと** (内部では個々の行を記録と見る記録集合として扱われる統計表が、半期分の抜粋として公表された後は全体で 1 つの記録と見なされる、など) を、英国の輸入統計

データや教会簿の実例で示している。

3.5 変化 5: 記録と具現化 (Instantiation) の分離

RiC-CM が導入したもう 1 つの区別は、記録 (伝えられる情報内容・メッセージ) と、その具現化 (instantiation: 担体上への記銘) の分離である (§2.2.2)。同じ議事録が Word ファイルとしても、PDF としても、紙の綴りとしても存在するとき、RiC ではこれを「1 つの記録 (Record) と 3 つの具現化 (Instantiation)」として記述できる。

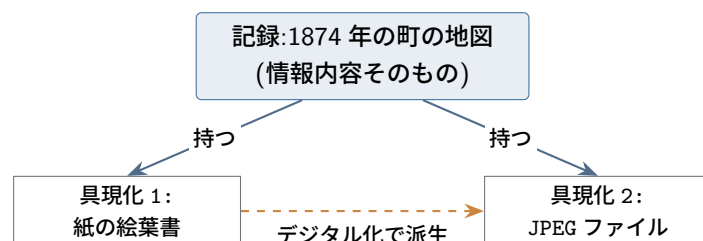


図 3.3 記録と具現化。RiC-E06 のスコープノートに基づく例。絵葉書をデジタル化した JPEG は「同じ記録の新しい具現化」とも「新しい記録」とも見なせる。どちらと見なすかは記述する主体の判断である。

規則は次のとおりである。記録と記録部分は、少なくとも 1 つの具現化を持たなければ存在しない (ただしその具現化が記述の時点で現存する必要はない—失われた原本の記述が可能になる)。記録集合は具現化を持ちうる (合冊製本された台帳など) が、必須ではない。1 つの記録が同時に、あるいは時を隔てて複数の具現化を持ちうる。具現化から別の具現化が派生しうる (複写、デジタル化、フォーマット移行)。

そして RiC はここでも判断を記述者に委ねる。派生した具現化を「同じ記録の具現化」と見るか「新しい記録」と見るかは、視点と利用の文脈に依存する (RiC-CM §2.2.2)。RiC-AG の FAQ は見分け方の目安を与えている。記述したい事柄が**具体物そのものの性質** (紙である、スキャン解像度が 600dpi である) なら具現化について、**実現のされ方から独立な事柄** (女王の手になる、この行政過程の産物である) なら記録資源についての記述である。また、機械的な複製は元の記録の新しい具現化にとどまるが、注釈が付くなど独自の歴史を持ち始めた時点で新しい記録と見なしうる、という時間依存の指針も示される。担体の物質性が意味の不可分な一部だと考える古文書学者にとって、デジタル画像は別の記録かもしれない。伝達される情報を重視する利用者にとっては同じ記録である。RiC は「二つ以上の等しく決定的な選択肢」を許す枠組みだと原文は明言する。この区別は PREMIS(デジタル保存メタデータ) の Intellectual Entity と Representation の区別に対応することも注記されており (RiC-CM 脚注 15)、デジタルアーカイブの実務と接続しやすい設計になっている。

読み分けが要る点:FRBR/IFLA LRM との違い

図書館情報学を学んだ人は、Work–Expression–Manifestation–Item という 4 層 (FRBR、現 IFLA LRM) を思い出すだろう。RiC の Record/Instantiation は 2 層で、両者は独立に設計されている。RiC-O の設計文書は、図書館・博物館・アーカイブズ概念モデルが「大きく異なる」こと、同じ用語 (person、title、author、provenance) でも意味範囲が違うことを、独自オントロ

ジーを開発した理由として挙げている。二つのモデルは、対応させるよりも別々の道具として使い分けるのが確実である。

3.6 記述の自由度と、そのトレードオフ

ここまでの変化を「記述者の自由」という観点でまとめ直すと、RiC が何を可能にし、その代償として何を要求するかが見えてくる。

3.6.1 何が自由になったか

1. **構造の自由**。記述は1本の階層に縛られない。多重所属、機関横断の関係、時間をまたぐ関係(旧機関と新機関、原本と写し)を直接表現できる。
2. **視点の自由**。誰の視点から記録を文脈づけるかをモデルが強制しない。蓄積者中心の記述も、記録に写された人々を中心とする記述も、同じ枠組みで共存できる。
3. **粒度の自由**。よく知らないシリーズは1つのノードと2~3の関係だけで粗く記述し、重要な記録は具現化・イベント・規則まで細かく記述する、という濃淡が同じデータセットの中で許される。RiC-O は上位の(粗い)クラス・プロパティと下位の(細かい)それらを階層化して提供し、どの水準で書いてもよいようにしている(RiC-O 設計原則3「柔軟な枠組み」)。
4. **表現方法の自由**。同じ事実に対して、簡便な書き方と精密な書き方の両方が用意されている(第5章)。簡便な書き方とは、値を文字列や数値そのままを書くことである。この「そのままの値」をリテラル(literal)と呼ぶ。「山田花子」「1868」のように、それ自体を読めば用が足り、そこから先へたどる相手を持たない値のことである。精密な書き方とは、同じ値を独立した記述対象として立て、そこを指すことである。作成者をリテラルで書けば名前が読めるだけだが、実体として立てれば生没年も他の記録との関係もたどれる。第4章4.1.2節の設計Aと設計Bの対比が、そのままここに現れる。日付についても同じで、文字列で書くことも、Date 実体として関係づけることもできる。
5. **成果物の自由**。入力と出力の分離により、同じデータから検索手段も LOD も展示も生成できる。

3.6.2 代償として何が大変になったか

自由の代償は、およそ次の5点に整理できる。

■(1) **判断の負担が増える** ISAD(G) は「埋めるべき26要素」を与えてくれた。RiC は「19の実体、42の属性、85の関係、480のプロパティ」という語彙を与えるが、**どれをどこまで使うかは記述者(機関)が決めなければならない**。記録か記録部分か、同じ記録の具現化か新しい記録か、日付をリテラルで書くか実体化するか—RiC が「どちらでもよい」と言う場面のすべてで、機関としての方針決定が必要になる。決定を積み重ねて文書化したものが、その機関の適用プロファイルになる(第7章)。

■(2) **一貫性の維持が難しくなる** 様式が固定されていれば、一貫性は様式が保証してくれた。自由な語彙では、同じ事実を人によって違う書き方で記述する余地が常にある。RiC-O が同じ情報に複数の表

現方法を認めている(移行を容易にするための意図的な設計である)ことは、裏面では「データセット内の不統一」というリスクである。検索・推論はデータの一貫性に依存するから、不統一は機能低下として跳ね返る。

■(3) 識別子の管理という新しい仕事が生まれる グラフのノードは安定した識別子 (IRI) を持たなければならない。作成者・活動・場所・規則を独立の実体にするということは、それらすべてに一意で永続的な ID を発行し、維持し続けるということである。従来「文字列としてのアクセスポイント」で済んでいたものが、ID 付きの資源の管理業務に変わる。これは組織的・長期的なコミットメントであり、技術というより運営の問題である。

■(4) 道具立てが未成熟である ISAD(G)/EAD には目録システム、編集ツール、交換の実績が揃っている。RiC 対応を謳うシステムはまだ少ない。2025 年 10 月に適用指針 RiC-AG の草案がようやく公開されたが、それ自身「段階的な手順書の提供は不可能」と述べる事例集であり(第 7 章)、「これに従えばよい」という様式は依然として存在しない。先行実装機関の経験と AG の事例を参照しながら、各機関が方針を作っている段階である。

■(5) 学習コストが上がる 多実体モデル、グラフ、オントロジーという考え方そのものに慣れる必要がある。ただし、必要な技術的素養は役割によって大きく異なる(第 8 章)。全員が OWL を書けるようになる必要はない。

■(6) 表現の一意性は、原理的に確保できない 最後の点は、(2) の一貫性の問題をさらに一段深めたものである。適用プロファイルをどれほど丁寧に定めても、同じ現実に対する RiC 準拠の表現は一つに定まらない。何を実体に切り出すか、どの粒度で切るか、どの関係を張るか、直接プロパティで書くか実体化するか—これらの選択の組み合わせの数は、モデルの表現力に比例して増える。記述できることが増えるほど、同じことの書き方も増える。表現の自由度と表現の曖昧さは同じものの両面であり、この曖昧さは設計方針の精緻化では消えない。正規形(これが標準的な書き方だ、という一意な形)を RiC は定めておらず、定めようとすれば自由度を手放すことになる。

この点は現場に重くのしかかる。「どうにでも書ける」は「どう書いてよいか分からない」と実質的に同じであり、書き方が機関ごと・担当者ごとに発散すれば、機関を越えた検索や交換という RiC の眼目そのものが損なわれる。様式の固定こそが標準の効用だったと考える立場からは、RiC は標準の名に値するのかという疑問さえ成り立つ。

RiC の側に用意されている緩和策は、部分的なものにとどまる。第一に、RiC-O の推論(第 6 章)は、あらかじめ预期された表現の揺れについては検索時の同値性を回復する。長い経路と近道はプロパティ連鎖で、粗い記述と細かい記述はプロパティ階層で、同じ問い合わせに答えるように設計されている。しかし第二に、预期されていない揺れ—記録と記録部分のどちらに切るか、新しい記録か同じ記録の新しい具現化か、実体をどこまで立てるか—は、機械的には吸収されない。これらは意味の水準の判断差であり、推論は救済しない。残るのは運用による収束、すなわちコミュニティで適用プロファイルを共有し、変換ツールや記述システムが事実上の正規形を作り、公式の事例集が判断の相場を形成する、という道である。

実際、2025 年に公開された RiC-AG 草案はこの分析を裏書きしている。AG は「段階的な手順書は不可能」と明言した上で、記録か記録集合かの判断に「決定的な正解・不正解はない」と書き、正規形

を定める代わりに、判断が分かれる状況を事例で示し、判断材料(たとえば「必要な属性・関係が一方にしかないなら、それが答えの手がかり」「判断は時間とともに変わってよい」)を提供する方針をとった。つまり EGAD 自身、一意性の回復ではなく判断の質の底上げを選んだのである。

RiC は「様式の標準」であることをやめて「語彙と意味の標準」になったのだから、書き方の収束はもはや標準文書の内側では起こらず、実装コミュニティの側で起こるしかない。この移行が成功するかどうかは現時点で未知数であり、本書はこれを RiC の未解決の構造的リスクとして明記しておく。なお、収束を支えるはずの**共有基盤そのもの**(拡張のレジストリ、機関を越えた識別子の典拠)が未整備であるという、もう1つの構造的な問題を第7章 7.5 節で論じる。

表 3.1 自由度とトレードオフの対応

得られた自由	引き換えに生じた負担
階層からの解放(多重所属・横断関係)	「どの関係を張るか」の選択と根拠の明示。関係の数だけ保守対象が増える
視点の複数化	記述方針(誰を・何を記述するか)の機関としての明文化
粒度の濃淡	粒度の判断基準の設定。粗い記述と細かい記述の混在の管理
複数の表現方法	機関内での書き方の統一(適用プロファイル)。不統一だと検索・推論が劣化
出力の自由	出力を作る実装力が別途必要(標準は出力を助けてくれない)
実体の独立(作成者・活動・場所など)	IRI の発行・維持という恒久的な管理業務
表現力の拡大(全般)	正規形の不在。同じ現実の表現が一意に定まらず、収束は実装コミュニティの運用に委ねられる

一言でまとめれば、RiC は「様式を埋める仕事」を「モデルを設計する仕事」に変えた。この変化は、目録作成を単純作業から知的設計へ引き上げるものと肯定的に評価することも、現場に過大な設計負担を転嫁するものと批判的に評価することもできる。EGAD の想定は、負担の多くをシステムと共有ガイドラインが吸収する、というものである(RiC-CM §1.3、§1.6.2)。RiC-AG 草案と公式クロスウォークの公開はその第一歩だが、想定が満たされるかどうかは、今後のツール開発と AG の成熟にかかっている。

確認問題(ヒントは付録 A.5)

- 問 3.1** 本章の5つの変化から、自分(または想定する機関)にとって最も価値が大きいものと、最も負担が大きいものを1つずつ選び、理由を述べよ。
- 問 3.2** 「表現の一意性がない」とはどういうことか。同じ事実の2通りの書き方の例を作り、それが検索にどう影響するかを述べよ。
- 問 3.3** 「検索手段のモデルから世界のモデルへ」という転換に対して、第二次 AI ブームの挫折を知る立場からはどんな懸念が可能か。またそれに対する RiC 側の限定は何か。

第 4 章

RiC-CM を読む — 実体・属性・関係

本章は RiC-CM 1.0 の本体 (第 2～6 章) の読解である。まず前提となる実体関連モデルの考え方を説明し、次に実体・属性・関係を順に見ていく。

4.1 準備: 「実体」という言葉

RiC-CM を読む前に、entity(実体) という言葉について立ち止まっておきたい。この語は分野によって意味の力点が異なるので、RiC-CM がどの用法を採っているかをはっきりさせておくと、以降の読解がなめらかになる。

RiC-CM が採用するのは、情報システム設計の伝統である実体関連モデル (ERM: Entity-Relationship Model) の用法である (RiC-CM §1.4)。1976 年に P. Chen が提案した ERM では、関心の対象となる「もの」を実体 (entity)、そのものの特徴を属性 (attribute)、もの同士のつながりを関係 (relation/relationship) と呼ぶ。哲学の存在論で言う実体 (substance) のような重い意味はなく、「データベースで 1 件のレコードとして管理したい対象」くらいの実務的な概念である。図書館の貸出システムなら「利用者」「資料」「職員」が実体、「氏名」「請求記号」が属性、「借りる」「所蔵する」が関係になる。

ただし正確には、ERM は「実体型 (entity type)」と「実体インスタンス (entity instance)」を区別する。「個人 (Person)」という型と、「ネルソン・マンデラ」という個々の実例である。RiC-CM が定義する 19 の実体はすべて実体型であり、各定義に添えられた Examples が実例に当たる。日常語の「実体」はこの 2 つを区別しないので、RiC-CM の実体一覧を「19 個のものの一覧」と読むのではなく、「もの 19 種類の分類体系」と読むのが正しい。

さらに紛らわしいことに、この後の章で登場する RDF/OWL の世界では、ほぼ同じ概念に別の名前が付いている。実体型に相当するのがクラス (class)、実体インスタンスに相当するのが個体 (individual) ないし資源 (resource) である。RDB では実体型がテーブルに、インスタンスが行に対応する。本書では対応関係を都度示すが、まず「同じ発想に分野ごとの方言がある」と理解しておけばよい。

4.1.1 ER 図の読み方

ERM には図の記法が伴う。1976 年の Chen の原論文に由来する古典的な記法では、実体を長方形、関係をひし形、属性を楕円で描き、線でつなぐ (図 4.2)。実務では長方形の中に属性を列挙する簡略な

表 4.1 「実体」をめぐる用語の対応 (概念は完全一致ではないが発想は共通)

	型 (種類)	インスタンス (実例)	つながり
ERM(RiC-CM)	実体 entity	実体の実例	関係 relation
RDF/OWL(RiC-O)	クラス class	個体 individual/資源 resource	プロパティ property
RDB	テーブル table	行 row	外部キー・結合
オブジェクト指向	クラス class	オブジェクト object	参照 reference

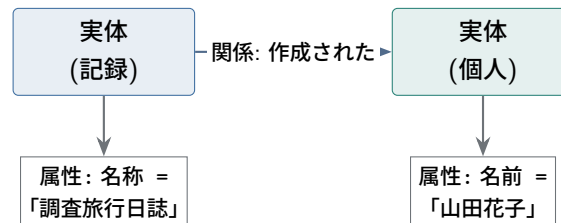
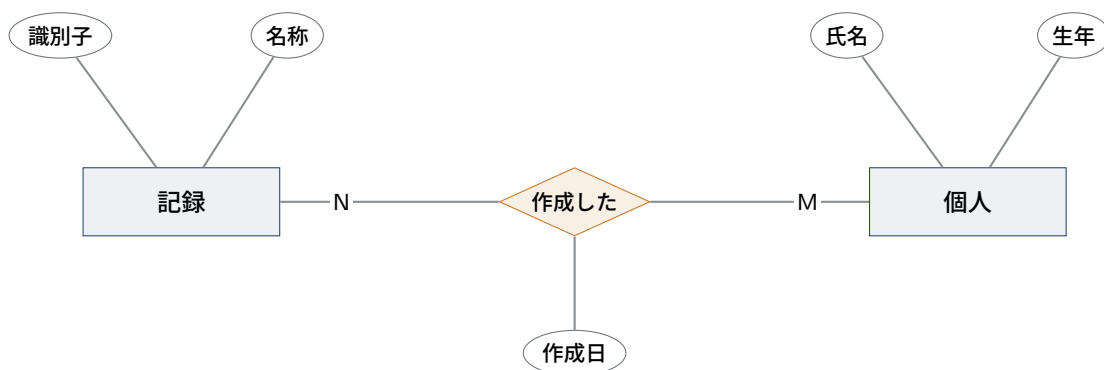


図 4.1 ERM の基本要素 (RiC-CM 図 1 に対応)。箱が実体、箱同士を結ぶのが関係、箱にぶら下がる値が属性である。

記法 (いわゆる IE 記法、あるいは UML のクラス図) を使うことも多いが、読み方の原理は同じである。RiC-CM 自身は特定の記法に縛られない散文と表で実体・属性・関係を定義しており、本書の図も RiC-CM の内容を読みやすく図示したものである。



長方形=実体、ひし形=関係、楕円=属性。N と M は多重度で、「1 つの記録が複数の個人に作成される。1 人の個人が複数の記録を作成する」を表す

図 4.2 Chen 記法による ER 図の例。関係にも属性 (作成日) を付けられる点に注目してほしい。RiC-CM が関係の属性 (§4.4) を定義するのは、この伝統を引いている。

図 4.2 で読み取るべき要素は 4 つある。第一に、長方形が実体型である。第二に、ひし形が関係で、2 つの実体型を結ぶ。第三に、楕円が属性で、実体だけでなく**関係にも**付けられる。「いつ作成したか」は記録の属性でも個人の属性でもなく、作成という**関係の属性**である。第四に、線に添えた記号が**多重度 (cardinality)** で、片方の 1 件がもう片方の何件と結びつきうるかを示す。1 対 1、1 対多、多対多の 3 種類があり、この図は多対多である。

RiC-CM も関係ごとに多重度を明記している (§5.4 の Cardinality 欄。たとえば R026 出所を持つ

は M to M、R002 部分を持つ は 1 to M)。関係に属性を付けられるという ERM の性質も、RiC-CM がそのまま受け継いでいる (§4.4)。

4.1.2 同じ対象に対してモデルは 1 つに決まらない

ERM を学ぶうえで最も大切な点は、同じ現実に対して正しいモデルが複数ありうることである。何を実体として切り出し、何を属性にとどめるかは、記述の目的によって変わる。図 4.3 に、同じ事実「調査旅行日誌は山田花子が作成した」に対する 2 つの設計を並べた。

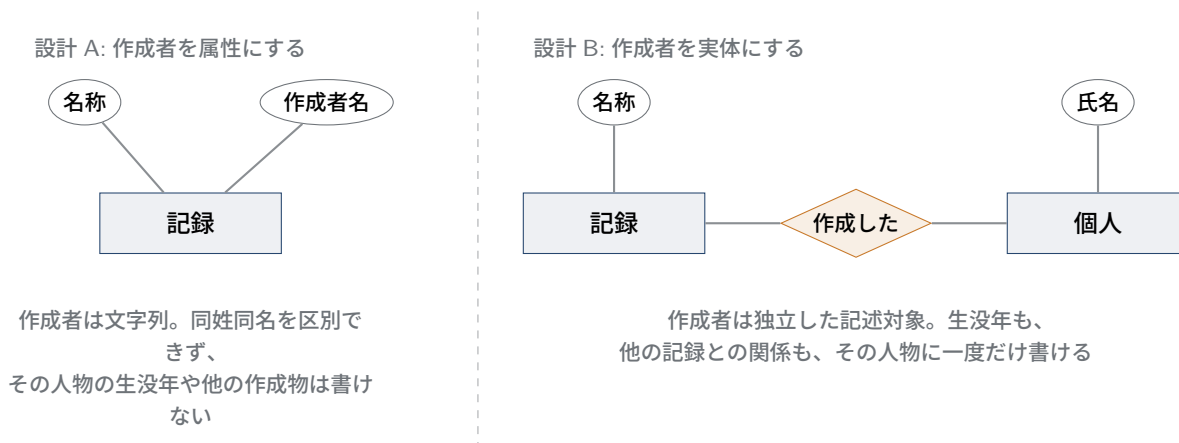


図 4.3 同じ事実に対する 2 つのモデル化。どちらも誤りではなく、記述の目的が選択を決める。

設計 A は簡便で、記録 1 件だけを見るぶんには十分である。設計 B は手間がかかるが、作成者を独立の記述対象に昇格させたことで、同姓同名の区別、生没年や経歴の記述、「この人物が作成した記録の一覧」という検索が可能になる。どちらが正しいかは一概に言えず、選ぶのは記述の目的である。

この選択は本書の主題そのものに直結する。ISAAR(CPF) が作成者の記述を典拠レコードとして独立させたのは、設計 A から設計 B への移行だった (第 2 章)。RiC-CM が 19 の実体を立てたのも同じ方向の徹底であり、逆に「概念的に統制値をとる属性は、実装では実体として扱うほうが便利なが多い」という RiC-CM 自身の注記 (§4.3) は、この 2 つの設計のあいだを行き来してよいという表明である。第 3 章 3.6 節で述べた「表現の自由度と表現の曖昧さは同じものの両面」という論点も、ここに根がある。同じ現実に対して RiC 準拠の表現が一つに定まらないのは、ERM という枠組みが最初からそういう性質のものだからである。

4.2 実体の階層

RiC-CM 1.0 は 19 の実体を定義する。実体は平らに並んでいるのではなく、「A は B の一種である (A is a kind of B)」という分類の関係で階層をなす。頂点はすべてを包み込む「もの (Thing)」である。

このうち、記録資源 (とそれに密接する具現化)、エージェント、活動の 4 つが中核実体 (core entities) とされる (RiC-CM §2.1)。「エージェントが世界の中で活動を遂行し、その過程で記録を生み、使う。記録は活動の遂行の証拠である」—この一文が RiC の世界観の要約であり、中核実体はその登場人物である。残りの実体 (イベント、規則、日付、場所) は、中核実体を十全に記述するための脇役と位置づけられる。

OWL の根クラス `owl:Thing` との対応も注記されている。

4.2.2 記録資源 (Record Resource, E02) とその下位実体

「エージェントが生活または業務活動の中で作成または取得し、保持した情報」。下位に記録集合 (E03)、記録 (E04)、記録部分 (E05) を持つ。3 者の区別と意義は第 3 章 3.4 節で述べた。定義上の要点は、記録の同一性は記録自体に由来するのに対し、記録集合の同一性はメンバーに依存して派生し、記録部分の同一性は属する記録に依存して派生する、という非対称性である。

例示が紙の文書に偏っていないことにも注意したい。RiC-CM が挙げる記録の公式例には、国王の印章が付された 17 世紀の任命状と並んで、「添付ファイル 2 点を持つ、デジタル署名付きの電子メール」がある。その添付ファイルは記録部分の例として現れる (RiC-E04, E05)。さらにスコープノート (RiC-E02) は、ボーンデジタルの情報オブジェクトをどう数えるかという問いを正面から論じている。文書ファイルに埋め込まれた画像のビットストリームは記録か記録部分か。ZIP に圧縮されてひとつのファイルになった複数のビットストリームは、記録か記録集合か。定型的な文書形式 (documentary form) を持たないデジタル情報では境界の判定が難しくなる、という認識が、この実体設計の前提にある。

「集合」という名前の読み方 — Record と Record Set は分類上の兄弟である

Record Set という名前からは、2 つの読み違いが生じやすい。「記録集合とは、記録をたくさん集めた**だけ**のものだ」という読みと、「記録は、記録集合の (1 件だけの) 特別な場合だ」という読みである。順にほどいておく。

第一に、実体階層 (図 4.4) の線が表すのは「～の一種である」という**分類**だけである。記録も記録集合も「記録資源の一種」だが、記録が記録集合の一種であることも、その逆もない。両者は分類上の兄弟であり、別の種類のものである。一方、記録が記録集合に「含まれる・属する」ことは、この分類の線とは別に、通常**の関係** (RiC-R024, RiC-O では `rico:hasOrHadMember` など) として記述される。「一種である」(分類) と「含まれる」(関係) を混同しないことが、階層図を読む第一歩である。

第二に、記録集合は「中身を集めた**だけ**のもの」ではない。文集にたとえるとよい。同じ 10 篇の文章を、ある編者は「追悼文集」として、別の編者は「地域史料集」として編んだとする。収録作品がまったく同じでも、この 2 つの文集は別のものである。誰が、いつ、何のために編んだかが違うからだ。RiC の記録集合も同じで、その同一性は「まとめるという行為」の来歴に支えられており、メンバーの顔ぶれだけでは決まらない (RiC-E03 スコープノート)。だから同じ記録群から複数の記録集合が作られてよいし、逆に、追加受入でメンバーが増えても同じ記録集合であり続けられる。

第三に、1 件の記録だけを収めた記録集合も作れる (定義は「1 つ**以上**の記録」) が、それはその記録自体とは別の実体である。1 篇だけを収めた文集が、その 1 篇と同じものでないのと同じである。文集には編者と編集の事情があり、それは収められた文章の作者や執筆の事情とは別に記述される。

4.2.3 具現化 (Instantiation, E06)

「時間と空間を越えて情報を伝達する手段として、エージェントが担体上に行った、持続的で復元可能な形で情報の記録」。第3章3.5節で述べたとおり。「担体への記録」と聞くと紙やフィルムを思い浮かべがちだが、スコープノートの例はむしろデジタル側に重心がある。同じ記録が DOCX と PDF/A と紙出力という3つの並存する具現化を持つ例、大きな記録集合 (フォンドやシリーズ) が1つのデータベースとして維持される例、ケースファイル一式が1つの PDF にまとめてスキャンされる例が挙げられる。ビニールレコード、ビデオカセット、デジタルファイルのように、情報の伝達に機器 (メディアエータ) を要する具現化があるという指摘も、デジタル保存の課題と直結する。

4.2.4 エージェント (Agent, E07) とその下位実体

「世界の中で活動を遂行するもの」。人間とその集まりに限らず、物理的な姿を持たない役割 (職位) や、ソフトウェアとして存在するもの (メカニズム) までがエージェントになれる。下位実体は次の4系統である。

個人 (Person, E08) 人間個人。スコープノートの大半は**ペルソナ** (社会的に構成された同一性) の議論に割かれている。筆名が本名を凌駕する場合 (Mark Twain)、複数人が1つのペルソナを共有する場合、人が生涯の中で同一性を変える場合など、「1人=1つの同一性」が成り立たない現実への対応が明記されている。

集団 (Group, E09) 「エージェントとして共に行動する2以上のエージェント」。下位に家族 (E10) と団体 (E11)。ISAAR(CPF) の3区分 (団体・個人・家族) はここに吸収されるが、RiC は家族と団体を「集団」の下位に位置づけ直し、有権者団のような形式化されていない集団も扱えるようにした。

職位 (Position, E12) 「集団の中での個人の機能的役割」。学長、県知事、文書課長など。個人と集団の交点を独立の実体としたことが新しい。ある職位が生んだ記録集合を、歴代の在職者個人を特定しないまま記述する、という実務上よくある状況を正確に表現できる。

メカニズム (Mechanism, E13) 「個人または集団が作成した、活動を遂行するプロセスまたはシステム」。ソフトウェア、ロボット、探査機など。設計者が定めた規則に基づいて活動し、記録を作成・改変しうる。ウェブクローラや火星探査車が例に挙がる。自動処理が記録を量産する現代への対応である。

4.2.5 イベント (Event, E14) と活動 (Activity, E15)

イベントは「時間と空間の中で起こる出来事」。自然現象 (地震、洪水) も、エージェントが引き起こすもの (選挙、文書のフォーマット変換) も、両者の複合 (水損記録の救済) も含む。活動はイベントの下位実体で、「エージェントが目的をもって設計し遂行するイベント」。団体の文脈では従来の「機能 (function)」と同じ範囲を指すが、個人や家族の営み (詩作など) には「機能」という語がなじまないため、より一般的な「活動」が採用された (RiC-CM §2.2.4)。ISDF が扱っていた領域はここに吸収される。

活動には2つの見方があるとスコープノートは言う。目的 (何のために) から見る見方と、過程 (どの

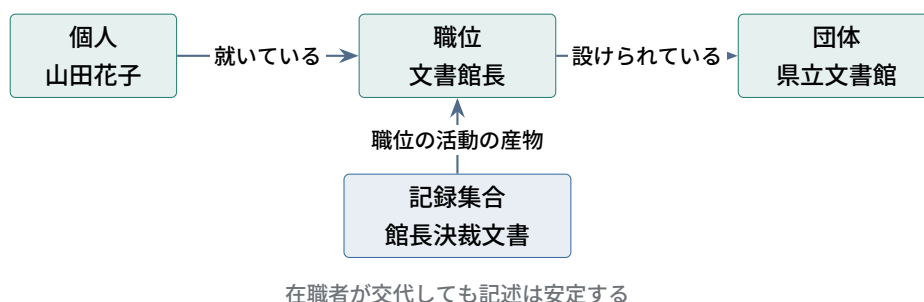


図 4.5 職位 (Position) の使いどころ。記録を「個人」でも「団体」でもなく「職位」に結びつけることで、人事異動に左右されない記述ができる (RiC-E12 スコープノートに基づく)。

ように) から見る見方である。組織の機能分類 (目的の階層) も、業務フロー (過程の連なり) も、どちらも活動の記述として表現できる。

4.2.6 規則 (Rule, E16) とマンドート (Mandate, E17)

規則は「エージェントの存在・責任・権限、活動の遂行、またはエージェントが作成・管理するものの特性を支配する条件」。成文の法令・規程・標準だけでなく、**不文**の社会規範や慣習も含むことが明記されている。マンドートは規則の下位実体で、「あるエージェントが別のエージェントに、特定の活動を遂行する責任または権限を委任すること」。

初学者が注意すべき点が2つある。第一に、規則・マンドートはそれを証拠立てる文書 (記録) とは別の**実体**である (「A documentary source is a record」と原文は釘を刺す)。「文書館法」という記録と、それが定める権限という規則は区別される。第二に、記述という活動自体も規則 (RiC-CM そのもの、各国の目録規則) に支配されており、規則実体は記述の記述 (§4.5) でも使われる。

4.2.7 日付 (Date, E18) と場所 (Place, E22)

日付は「実体の識別と文脈化に寄与する年代情報」。なぜ属性ではなく実体なのかという当然の疑問には、「年代の記述は本質的に複雑だから」と答えられている (RiC-CM §2.2.6)。自然言語表記 (「元禄8年」) と機械可読な標準表記 (ISO 8601、EDTF) の併存、確実性や精度の度合い—これらを担うには、値1つの属性では足りない。場所は「境界を持つ、名前のついた地理的領域」。管轄区域 (jurisdiction)、人工構造物、自然地形を含み、歴史的実体として始点・終点・境界の変遷を持つとされる。座標への参照も可能である。

4.3 属性

属性 (attribute) は実体の特性である。RiC-CM 1.0 は42の属性 (RiC-A01～A45、欠番あり) をアルファベット順に定義し (§3.6)、続く第4章でどの実体にどの属性が使えるかを一覧化する。名称、識別子、一般記述 (General Description)、歴史 (History)、言語、法的地位、記録資源の範囲 (Extent)、範囲と内容 (Scope and Content) など、ISAD(G) の記述要素の多くはここに対応物を見出せる。

属性を読むときの要点は3つある。

■(1) 値スキーマは「指示的」であって規範ではない 各属性には値スキーマ (Value Schema) として、自由テキスト、モデル準拠テキスト、統制値、規則準拠値の4種のいずれかが示される (§3.5)。ただし原文は「値スキーマは指示的 (indicative) なものであり、規範 (prescriptive) でも禁止 (proscriptive) でもない」と明言する。ここでも RiC は内容規則を定めない。何をどの統制語彙で書くかは実装の決定事項である。

■(2) 属性か実体かは実装で入れ替わる 概念的に「統制値」をとる属性 (活動種別、記録集合種別、職業種別など) は、実装では実体 (クラス) として扱うほうが便利なのが多い、と RiC-CM 自身が述べる (§3.3)。共有語彙 (シソーラス) を作れば複数の記述から参照でき、LOD 環境では外部の語彙と接続できるからである。実際 RiC-O はこれらを *Type クラスとして実体化した (第5章)。「CM で属性だったものが O ではクラスになっている」という対応関係は、両文書を往復するときの最重要の読み替え規則である。

■(3) 記録集合の属性には2種類の意味がある 第3章で述べた区別がここで形になる。記録集合に「言語: 日本語」と書いたとき、それは集合それ自体の属性ではなく、**メンバーである記録たちの属性の要約**である。RiC-CM §3.4 はこの問題を正面から論じ、厳密には「集合はすべてのメンバーが属性 X を持つ」「集合は一部のメンバーが属性 X を持つ」と関係を分化させるべきだが、ERM の枠内で属性を倍増させるのは煩雑なので、CM では * 印 (メンバーの記述である属性) と + 印 (集合とメンバー双方の記述でありうる属性) を付けるにとどめた、と説明する。RiC-O はこれを `rico:hasOrHadAllMembersWithLanguage` / `rico:hasOrHadSomeMembersWithLanguage` のようなプロパティ群として厳密に実装した。曖昧さの元だった日常語法 (「このシリーズは閲覧制限あり」) を、機械可読なレベルで解消する仕組みである。

4.4 関係

関係 (relation) は実体同士のつながりであり、RiC の「Contexts」を実際に担う装置である。RiC-CM 1.0 は 85 の関係を定義する (§5.4)。ID は RiC-R001 から RiC-R086 まで振られ、R043 は欠番である。以下では実体の ID (RiC-E01 など) と同様に、接頭辞を省いて R026 のように書く。ほとんどの関係には逆向きの関係 (Rnnni) が対で用意されており、逆向きを別に数えれば総数はさらに増える。

実体の階層 (図 4.4) は「どんな種類のものがあるか」を示すだけで、それらがどう結ばれるかは何も語らない。関係の体系がその部分を受け持つ。以下、13 の型による分類 (§4.4.1)、実体どうしを結ぶ実際の配線 (§4.4.2)、関係自身の階層 (§4.4.3) の順に見ていく。

原典で関係を数えるときの注意

§5.6 「List of Relations」の一覧表は R079 で終わっており、日付関係の R080~R086 (「~の作成日である」など) が載っていない。これらは §5.3 の階層チャートにも §5.4 の個別定義にも現れる。一覧表だけを見て数えると7つ足りなくなる。

4.4.1 13 の型

RiC-CM はすべての関係を 13 の概念的な型 (types of relations) に分類する (§5.2)。1 つの関係が複数の型に属することもある。表 4.2 に、13 の型と、各型でもっとも広い関係を、定義域 (出発点になれる実体) と値域 (到達点になれる実体) つきで挙げる。

本書では以降、関係を日本語の短い名前で呼ぶ。原文の名称と対照できるよう、表 4.2 には原文名を併記し、全 85 関係の対照表を付録 A.3 節に置いた。本文と図表で使う訳語は、この対照表と一致させてある。型の名称の原語は付録 A.2 節にある。

表 4.2 関係の 13 の型と、各型の最上位の関係 (RiC-CM §5.2・§5.3・§5.4 に基づく。原文の has or had・is or was は「現在または過去において」の含意で、訳では省いた)

型	最上位の関係 (訳)	原文名	定義域	値域
全体一部分	R002 部分を持つ	has or had part	もの	もの
順序	R008 先行する	precedes or preceded	もの	もの
主題	R019 主題を持つ	has or had subject	記録資源	もの
記録資源間	R022 記録資源として関連する	is record resource associated with record resource	記録資源	記録資源
記録資源-具現化	R025 具現化を持つ	has or had instantiation	記録資源	具現化
出所	R026 出所を持つ R033 文書化している	has provenance documents	記録資源・具現化	エージェント活動
具現化間	R034 具現化として関連する	is instantiation associated with instantiation	具現化	具現化
管理	R036 に権限を持つ	has or had authority over	エージェント	もの
エージェント間	R044 エージェントとして関連する	is agent associated with agent	エージェント	エージェント
イベント	R057 イベントとして関連する	is event associated with	イベント	もの
規則	R062 規則として関連する	is rule associated with	規則	もの
日付	R068 日付として関連する	is date associated with	日付	もの
空間	R074 場所として関連する	is place associated with	場所	もの

値域の列を縦に見ると、13 のうち 6 つが「もの」になっている。「もの」はすべての実体の上位だから (§4.2)、これらの関係は到達点をどの実体にとってもよい。残りは定義域も値域も特定の実体に限られる。つまり関係の体系は、実体を特定して配線する関係と、相手を限定しない汎用の関係の 2 種類でできている。前者が記録の生成をたどる骨格をつくり、後者が時間・場所・規則・主題といった文脈を任意の実体に添える役を果たす。

4.4.2 実体間の関係マップ

型の一覧だけでは、どの実体からどの実体へ矢印が伸びるかが見えにくい。特定の実体を結ぶ関係を図 4.6 に、相手を限定しない関係を図 4.7 に描き分ける。

図 4.6 は、第 3 章で見た RiC の世界観をそのまま配線図にしたものである。エージェントが活動を遂

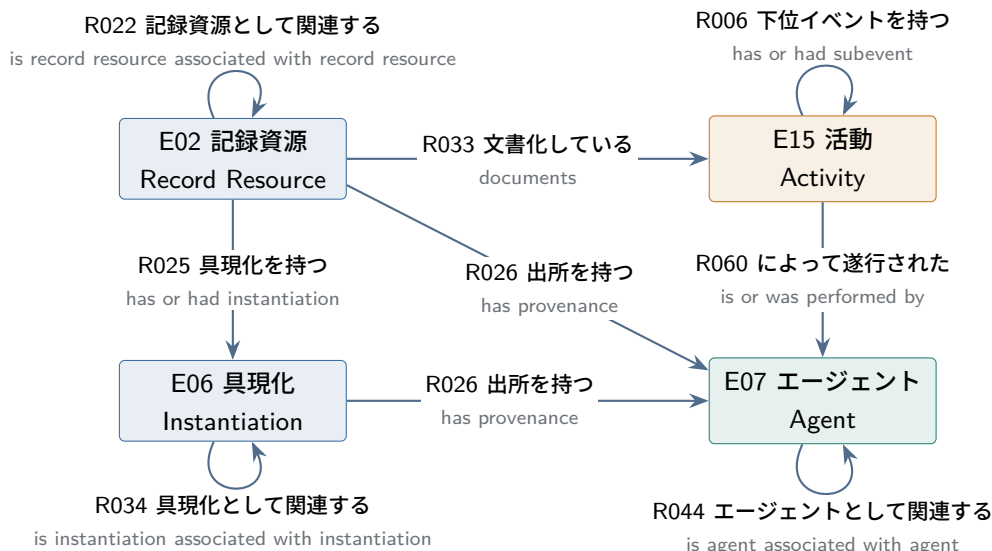


図 4.6 中核実体を結ぶ関係 (RiC-CM §5.3・§5.4 に基づく)。矢印は定義域から値域へ向かう。同じ箱に戻る矢印は、その実体同士を結ぶ関係 (記録資源と記録資源、エージェントとエージェントなど) を表す。逆向きの関係も対で定義されているので、実装ではどちらの向きからも記述できる。関係名の原文は付録 A.3 節の対照表にある。

行し (R060)、その活動が記録資源として記録され (R033)、記録資源はエージェントを出所として持ち (R026)、担体の上では具現化として存在する (R025)。この 5 本を押さえれば、記録の生成をたどる経路は引ける。残りは同種の実体同士をつなぐ関係で、「この記録集合はあの記録集合と関連する」(R022) 「この職員はあの職員の部下である」(R044 の下位) のように、同じ種類のもののあいだの横のつながりを担う。

図 4.7 を読むときの要点は矢印の向きである。日常語では「記録に日付を付ける」と言うが、RiC-CM の R068 は日付を定義域、ものを値域にとり、「日付が～に関連する」と定義されている。イベント・規則・場所も同じで、文脈を担う実体の側が出発点になっている。ただし RiC-CM は各関係に逆向きの関係を用意しており (R068i)、RiC-O では `rico:isDateAssociatedWith`(日付→もの) と `rico:isAssociatedWithDate`(もの→日付) の両方がプロパティとして定義されている。記述の現場では書きやすい向きを選べばよく、推論器が逆向きを補う (第 6 章 6.3 節)。図の向きは、原典の定義域・値域の表記に合わせたものと理解しておけばよい。

4.4.3 関係自身の階層

関係は平らなリストではなく、広い関係から狭い関係へと階層をなす。頂点は「ものはものに関係する (R001 is related to)」で、そこから 13 の型の最上位関係 (表 4.2 の第 2 列) へ枝分かれし、さらに最大 5 階層まで細分される (§5.3)。この階層はポリヒエラルキーで、1 つの関係が複数の親を持ち、チャートの複数の場所に現れる。原典のチャートで同じ関係に “(see also above/below)” と注記されているのはそのためである。

出所の型を例にとると、階層は図 4.8 のようになっている。

この階層構造は、記述と検索の両方を支える仕掛けである。狭い関係 (「この書簡は山田花子を送信者として持つ」 R031) が成り立てば、その上位の広い関係 (「出所を持つ」 R026、「に関係する」 R001)

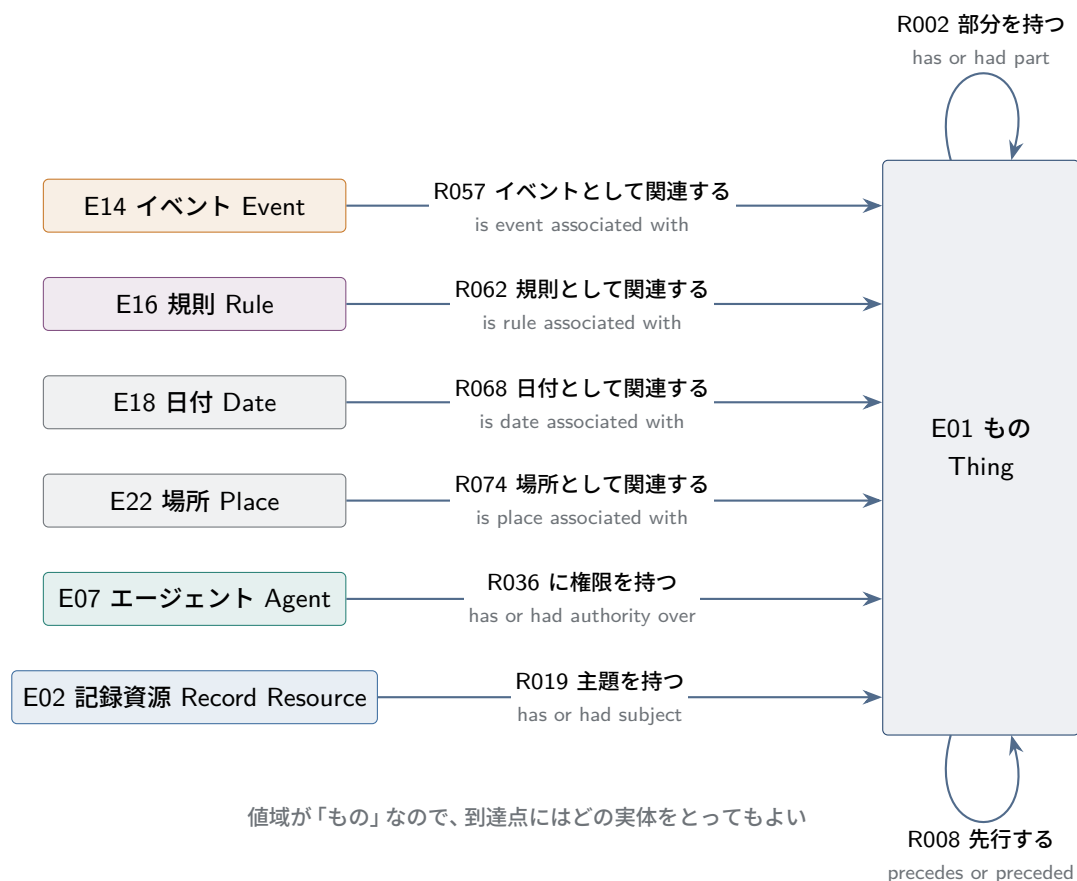


図 4.7 値域が「もの」である関係 (RiC-CM §5.3 に基づく)。イベント・規則・日付・場所は、記録資源にもエージェントにも活動にも、そして互いにも結びつけられる。「もの」に戻る 2 本の矢印は、全体-部分と順序が実体の種類を問わずに成り立つことを表す。関係名の原文は付録 A.3 節の対照表にある。

も自動的に成り立つ。記述者は分かる範囲で最も狭い関係を選べばよく、検索者は広い関係で横断的に問い合わせられる。粒度の自由 (第 3 章) を支えるのはこの設計であり、RiC-O ではこれが下位プロパティ宣言として形式化され、推論器が上位関係を補う (第 6 章 6.4 節)。

4.4.4 関係そのものに属性がある

RiC-CM §5.5 は、関係そのものに付く属性を定義する (RA01 確実性、RA02 日付、RA03 記述、RA04 識別子、RA05 出典、RA06 場所の 6 つ)。「A が B を作成した」という関係に対して、「いつ」「どのくらい確実か」「典拠は何か」を付記できるということである。たとえば作成関係に「日付:1873～1875 年頃」「確実性: 不確実」「出典: 表紙の書き込み」を添える。ISAAR(CPF) が関係の記述に持っていた発想を一般化したものだが、この「関係の属性」は RDF で素直に表現できないため、RiC-O では関係の実体化 (reification) という技法で実装される (第 5 章 5.5.3)。

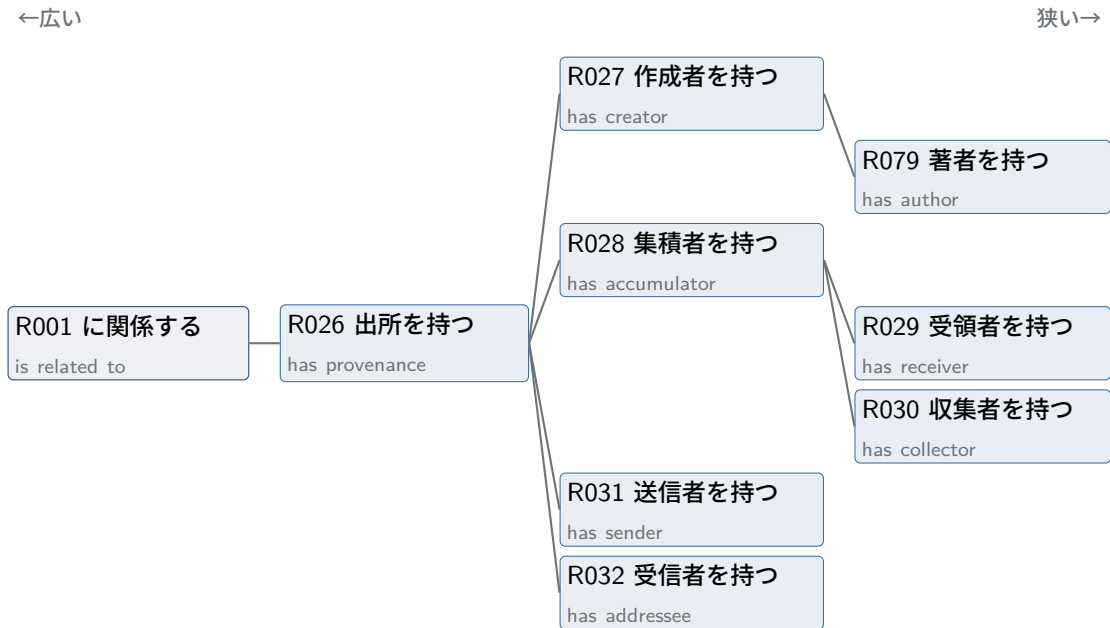


図 4.8 出所の型に属する関係の階層 (RiC-CM §5.4 の Broader/Narrower relations 欄に基づく)。各箱の下段が原文の関係名である。線は「～より狭い関係である」を表す。R079 だけは定義域が記録に、値域が個人・集団・職位に限られ、他は定義域が記録資源・具現化、値域がエージェント全体である。

4.4.5 読み方の実際

85 の関係の各定義 (§5.4) は、ID、名称、定義域 (domain: 関係の出発点になれる実体)、値域 (range: 到達点になれる実体)、多重度、定義、スコープノート、例、関係の型、上位関係、下位関係からなる。たとえば出所の型の最上位にある R026 は、名称が has provenance、定義域が「記録資源または具現化」、値域が「エージェント」、型が出所、上位関係が R001、下位関係が R027・R028・R031・R032 である。定義域と値域という言葉は RDF/OWL でもそのまま使われるので、ここで慣れておくとよい。

CM と O を往復するとき、名称のずれに注意が要る。R026 に対応する RiC-O のプロパティは `rico:hasOrganicProvenance(has organic provenance)` で、`organic`(有機的) が加わっている。RiC-O は各プロパティに `rico:RiCCMCorrespondingComponent` という注釈で対応する CM の関係 ID を記録しているので、名前ではなく ID で突き合わせるのが確実である。

4.5 記述を記述する

RiC-CM 第 6 章は、他の標準にはあまり見られない独特の章である。主題は「記述という行為そのものの記述 (documenting description)」。

記述レコードもまた記録であり、エージェント (文書館、アーキビスト、変換プログラム) が活動 (記述) を遂行して作成したものだから、RiC の実体と関係でそのまま記述できる—専用の語彙は要らない、というのが主張である。

原文は 4 つの層を「地図のズームイン」にたとえる (§6.1)。所蔵機関 (どんなマンデートと規則の下で記録を保存・記述しているか)、職位 (どの担当がその作業を行うか)、記述レコード (この目録データ

自体がいつ誰により作成・改訂されたか)、そして記述内の個々の言明の検証 (この生没年はどの史料に基づくか) である。

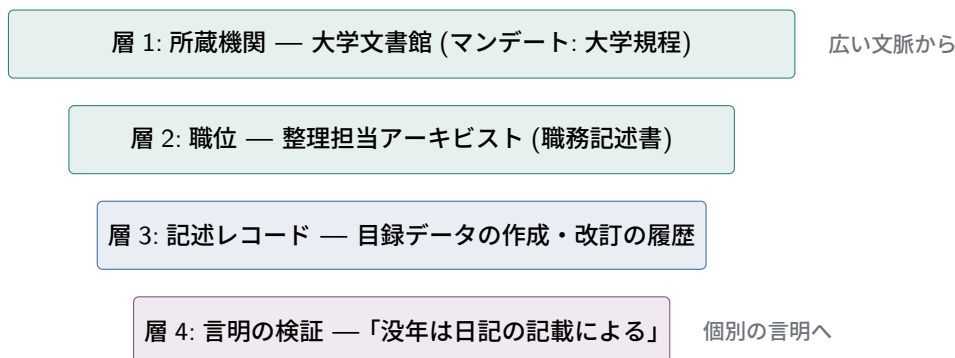


図 4.9 記述の記述の 4 層 (RiC-CM §6.1)。それぞれの層が、より特定の層の文脈になる。

この章は、メタデータの伝統的な二分法に対する RiC の回答でもある。従来の実務では、**記述メタデータ** (記録の内容・文脈を利用者の発見のために記述するもの) と **管理メタデータ** (その記述自体を誰がいつどの規則で作り、いつ改訂したか) を区別し、後者には専用の置き場が用意されてきた。ISAD(G) の第 7 エリア (記述コントロール: アーキビストノート、規則・慣行、記述作成日)、EAD の eadheader/control、EAC-CPF の control である。つまり 2 種類のメタデータは、**同じ文書の別区画**に住んでいた。

RiC はこの区別を、区画の違いではなく**階の違い**として扱う。管理メタデータとは「記述レコードという記録についての、記述メタデータ」である。だから専用の語彙は要らず、同じ実体と関係を一段上に適用すればよい。記述したアーキビストは `rico:Person` (歴史上の人物とまったく同じクラス)、準拠した目録規則や適用プロファイルは `rico:Rule`、記述という仕事は `rico:Activity`、作成・改訂の時期は `rico>Date` である。具体的に書いてみる (図 4.10)。

```

# 記述レコード: この目録データそのものを1個の記録として立てる
ex:desc83 a rico:Record ;
    rico:describesOrDescribed ex:fondsP83 ; # 何を記述したものか
    rico:hasCreator ex:sato ; # 記述したアーキビスト
    rico:isOrWasRegulatedBy ex:profile2026 ; # 準拠した規則
    rico:hasOrHadLanguage [ a rico:Language ;
        rico:name "日本語" ] ;
    rico:isAssociatedWithDate [ a rico>Date ;
        rico:normalizedDateValue "2026-07" ] .

ex:sato a rico:Person ; # 記述者も Person 実体
    rico:name "佐藤一郎" ;
    rico:occupiesOrOccupied ex:pos1 . # 職位(層2)への接続

ex:pos1 a rico:Position ;
    rico:name "整理担当アーキビスト" ;
    rico:existsOrExistedIn ex:archives . # 機関(層1)への接続

ex:archives a rico:CorporateBody ;
  
```

```
rico:name "〇〇文書館" .

ex:profile2026 a rico:Rule ;
  rico:name "〇〇文書館 記述適用プロファイル 2026年版" .
```

記述の階 (いわゆる管理メタデータ)

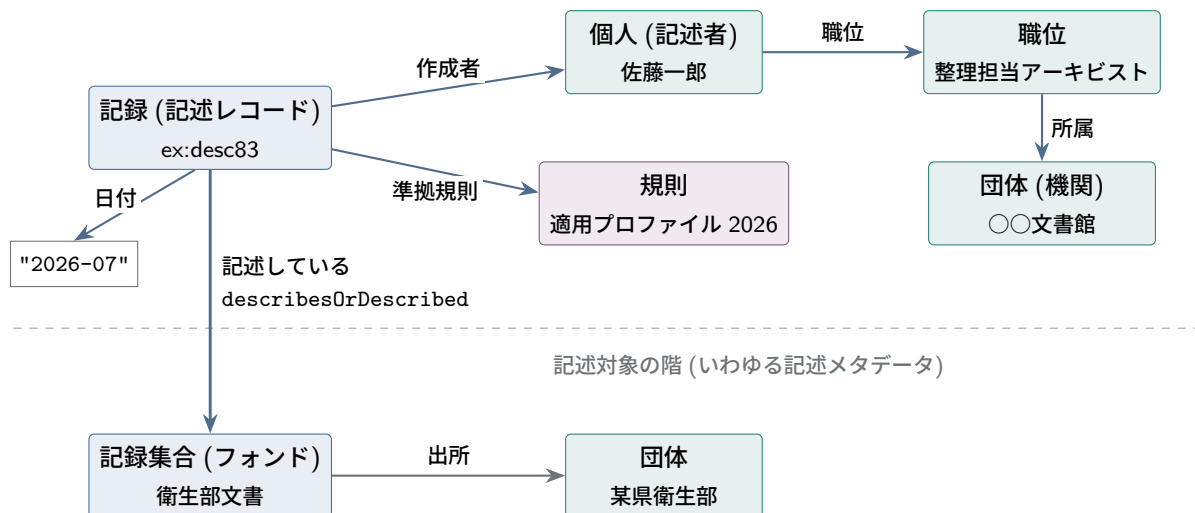


図 4.10 記述の記述の仕組み。目録データそのもの (記述レコード) が一段上の階の記録になり、記述者・規則・日付はその属性・関係として付く。2つの階は `rico:describesOrDescribed` という通常の関係でつながる。記述者・職位・機関は、記述対象の階の実体とまったく同じクラスである。

要点は2つある。第一に、`rico:describesOrDescribed` という1本の関係が、記述レコード (desc83) と記述対象 (fondsP83) という階の違う2つの記録をつなぐ。ISAD(G) 第7エリアに書かれていた内容は、すべて desc83 の側の属性・関係として表現される。第二に、**記述者は特別扱いされない**。アーキビスト佐藤一郎は、記録を作成した歴史上の行為者たちと同じ `rico:Person` であり、職位 (`rico:Position`) を介して機関 (層1、`rico:CorporateBody`) につながる。これは語彙の節約であると同時に、思想の表明でもある。記述もまた歴史的な行為であり、記述者もいずれ記録の歴史の登場人物になる—アーキビストの介入自体が出所の一部だ、という現代のアーカイブズ学の主張が、モデルの形でそのまま実装されている。

なお実務では、記述レコードをここまで実体化する代わりに、名前付きグラフ (第7章 7.3.4 節) の単位で作成者・日付・準拠規則を記録する近道もよく使われる。粒度は粗いが安価で、両者は排他的でない—グラフ単位の管理情報から始めて、必要な部分だけ記述レコードに精密化する、という段階的な進め方が現実的である。

この章が意味するところは大きい。記述の作成者・根拠・改訂履歴が構造化されて残るなら、目録は「無署名の権威」であることをやめ、検証可能な言明の集まりになる。誰がいつどんな根拠でそう記述したのかを利用者が確かめられることは、第3章で見た倫理的批判 (記述には視点があり、選択がある) への実践的な応答でもある。ただし、すべての言明に出典を付ける記述は高コストであり、どこまでやるかはやはり実装の判断である。

確認問題 (ヒントは付録 A.5)

- 問 4.1** 記録集合に記録が「属する」ことと、記録が記録資源の「一種である」ことの違いを、本章の文集のたとえを使って説明せよ。
- 問 4.2** 身近な出来事 (たとえば授業のレポート提出) を、RiC-CM の実体 (記録・エージェント・活動・日付) と関係で図示せよ。
- 問 4.3** ある目録データそのものを記述レコードとして実体化すると、何ができるようになるか。2つ挙げよ。
- 問 4.4** 図 4.10 で、アーキビストと歴史上の人物が同じクラス (Person) であることには、どんな思想的な意味があるか。

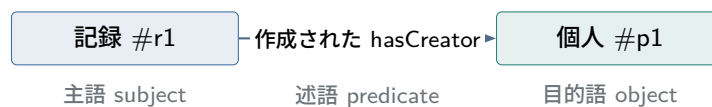
第 5 章

RiC-O — オントロジーという実装

本章では RiC-O を扱う。前半は前提知識 (RDF とオントロジー) の入門、後半は RiC-O 本体の設計の読解である。

5.1 RDF 入門 — 3 語の文でできた世界

RDF(Resource Description Framework) は、W3C が標準化したデータ表現の枠組みである。発想は単純で、あらゆる情報を「主語・述語・目的語」の 3 つ組 (トリプル) の集まりとして表す。



トリプルを何本も集めると、ノード (主語・目的語) がアーク (述語) で結ばれたグラフができる。第 2 章の図 2.2 右側は、実はトリプルの束である。RDF の重要な約束事は次の 3 つだけである。

1. **ものには IRI で名前を付ける。** IRI(Internationalized Resource Identifier) は、世界中で 1 つに定まる「名前」である。見た目はウェブのアドレス (URL) と同じ形だが、それには理由がある。ウェブのドメイン名 (ica.org や example.jp) は登録制で、同じものは世界に 2 つ存在しない。だから「自分の持つドメインで始まる名前を付ければ、世界の誰の名前とも衝突しない」ことが保証される。IRI は、この**一意性の仕組み**を名前の付け方として借用したものである。たとえば <https://archives.example.jp/records/r1> は「example.jp を持つ機関が付けた、記録 r1 の名前」であり、他機関が付けるどの名前ともぶつからない。注意してほしいのは、IRI はあくまで名前であって、「ブラウザで開けるページ」を約束するものではないことだ (RiC-O の IRI を開くと解説文書が表示されるが、これは丁寧な運用であって、名前として機能するための条件ではない)。世界規模で一意的な名前を使うからこそ、異なる機関のデータを機械的につなげられる。
2. **値はリテラルで書ける。** 目的語の位置には、別のものへの参照 (IRI) のほかに、**リテラル (literal)** を置ける。リテラルとは「調査旅行日誌」「1868」「2026-07-31」のような、**それ以上たどる先を持たない生の値**である。「#r1 — 名称 → “調査旅行日誌”」のように書く。IRI との違いは、たどれるかどうかにある。IRI は「その名前と呼ばれるもの」を指し、その先にさらに記述が続く。リテラルはそこで行き止まりで、文字列や数値としてそのまま読むだけである。グラフの絵で言えば、IRI は矢印が出ていける丸、リテラルは矢印の受け手にしかなれない終端の箱に当

たる。だから「作成者」を IRI で書けばその人物の生没年もたどれるが、文字列で書けば名前が読めるだけになる。この選択が第 3 章 3.6 節の「簡便法か精密か」であり、第 4 章 4.1.2 節の設計 A と設計 B の違いでもある。

3. 述語も IRI である。「作成された」という関係自体にも世界規模の名前が付く。この述語の語彙集を提供するのがオントロジーであり、RiC-O の仕事である。

コラム: トリプルは「積み上げ型」の表である

表計算やデータ分析に馴染みがあれば、トリプルという発想は表の持ち方の転換として理解できる。成績のデータを例にする。ふつうに思い浮かべるのは左の形、行が生徒、列が科目の表だろう。同じ情報は、右の形でも持てる。

列展開型 (unstacked・横持ち)			積み上げ型 (stacked・縦持ち)		
氏名	数学	国語	氏名	科目	得点
佐藤	40	88	佐藤	数学	40
鈴木	98	70	佐藤	国語	88
			鈴木	数学	98
			鈴木	国語	70

右の積み上げ型は、1 行が「誰の・何の・いくつ」という 3 つ組である。ここまで来れば気づくだろう。トリプルの集まりとは、列を「主語・述語・目的語」と読み替えた、積み上げ型の表にほかならない。氏名が主語、科目が述語、得点が目的語である。トリプルストアの内部が実際にこの 3 列の巨大な表であることは、第 6 章 6.8 節で見る。

積み上げ型の利点は、そのまま RDF の利点である。第一に、新しい科目が増えても列を足さなくてよい。行を足すだけである。RDF が、スキーマ変更なしに新しい述語 (語彙) を受け入れられるのはこの性質による。第二に、空欄が存在しない。鈴木が英語を受けていなければ、その行を書かないだけで、列展開型のような空セル (NULL) は生じない。「書かれていないことは不明」という開世界の解釈 (5.2 節) は、この持ち方の自然な帰結である。第三に、個体ごとに持つ属性がばらばらでも平気である。記録には記録の、人物には人物の述語だけが並べばよく、全個体に共通の列を設計する必要がない—多様な実体が混在するアーカイブズ記述に都合がよい。

逆方向も見ておく。見慣れた列展開型の表は、積み上げ型からピボット (pivot・unstack) という操作で生成できる。つまり従来型の一覧表は、トリプルの集まりから作る出力の 1 つである—第 3 章で見た「入力と出力の分離」の、最小の実例がここにある。なおこの持ち方は、データ分析では wide/long 形式、データベース実務では横持ち・縦持ち、設計パターンの名では EAV (Entity-Attribute-Value) と呼ばれてきた、実績のある技法の親戚である。

RDF はモデル (考え方) であって、書き方 (構文) は複数ある。本書では人間に読みやすい Turtle 記法を使う。フォンドとその中の 1 記録を Turtle で書くところなる。

```
PREFIX rico: <https://www.ica.org/standards/RiC/ontology#>
PREFIX rst:  <https://www.ica.org/standards/RiC/vocabularies/recordSetTypes#>
PREFIX ex:   <https://archives.example.jp/>

ex:fonds01 a rico:RecordSet ;
```

「a」は「～のインスタンスである」

```

rico:hasRecordSetType rst:Fonds ;           # 種別:フォンド(同梱の個体)
rico:name "山田家文書" ;
rico:includesOrIncluded ex:r1 .             # 記録r1を含む

ex:r1 a rico:Record ;
rico:name "調査旅行日誌" ;
rico:hasCreator ex:p1 ;
rico:hasOrHadDigitalInstantiation ex:r1scan . # デジタル具現化

ex:r1scan a rico:Instantiation .           # スキャン画像(TIFF)

ex:p1 a rico:Person ;
rico:name "山田花子" .

```

記録 `ex:r1` は情報内容としての日誌であり、そのスキャン画像は別の個体 (具現化) として現れる——第3章で見た区別が、データの形にそのまま写っていることを確かめてほしい。Turtle の文法規則は少ないので、次のコラムにまとめて示す。ここにある規則だけで、本書の例も RiC-O の公式サンプルもおおむね読める。

コラム:Turtle の文法 — これだけ知れば読める

文とピリオド: Turtle の 1 文は「主語 述語 目的語」の 3 語で、ピリオド `.` で終わる。`ex:r1 rico:hasCreator ex:p1 .` は「`r1` の作成者は `p1` である」という 1 つの文である。文の切れ目を決めるのはこのピリオドだけで、改行ではない。

空白と改行: 語と語の区切りには、半角スペース・タブ・改行のどれを何個使ってもよい。処理系にとってはすべて同じ「1 つ以上の区切り」である。したがって次の 3 つは、まったく同じ 1 文である。

```

ex:r1 rico:hasCreator ex:p1 .
ex:r1   rico:hasCreator   ex:p1 .
ex:r1
    rico:hasCreator
        ex:p1 .

```

つまり字下げや桁揃えは、人間が読みやすいように付けるだけの飾りである。Python のように字下げが意味を持つことはなく、XML のように閉じタグで範囲を示すこともない。範囲を決めるのは区切り記号 (ピリオド、セミコロン、コンマ、角括弧) だけである。本書の例で述語の位置を揃えているのも、読みやすさのためにすぎない。

逆に、**語の内部に区切りを入れてはいけない**。`rico:hasCreator` のように接頭辞の後ろで改行や空白を入れると別の意味になってしまう。長い IRI が行からはみ出すときは、途中で折り返さず、行そのものを長くするか、接頭辞を使って短く書く。

ピリオドは直前の語にくっつけて `ex:p1.` と書いても通るが、`ex:p1 .` と 1 つ空けるのが慣習で、そのほうが誤読も誤記も減る。

名前空間: IRI を見ると、<https://www.ica.org/standards/RiC/ontology#Record> のように、前半と後半に分かれている。後半の `Record` が個々の名前で、前半の

`https://www.ica.org/standards/RiC/ontology#` の部分を**名前空間** (namespace) と呼ぶ。「どの機関の、どの語彙集に属する名前か」を表す部分である。

名前空間が要る理由は、名前の衝突を避けることにある。`Record` という語は、アーカイブズでは「記録」、データベースでは「表の 1 行」、音楽ではレコード盤を指す。もし短い名前だけで済ませれば、別の分野の `Record` と区別がつかない。名前空間を前に付ければ、「ICA の RiC オントロジーの `Record`」と「別の語彙の `Record`」は違う IRI になり、同じデータの中に混ざっても取り違えられない。日本語の姓名で同姓同名を所属で区別するのに似ている。名前空間にドメイン名を使うのは、§5.1 節で述べたとおり、ドメインの登録制度が一意性を保証してくれるからである。

名前空間の末尾の #:RiC-O の名前空間は `.../ontology#` と、シャープ記号で終わる。この記号はウェブの URL で「文書内の位置」を指す**フラグメント** (fragment) の区切りで、RDF ではこれを名前空間と個々の名前の境目に流用している。`rico:Record` を展開すると `.../ontology#Record` になり、# の前が語彙集、後が語彙集の中の 1 語という読み方になる。この書き方には理由がある。HTTP はフラグメントより後ろをサーバに送らない決まりなので、`.../ontology#Record` を取りに行くと、実際に取得されるのは `.../ontology` という 1 つのファイルである。つまり**語彙全体が 1 回の通信でまとめて手に入る**。まとまりが小さく、要素が一緒に更新されていく語彙—まさにオントロジーがそうである—にはこの方式が向く (W3C の指針 *Cool URIs for the Semantic Web*)。

区切りに `/` を使う流儀もある。本書の例で個々の記録に `https://archives.example.jp/records/r1` という IRI を与えているのがそれで、この場合は名前ごとに別々に取得できる。何万件もの記録を公開する側では、1 件だけ見たい利用者に全件を渡すわけにいかないで、`/` が向く。語彙は #、実データは `/` という使い分けは、この違いから来ている。

接頭辞: 名前空間は長く、しかも同じデータの中で何度も繰り返される。そこで Turtle では、冒頭の PREFIX 宣言で「この長い前半を、以後は短い呼び名で略記する」と約束する。辞書の凡例が「【日国】=『日本国語大辞典』」と定めるのと同じ仕組みである。上の例の 1 行目の宣言により、以後 `rico:Record` と書けば `<https://www.ica.org/standards/RiC/ontology#Record>` と書いたのとまったく同じ意味になる。つまり `rico:` は「ICA の RiC オントロジーという語彙集の中の」という所属の表示であり、`ex:` は本書の例のために仮に立てた架空機関の名前空間である。略さずに書くときは、IRI 全体を山括弧 `<>` で囲む。宣言の書き方には 2 通りある。PREFIX `rico: <...>` という形 (ピリオドは不要) と、`@prefix rico: <...> .` という形 (末尾にピリオドが要る) である。前者は SPARQL に合わせて Turtle 1.1 で追加されたもので、後者が古くからの書き方である。意味は同じで、公開データではどちらも見かける。ピリオドの有無だけが違うので、書き写すときはそこに注意する。

述語 `a:ex:rel01 a rico:OrganicProvenanceRelation` のような文に現れる `a` は、それ自体が 1 つの述語で、「~というクラスのインスタンスである」を意味する `rdf:type` の略記である (英語の *is a* に由来する)。つまりこの文は「`rel01` は有機的出所関係 (というクラス) のインスタンスである」と読む。なお `a` を他の述語と同じ調子で「2 つのものの関係」と読もうとすると、目的語がものではなく**クラス**だという点で少しねじれる。このねじれは 5.2 節で論理と突き合わせる時に改めて扱う。

セミコロンとコンマ: 同じ主語について複数の文を書くときは、主語を繰り返す代わりにセミコ

ロン ; で「同じ主語のまま、次の述語へ」と続ける。さらに、述語まで同じで目的語だけが複数あるときは、コンマ , で目的語を並べる。次の3通りはまったく同じ意味である。

```
# (1) 全部を別の文で書く
ex:f rico:includesTransitive ex:a .
ex:f rico:includesTransitive ex:b .
ex:f rico:name "フォンドF" .

# (2) セミコロンで主語を共有する
ex:f rico:includesTransitive ex:a ;
    rico:includesTransitive ex:b ;
    rico:name "フォンドF" .

# (3) さらにコンマで述語も共有する
ex:f rico:includesTransitive ex:a , ex:b ;
    rico:name "フォンドF" .
```

リテラル: 文字列は "... " で囲む。言語タグを "名主"@ja のように付けられ、"1868"^^xsd:gYear のようにデータ型を付けることもできる。引用符の中では、空白も改行も**意味のある文字**として扱われる点が、引用符の外との大きな違いである。ただし "... " の中に生の改行は書けないので、改行を含む文章を書きたいときは三重引用符 "..." を使う。引用符そのものを値に含めたいときは \" と書く。

空白ノード: 角括弧 [...] は「名前 (IRI) を付けない、その場限りの個体」を作る。括弧の中身は、その個体を主語とする「述語 目的語」の並びである。

```
ex:r1 rico:isAssociatedWithDate [ a rico:Date ;
                                rico:expressedDate "慶応4年" ] .
```

これは「r1 はある日付と結びつく。その日付は Date のインスタンスで、表記は慶応4年」と読む。日付や数量のような「わざわざ世界規模の名前を与えるほどではない」補助的な個体に便利で、RiC-O の公式サンプルでも多用される。ただし空白ノードは IRI を持たないため外部から参照できない。長く維持し共有するデータでは、主要な実体には IRI を与えるのが原則である。

コメント: # から行末までは注釈で、データとしては無視される。範囲が行末までなので、複数行にわたる注釈は各行の先頭に # を置く。

文字コードと大文字小文字: ファイルは UTF-8 で書く。日本語をそのまま IRI にもリテラルにも書けるが、IRI に日本語を使うと処理系や公開先で扱いが揺れやすいので、実務では IRI を ASCII に保ち、日本語はリテラルに置くのが安全である。IRI と接頭辞名は大文字小文字を区別する (rico:Record と rico:record は別物)。区別しないのはキーワードだけで、PREFIX と prefix、データ型の true/false などがこれに当たる。

5.1.1 RDF の書き方は 1 つではない — Turtle・JSON-LD・RDF/XML

RDF は抽象的なデータモデルであり、同じトリプルの集合を複数の構文 (直列化形式、serialization) で書き表せる。どの形式で書いても意味は同一で、ツールで機械的に相互変換できる。主なものを挙げる。

Turtle 上で使った形式。名は Terse RDF Triple Language(簡潔な RDF トリプル言語) の略である。最も簡潔で、人間が読み書きするのに向く。本書の例も、RiC-O の公式サンプルの多くも Turtle である。学習・例示・手書きにはこれを使えばよい。

JSON-LD **JSON**(JavaScript Object Notation) の形をした RDF。JSON は「鍵と値の組」を波括弧で囲んで並べ、値の位置にさらに同じ形を入れ子にできるデータ記法である^{*1}。LD は Linked Data(第 1 章の Linked Open Data の LD) を指す。ふつうの JSON との違いは、鍵や値に現れる名前を IRI に対応づける仕組み (下の例の @context) を備え、それによって全体が RDF のトリプルとして読める点にある。ウェブ開発で標準的な JSON と同じ道具 (あらゆるプログラミング言語のライブラリ、Web API) で扱えるため、開発者との接点やシステム間連携ではしばしば第一候補になる。RiC-AG も、変換先の形式として RDF(Turtle 等) と並べて JSON-LD を挙げている。

RDF/XML 最も古い形式。RiC-O のオントロジー本体 (RiC-O_1-1.rdf) はこの形式で配布されている。XML ツールで処理できる利点はあるが、人間が読むには冗長で、新規に選ぶ理由は少ない。

N-Triples 1 行に 1 トリプルを略記なしで書き並べる形式。冗長だが構造が自明なので、大量データの交換や行単位の処理に向く。

見た目の違いを 1 つだけ示す。上の ex:r1 の一部を JSON-LD で書くと、次のようになる。

```
{ "@context": { "rico": "https://www.ica.org/standards/RiC/ontology#" },
  "@id":      "https://archives.example.jp/r1",
  "@type":    "rico:Record",
  "rico:name": "調査旅行日誌" }
```

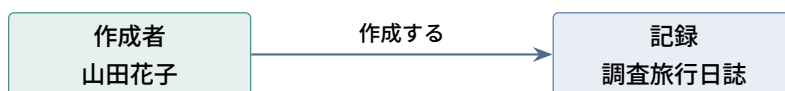
つまり「どれが正しい形式か」という問いは立たず、「誰が (何が) 読むか」で選ぶ。人が読むなら Turtle、ウェブアプリケーションと話すなら JSON-LD、既存の XML 基盤に載せるなら RDF/XML、機械同士の大量転送なら N-Triples、という使い分けが实际的である。本書は以後も Turtle を使う。

^{*1} もとはウェブブラウザ上で動く言語 JavaScript の書き方だったが、いまは言語を問わず、ウェブサービスどうしがデータをやりとりするときの共通形式になっている。XML(第 2 章) と役割は近く、開始タグと終了タグの対ではなく波括弧と鍵で構造を表すぶん、記述量が少ない。アーカイブズの実務にも入っている。米国の記述規則 DACS に基づく記述を扱う管理システム ArchivesSpace は、「記録を JSONModel オブジェクトとして表現しており、これがシステムとやりとりする単位である」と述べ、エージェントの名称には適用した規則を示す "rules": "dacs" という項目が現れる (API 文書、バージョン 4.2.0)。ただしここで使われるのは JSON であって JSON-LD ではない。DACS の標準そのものは出力形式を定めず、本文は Markdown で管理されている。

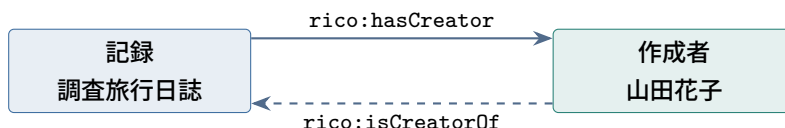
5.1.2 矢印の向きと参照の向き — 図とデータの読み方

本章の冒頭から、本書は関係を矢印で描いてきた。ここで一度立ち止まって、矢印の**向き**が何を意味するのかを正面から扱っておく。作図の流儀の話に見えて、実はデータ設計の中身に直結する。矢印の実体は**参照** (reference)—ある記述の中から別のものを名指すこと—であり、参照はデータ設計の骨格である (その歴史的経緯は第2章 2.4 節で追った)。どちらの側が相手を名指すか、名指しの手段は何か (文字列か、キーか、IRI か)、逆向きにたどれることは保証されるか。この3点はシステムの検索能力と保守性を直接決める。ところが図解では、向きは「なんとなく」で描かれ、読む側も雰囲気ですり流しがちである。同じ1つの事実「山田花子が日誌を作成した」を、3つの流儀で描き分けたのが図 5.1 である。

流儀 1: 現実の行為の向き (出来事・因果を描く)



流儀 2: 名前付きプロパティの向き (主語 → 目的語)



2 つは `owl:inverseOf` で結ばれ、片方を書けばもう片方は推論で得られる

流儀 3: 文書間の参照の向き (どの文書がその記載を持つか)

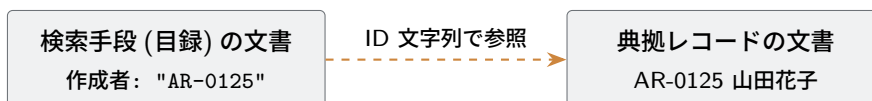


図 5.1 同じ1つの事実「山田花子が日誌を作成した」を描く、3つの矢印の流儀。向きはそれぞれ別の理由で決まっており、互いに逆でも矛盾ではない。

■**流儀 1: 現実の行為の向き** 「作成者が記録を**作成する**」という出来事の向きである。因果や手順を説明する概念図はふつうこの向きをとり、直感にもよく合う。ただしこの矢印は、データの中のどの記載にも直接は対応しない。描かれているのは世界の側の出来事であって、記述の側の構造ではないからである。

■**流儀 2: 名前付きプロパティの向き** RDF のトリプルは主語→目的語の向きを必ず持つ。だからプロパティの矢印は、**そのプロパティ名が定める向き**に描く。`rico:hasCreator` は「記録が作成者を持つ」だから記録→作成者で、流儀 1 と逆になる。ここで思い出してほしいのは、RiC-CM がほぼすべての関係を**両方向の名前**で与えていることである (“Record has creator Agent” と “Agent is creator of Record”)。RiC-O はこれを 419 個の `owl:inverseOf` 宣言として実装しており、どちらの向きで

書いても、もう一方は推論で得られる (第 6 章場面 2)。したがって矢印の向きそのものに正解はなく、ラベルと向きが対応しているかだけが正誤を決める。「作成する」と書いた矢印は作成者から、`rico:hasCreator` と書いた矢印は記録から出ていれば、どちらも正しい。なお、向きに意味がない関係もある。`rico:hasSibling`(きょうだいである) や `rico:hasOrHadSpouse`(配偶者である) など、RiC-O の 16 個のプロパティは**対称プロパティ** (`owl:SymmetricProperty`) として宣言されており、どちら向きに書いても同じ意味になる。

■**流儀 3: 文書間の参照の向き** 検索手段の文書の中に典拠レコードの ID を書き込むとき、参照の矢印は検索手段→典拠である。これは関係の意味の向きではなく、**どのファイルのどの欄にその記載があるか**という置き場所の向きである。文書ベースの記述では、この向きは書き手が選べるものではなく、標準と文書構造が決めてしまう—この点が従来標準と RiC の重要な分かれ目になるので、第 7 章 7.3 節で図とともに詳しく扱う。

■**実務上の帰結** 流儀 2 に関して 1 つ、実務的な注意がある。逆向きが「推論で得られる」のは推論を使う場合の話で、推論を使わないストアでは、**実際に書いた向き**のトリプルしか検索に現れない。SPARQL には述語の前に `~` を付けて逆向きに辿る記法があるので検索側で吸収もできるが、データを受け取る側がそれを知らなければ「作成者が見つからない」事故になる。機関の適用プロファイル (第 7 章) で「この関係はこの向きで書く (または両方書く)」と決めておくのが確実である。

5.2 オントロジーとは何か

オントロジー (ontology) という語は哲学の存在論に由来するが、情報科学では「ある領域の概念 (クラス) と関係 (プロパティ) を、機械が処理できる形で定義した語彙体系」を指す。RDF の上でオントロジーを記述するための言語が RDFS(RDF Schema) と OWL(Web Ontology Language) であり、RiC-O は OWL 2 で書かれている。

オントロジーが定義する主な部品は次のとおりである。

クラス (class) ものの種類。RiC-CM の実体に対応する。`rico:Record`、`rico:Person` など。クラス間には下位クラス関係 (`rdfs:subClassOf`) が定義でき、「`rico:Person` は `rico:Agent` の下位クラス」のように実体階層 (図 4.4) をそのまま表現する。

オブジェクトプロパティ (object property) もの同士を結ぶ述語。RiC-CM の関係に対応する。`rico:hasCreator`、`rico:isOrWasIncludedIn` など。定義域 (domain) と値域 (range)、逆プロパティ (`owl:inverseOf`)、上位プロパティ (`rdfs:subPropertyOf`)、推移性 (transitivity) などの論理的性質を宣言できる。

データタイププロパティ (datatype property) ものとリテラル値を結ぶ述語。RiC-CM の (自由テキスト系の) 属性に対応する。`rico:name`、`rico:scopeAndContent` など。

個体 (individual) クラスのインスタンス。RiC-O は 7 つの著名な個体を同梱している (後述)。

5.2.1 オントロジーの土台 — 記述論理

ここまでは「rico:Person は rico:Agent の下位クラス」のような宣言を日本語で書いてきた。機械に推論をさせるには、この日本語を、読み手によって解釈が変わらない形式言語に置き換えなければならない。その言語が**記述論理** (Description Logic、DL) であり、OWL の意味論はこの体系に基づいている。

以下は、論理式を見るのが初めての読者を想定して、道具を1つずつ組み立てる。本書で記述論理の記法が実際に必要になるのは第6章の補足だけなので、読み飛ばして先へ進んでもさしつかえない。ただし RiC-O の仕様書は公理をこの記法で書くので、読めるようになっておくと同様に直接あたれる。

■登場人物は3種類だけ 記述論理が扱うものは3つに限られる。

個体 (individual) 「甚兵衛」「フォンド 01」のような、名前のついた個々のもの。第4章の言葉では実体インスタンスに当たる。

クラス (class) 個体の**集まり**。Person というクラスは、甚兵衛や花子といった個体を要素とする集合である。

プロパティ (property) 個体2つの組の**集まり**。hasCreator というプロパティは、(日誌, 甚兵衛) のような組の集合である。

クラスを集合と見る点が要である。「rico:Person は rico:Agent の下位クラス」は、集合の言葉では「Person という集合は Agent という集合にすっかり含まれる」となり、記述論理では

$$\text{Person} \sqsubseteq \text{Agent}$$

と書く。 $\sqsubseteq$ は数学の $\subseteq$ (部分集合) に対応する記号で、「左は右に含まれる」と読む。図 5.2 のとおり、包含が成り立っていれば、個人であるものは自動的にエージェントでもある。「甚兵衛は個人である。ゆえに甚兵衛はエージェントである」という推論は、この包含を1歩たどることで得られる。



内側の4つの箱は、どれも外側の Agent にすっぽり収まっている。だから「甚兵衛は Person に属する」と書けば、「甚兵衛は Agent にも属する」は書かなくても成り立つ

図 5.2 クラスを集合と見る。下位クラス関係 $\sqsubseteq$ は集合の包含である。RiC-CM がエージェントの下に4つの実体を置くのは (図 4.4)、集合の言葉では Person・Group・Position・Mechanism の4つが Agent の部分集合だ、と言っているに等しい。

■一階述語論理に翻訳して意味を確かめる 記述論理の記号は簡潔だが、意味を確かめたいときは一階述語論理に翻訳するのが確実である。一階述語論理は、「どの～についても」「～であるものがある」という言い方(量化)と、「かつ」「または」「ならば」を、読み手によって解釈の変わらない形で書き表すための記法である。記述論理はその断片にあたる。使える述語を1項と2項に限り、量化の現れ方も限られた型に制限したもので、記述論理のどの公理も機械的に一階述語論理の式へ翻訳できる。以下、記号を1つずつ、この翻訳と RiC-O の実際の公理を添えて読んでいく。翻訳の記法そのものに馴染みがあれば、次のコラムを先に読んでほしい。

準備: 述語論理の式を読む

述語とは、ものについての主張の型である。引数の数で2種類ある。Person(x) のような1項述語は「 x は個人である」という性質を表し、クラスに対応する。hasCreator(x, y) のような2項述語は「 x は y を作成者として持つ」という関係を表し、プロパティに対応する。引数に実際の個体名を入れた Person(甚兵衛) は、真偽の定まった1つの主張になる。

x, y は変数である。中学校の数学で x が「まだ決まっていない数」を指したのと同じで、ここでは「まだ決まっていない1つの個体」を指す仮の名前にすぎない。名前は何でもよく、 x を u に書き換えても式の意味は変わらない。

変数を含む式は、そのままでは真偽が決まらない。「 x は個人である」は x が誰かを言わないうちは真とも偽とも言えない。そこで量子子で範囲を指定する。

- $\forall x(\dots)$ — 「どの x についても、 $\dots$ 」(全称量化)
- $\exists x(\dots)$ — 「 $\dots$ を満たす x が少なくとも1つある」(存在量化)

主張どうしをつなぐのが論理結合子である。 $\wedge$ は「かつ」、 $\vee$ は「または」、 $\rightarrow$ は「ならば」、 $\leftrightarrow$ は「ならばが両向きに成り立つ(同値)」、 $=$ は「同じものである」。 $\rightarrow$ だけは日常語と少しずれる。 $A \rightarrow B$ は「 A が成り立つ場面ではいつも B も成り立つ」だけを言い、 A が成り立たない場面については何も主張しない。

以上で式は読める。声に出して読む練習をしておく。

式	読み
$\forall x(\text{Person}(x) \rightarrow \text{Agent}(x))$	どの x についても、 x が個人ならば x はエージェントである
$\exists y \text{hasCreator}(\text{r1}, y)$	r1 が作成者として持つ相手 y が、少なくとも1つある
$\forall x \forall y (\text{hasCreator}(x, y) \rightarrow \text{Agent}(y))$	どの x, y についても、 x が y を作成者として持つならば、 y はエージェントである

読み方の型は決まっている。 $\forall$ で始まる式は「どれについても～」、 $\exists$ で始まる式は「～なものがある」。 $\rightarrow$ の左を条件、右を帰結として読めばよい。

なお「一階」とは、量化できるのが個体だけで、述語そのものを量化できないという制限を指す。「すべてのクラス C について…」とは書けない。この制限の見返りが、次に述べる決定可能性である。

表現力を絞る見返りは**決定可能性**である。決定可能とは、どんな入力に対しても有限の手数で必ず答えが出る手続きが存在する、という保証である。一階述語論理の全体では、ある文が別の文から導けるかどうかを必ず判定する手続きは存在しない(1936年の Church と Turing の結果)。記述論理は使える構文を制限することでこの保証を取り戻している。実務の言葉に直せば、推論器が答えを返さずに走り続ける事態を、体系の設計の段階で排除できるということである。第6章 6.8.4 節で見る OWL 2 のプロファイルは、この保証をさらに強め、計算時間の上限まで押さえた部分集合である。

もう1つ、記法の目立つ特徴を先に押さえておく。記述論理の式には**変数が現れない**。 $\text{Person} \sqsubseteq \text{Agent}$ には x がない。これは省略ではなく設計である。記述論理は、1970 年代の意味ネットワークやフレーム(第3章のコラムで触れた第二次 AI ブームの知識表現)を論理として厳密化する研究から生まれ、その際、個体を主役にする代わりに**クラスを集合として直接演算する**代数的な書き方を採った。この書き方には実利がある。公理が変数なしの1行で書いて見通しがよいこと、そして使う演算子の種類を選ぶことで表現力と計算量を細かく調整できることである^{*2}。以下、主な記号を1つずつ、一階述語論理への翻訳と RiC-O の実際の公理を添えて読んでいく。クラスは「個体の集まり」、プロパティは「個体と個体の二項関係」と解釈する。

■**下位関係** $\sqsubseteq$ 最も基本の記号で、「左は右に含まれる」と読む。クラスの公理 $C \sqsubseteq D$ を一階述語論理に翻訳すると

$$\forall x (C(x) \rightarrow D(x))$$

「どの個体 x についても、 C ならば D でもある」となる。RiC-O の実例は $\text{Person} \sqsubseteq \text{Agent}$ (個人はエージェントである)。プロパティの公理 $R \sqsubseteq S$ は変数2つで

$$\forall x \forall y (R(x, y) \rightarrow S(x, y))$$

と翻訳され、実例は $\text{directlyIncludes} \sqsubseteq \text{includesTransitive}$ (直接含むなら、含む)である。

■**逆関係** $-$ と**同値** $\equiv$ 肩に付く $-$ は、関係の向きをひっくり返す演算である。

$$R^-(x, y) \leftrightarrow R(y, x)$$

($\leftrightarrow$ は「どちらからでも導ける」)。hasCreator は「記録 $\rightarrow$ 作成者」という向きの関係だから、hasCreator $^-$ はその逆向き、「作成者 $\rightarrow$ 記録」の関係になる。 $\equiv$ は「左右がまったく同じ意味」で、逆向きの関係に名前を与える定義に使われる。RiC-O の公理 $\text{isCreatorOf} \equiv \text{hasCreator}^-$ の翻訳は

$$\forall x \forall y (\text{isCreatorOf}(x, y) \leftrightarrow \text{hasCreator}(y, x))$$

である。

■**連鎖** $\circ$ $R \circ S \sqsubseteq T$ は「 R で進み、続けて S で進めるなら、 T で一足飛びに行ける」と読み、翻訳は

$$\forall x \forall y \forall z (R(x, y) \wedge S(y, z) \rightarrow T(x, z))$$

である。推移性はこの形の特別な場合 $R \circ R \sqsubseteq R$ で、RiC-O の実例を翻訳すれば

$$\text{includesTransitive}(x, y) \wedge \text{includesTransitive}(y, z) \rightarrow \text{includesTransitive}(x, z)$$

(含むものの中のものは、含まれている)となる。

^{*2} 採用した演算子の頭文字を並べたものが *ALC* や *SR_QIQ* といった体系の名前になっている。OWL 2 DL の意味論は *SR_QIQ* に基づく。

■**クラスを作る演算** $\exists R.C \cdot \forall R.C \cdot \sqcup$ ここからが記述論理らしい部分である。これらの記号は文ではなく、**新しいクラス (個体の集合) を作る演算**である。Person や Agent のようにあらかじめ名前のついたクラスだけでなく、「作成者として個人を持つもの」のような**その場で定義した集合**を式で書けるようにする道具立てである。

$\exists R.C$ は「 R の相手として C に属するものを少なくとも 1 つ持つ個体」の全体を表す。たとえば $\exists \text{hasCreator.Person}$ は「作成者として個人を持つもの」というクラスである。 $\forall R.C$ は「 R の相手があれば、それがすべて C に属する個体」の全体、 $\sqcup$ はクラスの合併 (「または」) である。3 つとも、ある個体 x がその集合に属する条件を一階述語論理で書けば意味がはっきりする。

$$\begin{aligned} x \in \exists R.C &\iff \exists y (R(x, y) \wedge C(y)) \\ x \in \forall R.C &\iff \forall y (R(x, y) \rightarrow C(y)) \\ x \in C \sqcup D &\iff C(x) \vee D(x) \end{aligned}$$

記号の $\exists$ と $\forall$ が、翻訳では相手 y にかかる量子子として現れる。これが記述論理の変数なしの記法と、変数を明示する一階述語論理の記法の接点である。

$\forall R.C$ は、読み方に一点だけ注意が要る。翻訳の右辺は $\rightarrow$ の形をしているので、 R の相手が **1 つもない個体**についても真になる。条件が成り立つ場面が存在しなければ、「そのすべてが C である」という主張は破られようがないからである。つまり $\forall \text{hasCreator.Agent}$ は「作成者が 1 人以上いて、その全員がエージェント」ではなく、「作成者として書かれているものがあれば、それはエージェント」を意味する。「少なくとも 1 つある」ことまで言いたければ $\exists$ と組み合わせる必要がある。

■**T —「すべての個体」のクラス** T (トップと読む) は、**どの個体も無条件に属するクラス**である。所属の条件が常に真のクラス、と考えればよい。それ自体は当たり前なものに見えるが、変数を持たない記述論理では、 T が 2 つの決まり文句の部品として働く。

第一の使い方は $\exists R.T$ 、「**相手は何でもよい**」である。 $\exists R.C$ の C の位置に「何でも属するクラス」を置くことで、相手の種類を問わない「とにかく R の相手を持つ個体」のクラスを作る。 $\exists \text{hasCreator.T}$ は「(誰でもよいから) 作成者を持つもの」である。

第二の使い方は $T \sqsubseteq X$ 、「**すべての個体について X** 」である。 $\sqsubseteq$ は「左は右に含まれる」だから、この式は「すべての個体のクラスが X に含まれる」、すなわち「どの個体も X に属する」という全称の主張になる。一階述語論理の $\forall x(\dots)$ に当たる言い回しを、変数なしで書くための定型である。

この 2 つを使うと、定義域と値域の公理が書ける。RiC-O の定義域公理

$$\exists \text{hasCreator.T} \sqsubseteq \text{RecordResource} \sqcup \text{Instantiation}$$

は「作成者を (誰でもよいから) 持つ個体は、記録資源か具現化である」と読み、一階述語論理では

$$\forall x (\exists y \text{hasCreator}(x, y) \rightarrow \text{RecordResource}(x) \vee \text{Instantiation}(x))$$

となる。値域公理 $T \sqsubseteq \forall \text{hasCreator.Agent}$ は、2 つ目の決まり文句で外側を読み、 $\forall$ で内側を読む。「どの個体についても ($T \sqsubseteq$)、その作成者はあればすべてエージェントである ($\forall \text{hasCreator.Agent}$)」。翻訳は

$$\forall x \forall y (\text{hasCreator}(x, y) \rightarrow \text{Agent}(y))$$

「作成者の側は必ずエージェント」である。見た目は暗号めいているが、 $\exists R.T \sqsubseteq C$ (定義域は C) と $T \sqsubseteq \forall R.C$ (値域は C) は、この形のまま慣用句として覚えてよい。

■**数の制約** $=1$ $R.C$ は「 C に属する R の相手をちょうど1つ持つ個体」の全体である。RiC-O の実例 $\text{Proxy} \sqsubseteq =1 \text{ proxyFor.RecordResource}$ は「Proxy はちょうど1つの記録資源を代理する」。一階述語論理に翻訳するには「少なくとも1つ存在し、2つあれば実は同じ」という等号付きの式が要る:

$$\begin{aligned} \forall x (\text{Proxy}(x) \rightarrow \exists y (\text{proxyFor}(x, y) \wedge \text{RecordResource}(y)) \\ \wedge \forall y \forall z (\text{proxyFor}(x, y) \wedge \text{RecordResource}(y) \\ \wedge \text{proxyFor}(x, z) \wedge \text{RecordResource}(z) \rightarrow y = z)) \end{aligned}$$

一意性を述べる後半にも **RecordResource** の条件が要ることに注意したい。この条件を落とすと「**proxyFor** の相手はどんな種類のものも含めてただ1つ」という別のもっと強い主張になる。数を数える対象をクラスで限定するこの形を**限定数制約** (qualified number restriction) と呼び、*SROIQ* の *Q* がこれを指す。等号を使う分だけ強い装置で、体系の計算量を左右する。

まとめると、 $\sqsubseteq$ の記法は公理を変数なしで短く一覧するのに向き、一階述語論理への翻訳 (規則の形) は意味を確かめるのに向く。以下では両方を場面に応じて使う。ここまでに出了記号を表 5.1 にまとめる。

表 5.1 記述論理の記号早見表 (C, D はクラス、 R, S はプロパティ、 x, y, z は個体を表す変数)

記号	読み	一階述語論理への翻訳
$C \sqsubseteq D$	C は D に含まれる	$\forall x (C(x) \rightarrow D(x))$
$R \sqsubseteq S$	R は S に含まれる	$\forall x \forall y (R(x, y) \rightarrow S(x, y))$
$C \equiv D$	C と D は同じ	$\forall x (C(x) \leftrightarrow D(x))$
R^-	R の逆向き	$R^-(x, y) \leftrightarrow R(y, x)$
$R \circ S \sqsubseteq T$	R から S とたどれるなら T	$\forall x \forall y \forall z (R(x, y) \wedge S(y, z) \rightarrow T(x, z))$
$\exists R.C$	C の相手を1つ以上持つもの	$\exists y (R(x, y) \wedge C(y))$
$\forall R.C$	相手があればすべて C であるもの	$\forall y (R(x, y) \rightarrow C(y))$
$C \sqcup D$	C または D	$C(x) \vee D(x)$
$C \sqcap D$	C かつ D	$C(x) \wedge D(x)$
$\top$	すべての個体のクラス	常に真
$=1 R.C$	C の相手をちょうど1つ持つもの	C の相手が存在し、 C の相手が2つあれば等しい

■**RDF トリプルとの対応** この記法と 5.1 節で学んだトリプルの対応は、2 段に分けると正確に言える。第一に、**データのトリプルは、論理の原子式** (それ以上分解できない事実の文) に対応する。

Turtle	記述論理	一階述語論理
<code>ex:r1 rico:hasCreator ex:p1 .</code>	<code>hasCreator(r1, p1)</code>	<code>hasCreator(r1, p1)</code>
<code>ex:p1 a rico:Person .</code>	<code>Person(p1)</code>	<code>Person(p1)</code>
<code>rico:Person rdfs:subClassOf rico:Agent .</code>	<code>Person $\sqsubseteq$ Agent</code>	$\forall x (\text{Person}(x) \rightarrow \text{Agent}(x))$

上2行が**事実**である。1行目のようなふつうのトリプルは、素直に2項述語の原子式になる。ところが2行目、**a** のトリプルには見過ごせない**ねじれ**がある。1行目と同じ調子で述語化するなら `a(p1, Person)`、

すなわち `type(p1, Person)` という 2 項述語の原子式になるはずである。しかし表では `Person(p1)`、つまり **1 項述語の適用**として読んだ。2 つの読みでは `Person` の立場が違う。前者では**引数**(ものの名前)の位置に、後者では**述語**の位置に現れている。要するに `a` だけは、目的語の位置に述語の**名前**が入るという点で、他の述語と対称でないのである。論理の標準的な読み方は後者 (`Person(p1)`) であり、この読みをとる限り、事実の水準では記述論理と一階述語論理の記法は**同形**である (記述論理の文献には `p1 : Person` と書く流儀もある)。ただし前者の読み方—`type` という 2 項述語—も間違いではない。それが後のコラム「クラスが主語になる書き方」の主題である。3 行目の**公理**で、はじめて 3 欄が別の顔になる。一階述語論理は量化された変数を明示し、記述論理は前項で見たとおり変数なしの 1 行に畳み、Turtle はそれを 1 本のトリプルに符号化する (この符号化については後述)。つまり記述論理らしさは公理の水準に現れ、事実の水準の RDF グラフとは、原子式の集合—事実の集まり—にほかならない。記述論理はこの事実の集まりを **ABox**(assertion box) と呼ぶ。

第二に、**公理の側もトリプルである**。これらの公理は、`RiC-O` のファイルの中では**それ自体が RDF トリプル**として書かれている。オントロジーも RDF データの一種なのである。上で読んだ公理のいくつかを Turtle で書くと、次のようになる (`RiC-O` 本体はこれと同内容の RDF/XML である)。

```
# 下位クラス:Person は Agent の一種
rico:Person rdfs:subClassOf rico:Agent .

# 下位プロパティ:直接含むなら、含む
rico:directlyIncludes rdfs:subPropertyOf rico:includesTransitive .

# 逆プロパティ:「～の作成者である」は「作成者を持つ」の逆
rico:isCreatorOf owl:inverseOf rico:hasCreator .

# 推移性(連鎖公理と同じ意味)
rico:includesTransitive a owl:TransitiveProperty .

# 3歩の連鎖の近道(丸括弧は経路のリスト)
rico:hasOrganicProvenance owl:propertyChainAxiom
    ( rico:thingIsSourceOfRelation
      rico:organicProvenanceRelation_role
      rico:relationHasTarget ) .
```

記号と Turtle は同じことの 2 つの表記であり、人が考えるときは記号が、機械に渡すときは Turtle(や RDF/XML) が使われる、という分担になる。公理の集まりのほうは **TBox**(terminology box) と呼ぶ。

コラム: クラスが主語になる書き方 — 2 つの読み方

上のリスティングをよく見ると、奇妙なことに気づくかもしれない。`rico:Person rdfs:subClassOf rico:Agent` を素直に述語として読むと `subClassOf(Person, Agent)` となり、`Person` が**項**(ものの名前)の位置に現れる。しかし `Person` は `Person(x)` のように使う**述語**だったはずだ。述語を項にして量化や主張の対象にするのは、一階述語論理の枠を越える (二階論理の) 操作に見える。先ほど `a` のトリプルで見たねじれ—述語の名前が項の位置に入る—と同じ問題が、公理のトリプルでは全面化するわけである。この扱いには標準的な答えが 2 つある。

読み方 1: トリプルは公理の符号化にすぎない (OWL 2 DL の立場)。この立場では、`subClassOf`

のトリプルは論理式そのものではなく、公理を運ぶための**構文**である。推論器は読み込みの段階でこれを $\forall x (\text{Person}(x) \rightarrow \text{Agent}(x))$ という一階の文に翻訳する。項の位置に現れる **Person** は、述語の**名前**を引用しているだけで、クラス全体にわたる量化はどこにも起きない。だから一階の範囲に収まる。

読み方 2: すべてを「もの」として扱いきる (RDF 意味論・規則エンジンの立場)。逆の方向もある。クラスも個体の一種とみなし、所属を $\text{type}(a)$ の正体) という普通の 2 項述語で書くのである。すると $\text{subClassOf}(C, D)$ はそのまま一階の原子式であり、次の橋渡しの規則を公理として足せば、読み方 1 と同じ推論が得られる。

$$\text{type}(x, C) \wedge \text{subClassOf}(C, D) \rightarrow \text{type}(x, D)$$

ここで量化されている変数 C, D が走る範囲は「クラスという名の**個体**」であって述語ではないから、これも一階に収まる。トリプルストアの実装は実際にこちらの発想で動いている。データ全体を $\text{triple}(s, p, o)$ という 1 つの 3 項述語の巨大な表とみなし、この種の規則 (データベース分野で Datalog と呼ばれる規則言語の形) を回して導出トリプルを蓄える。第 6 章で「推論器が破線のトリプルを追加する」と述べる処理の内実は、これである。そして、この立場をとれば、 a を 2 項述語 type と素直に読むことも正当化される。 $\text{Person}(p1)$ と $\text{type}(p1, \text{Person})$ は、同じ事実の 2 つの表記として、橋渡しの規則を介して行き来できる。

つまり、どちらの読み方でも一階論理の外へは出ない。そして「同じ名前がクラスとしても個体としても使われる」こと自体は、OWL 2 で**パニング (punning)** として公認された作法である。本書にも実例がすでに登場している。 rst:Fonds は、記録集合の種別を指す**個体** ($\text{rico:hasRecordSetType}$ の値) でありながら、SKOS の概念でもある (SKOS という標準そのものは第 7 章 7.4 節のコラムで説明する)。文脈によって役割を切り替えて読めばよい。

推論 (第 6 章) とは、TBox(公理) を ABox(事実) に適用して、新しい原子式—すなわち新しいトリプル—を導く計算にほかならない。たとえば「 $\text{Person} \sqsubseteq \text{Agent}$ で、甚兵衛は **Person** である。ゆえに甚兵衛は **Agent** である」という三段論法が、その最も単純な形である。第 6 章の各場面には、使われている公理をこの記法で添える。

コラム:RDB の世界地図で読むオントロジー

リレーショナルデータベースに馴染みがあれば、次の対応で概観できる。クラス $\approx$ テーブル定義、個体 $\approx$ 行、データタイププロパティ $\approx$ 列、オブジェクトプロパティ $\approx$ 外部キー (結合)、IRI $\approx$ 全世界で一意的な主キー。ただし決定的な違いが 3 つある。第一に、RDB の行は必ずどれか 1 つのテーブルに属するが、RDF の個体は複数のクラスに同時に属せる (公式サンプルには $\text{rico:Instantiation}$ かつ $\text{premis:Representation}$ という個体が実際に登場する)。第二に、RDB のスキーマは制約 (違反データは拒否される) だが、OWL の公理は推論の根拠である (「 rico:hasCreator の値域は rico:Agent 」とは「作成者と書かれたものはエージェントだと**推論してよい**」という意味であり、入力チェックではない)。第三に、RDB は閉世界仮説 (書かれていないことは偽) で動くが、OWL は開世界仮説 (書かれていないことは**不明**) で解釈される。「作成者のトリプルがない」ことは「作成者がいない」ことを意味しない。この違いは検索結果の解釈に直接影

響する (第 6 章)。

5.3 なぜ同じ土俵に載るのか — RDF・OWL・SKOS の層構造

ここまでに RDF、RDFS、OWL が登場し、後の章では SKOS や Dublin Core という名前も出てくる。似た顔の英字略語が並ぶので、互いの関係と、1 つのデータの中に混ぜて書ける理由を、ここで整理しておく。結論を先に言えば、これらは層をなす 1 組の積み木であり、積み木をつなぐ約束はただ 1 つ、「すべてをトリプルで書く」である (図 5.3)。

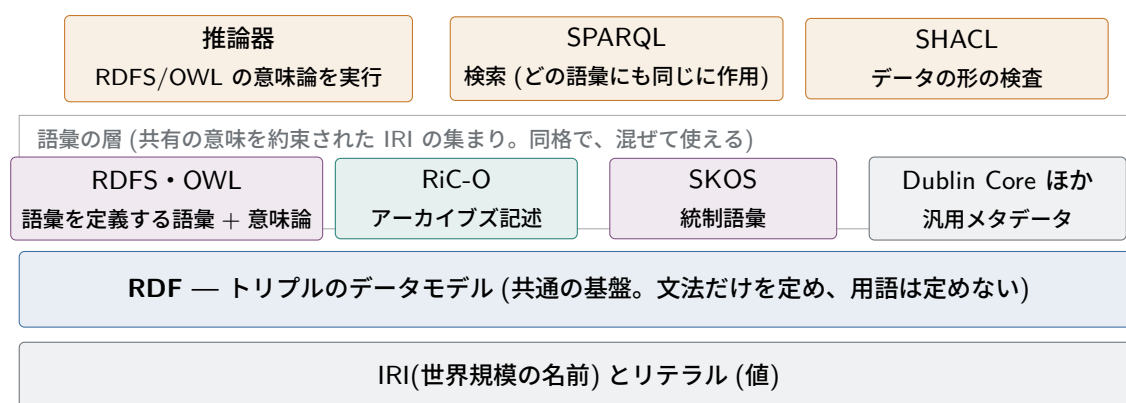


図 5.3 層構造の見取り図。基盤は RDF(トリプル) だが、その上に載る語彙はすべて同格である。RDFS・OWL だけが「語彙を定義し、推論の意味論を与える」という特別な役割を持ち、SPARQL や SHACL はどの語彙のトリプルにも同じに働く。

■**基盤は RDF だけ 1 つ** 最下層の約束は、名前を IRI で付けること (5.1 節)。その上の RDF は、データを主語・述語・目的語のトリプルで書くという**データモデル**である。重要なのは、RDF 自身はほとんど用語を持たないことだ (`rdf:type` が数少ない例外である)。RDF は「何を言うか」を決めず、「どう言うか」だけを定める—文法だけの言語である。

■**語彙はすべて同格** その上の層が**語彙** (vocabulary) である。語彙とは、突き詰めれば**共有の意味を約束された IRI の集まり**にすぎない。`rico:` も `skos:` も `dcterms:` (Dublin Core) も、そして `rdfs:`・`owl:` 自身も、この意味ではすべて語彙であり、同格である。トリプルの述語や型の位置には、どの語彙の IRI を置いてもよい。トリプルストアは述語の所属を区別しないから、1 つのデータの中に `rico:` と `skos:` と自前の `myo:` が同居できる (第 7 章 7.4 節の例がそうだった)。日本語という 1 つの文法の上で、法律用語と医学用語を同じ文章に混ぜて使えるのと同じで、文法は用語の混在を妨げない。

■**特別なのは RDFS と OWL だけ** 語彙の中に、役割の特別なものが 1 組だけある。`rdfs:` と `owl:` は「語彙を定義するための語彙」であり (`subClassOf`、`inverseOf`…), さらに数学的な**意味論**—この形のトリプルからは何が帰結するか、という規定—を持つ。推論器とは、この意味論の実行系である (第 6 章)。RiC-O は「OWL で定義された語彙」なので、OWL の意味論をまるごと受け継ぐ。SKOS は逆に、意味論を意図的に軽くした語彙である (`skos:broader` から論理的な帰結はほとんど出ない。前章のコラム)。つまり OWL と SKOS は別の層にいるのではなく、同じ語彙の層の中での**意味の重さ**の

違いである。

■**道具はどの語彙にも同じに働く** 最上段は言語・道具の層である。SPARQL は検索言語、SHACL は検査言語で、これらは語彙ではない。どの語彙のトリプルに対しても同じに作用する。だから「SKOS のデータを SPARQL で検索する」ことにも「RiC-O のデータを SHACL で検査する」ことにも、何の追加の仕掛けも要らない。

■**意味を解釈する主体** 統語の統一は以上のとおりだが、`skos:broader` や `rico:hasCreator` の意味を解釈する主体は別に押さえておく必要がある。語彙ごとに専用の処理系があるわけではない。推論器が組み込みで知っているのは RDFS と OWL の意味論だけで、RiC-O や SKOS の意味は、それ自身の定義ファイル—OWL の公理の集まり—として**データの側**に書かれている。推論器にデータと一緒にこの定義を読み込ませれば、OWL の意味論を実行するだけで各語彙の帰結が得られる。形式意味論に載らない意味 (どのラベルを画面に出すか、規約が守られているかの検査) はアプリケーションと SHACL が受け持ち、散文の仕様を読むのは人間である。

コラム: プログラミングにたとえると — 語彙はライブラリである

プログラミングの経験があれば、次の対応で見通しがよくなるはずである。

RDF の世界	プログラミングの世界
IRI	名前空間付きの識別子
RDF(トリプル)	言語の文法とデータモデル
語彙 (<code>rico:</code> 、 <code>skos:</code>)	ライブラリ
語彙の公理 (OWL ファイル)	ライブラリの実装コード
推論器	言語処理系
散文の仕様書	ドキュメント

NumPy を使うのに専用のインタプリタは要らない。動作は NumPy 自身のコードに書かれていて、処理系は言語の意味論でそれを実行するだけだからである。同じ理由で「RiC-O 専用推論器」は存在しない。SKOS は、実装がほとんど空の「名前と規約だけのライブラリ」に当たる。たとえの限界は、コードが手続きであるのに対し公理は宣言であること (第6章のコラム「公理と手続き」)。それでも「意味は同梱物の側に書かれ、処理系は汎用のものが1つあればよい」という仕組みは同じである。

この層構造が見えると、「RiC-O を使う」の意味が正確になる。RDF というデータモデルを選び、その上で RiC-O という語彙 (と、それが継承する OWL の意味論) を主たる用語集として使い、必要な場面で SKOS や Dublin Core の用語を混ぜる—それだけのことである。標準たちは競争していない。分業している。

5.4 RiC-O の全体像

RiC-O 1.1 の規模を数字で押さえておく (RiC-O 公式ドキュメントによる)。名前空間 IRI は <https://www.ica.org/standards/RiC/ontology#>、推奨接頭辞は `rico:` である。

- クラス:107(RiC-CM の実体 19 に対し、大幅に多い。差分の由来は次節)
- データタイププロパティ:75
- オブジェクトプロパティ:480(RiC-CM の 85 関係に対応するものを含む)
- 名前付き個体:7(Fonds、Series、File、Collection(記録集合種別、定義は ISAD(G) 用語集に由来)、FindingAid、AuthorityRecord、IIIFManifest(記録種別))

プロパティの命名には規則性がある。過去と現在の両方をカバーする関係は `hasOrHad.../isOrWas...` と命名される (例:`rico:hasOrHadMember`—「現メンバーまたは旧メンバー」)。アーカイブズ記述は本質的に歴史記述であり、「かつてそうだった」関係を常に扱うための工夫である。全クラス・全プロパティに英仏西 (1.1 からは独も) のラベルと定義が付き、RiC-CM のどの構成要素に対応するかも注記されるので、CSV 一覧と併せれば辞書として使える。

設計原則としては 5 つが掲げられている (RiC-O ドキュメント)。領域参照オントロジーであること (他分野のオントロジーからの借用をあえてせず、アーカイブズ共同体の管理下に置く)、即座に使えること (既存メタデータからの変換で情報を失わない)、柔軟な枠組みであること (粗くも細かくも書ける)、記述の新しい可能性を開くこと (SPARQL と推論)、拡張可能であること (下位クラス・下位プロパティの追加、SKOS 語彙—統制語彙を RDF で表すための標準。第 8 章—との接続) である。とりわけ「即座に使える」ことを優先して**同じ情報に複数の表現方法を意図的に残している**点は、利点 (移行が容易) と引き換えに一貫性の維持を実装側に委ねる設計判断であり、第 3 章のトレードオフ論とつながる。

5.5 RiC-CM から RiC-O へ — 何ができるか

5.5.1 実体はクラスへ

RiC-CM の各実体には同名のクラスが 1 対 1 で対応し、階層もそのまま写される。ここは素直である。

5.5.2 属性はデータタイププロパティまたはクラスへ

自由テキスト系の属性 (一般記述、歴史、範囲と内容など) はデータタイププロパティになる。一方、統制値系の属性 (担体種別、記録集合種別、活動種別など) は**クラス**に昇格する (`rico:CarrierType`、`rico:RecordSetType` など、`rico:Type` の下位クラス群)。名称と識別子も `rico:Appellation`(呼称) の下位クラスとして実体化されている。RiC-CM §3.3 が予告していた「実装では属性を実体として扱う」の実行である。

ただし、ここで RiC-O の「即座に使える」原則が顔を出す。クラス化された概念のほぼすべてに、**リテラルで済ませる簡便法**も併設されているのである。種別をクラスのインスタンスへの参照で書く代わりに `rico:type` プロパティに文字列を書いてよいし、日付を `rico>Date` 実体にせず `rico:date`(および多数の下位プロパティ) に文字列で書いてもよい。名称も `rico:Name` インスタンスを作らず `rico:name` に書けばよい。これらの簡便プロパティには「使えるのは、対応するクラスを使わない場合のみ。将来廃止されうる」という注記 (`skos:scopeNote`) が付いており、移行期の妥協であることが明示されている。

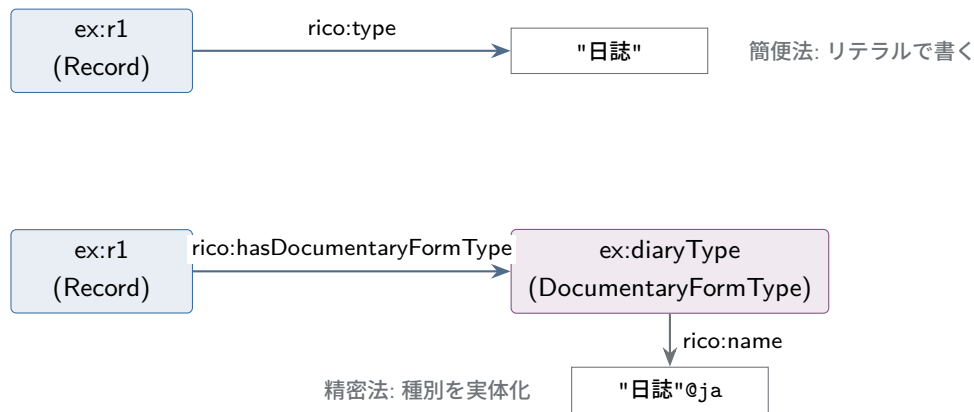


図 5.4 同じ情報の 2 つの書き方 (上: 簡便法、下: 精密法)。精密法なら種別自体を SKOS 語彙として整備し、多言語ラベルや上位下位関係を持たせ、機関を越えて共有できる。

5.5.3 関係はオブジェクトプロパティ + 関係クラスへ

RiC-CM の関係の実装は 2 本立てである。

第一に、各関係に対応する**直接のオブジェクトプロパティ**がある。R026(出所を持つ)なら `rico:hasOrganicProvenance` である (名称のずれは第 4 章 4.4 節末尾で述べたとおり)。トリプル 1 本で書いて簡単だが、関係そのものに属性 (日付・確実性・出典。RiC-CM §5.5) を付けられない。RDF のトリプルは 3 語で完結しており、「トリプルについてのトリプル」を素直には書けないからである。

第二に、関係を**クラスとして実体化** (reification) した関係クラス群がある (`rico:Relation` を頂点に、`rico:CreationRelation`、`rico:OrganicProvenanceRelation` など 48 種)。関係を 1 個のノードにしてしまえば、そのノードに日付や確実性をぶら下げられる。複数の主体が 1 つの関係に関与する場合 (n 項関係) も表現できる。

どちらで書くべきかは、また実装の判断である。関係について語るべきこと (日付、根拠) があるなら実体化し、なければ直接プロパティで十分、というのが実際的な指針になる。重要なのは、両者が無関係に重複しているのではなく、OWL 2 のプロパティ連鎖公理 (property chain axiom) によって形式的に結ばれていることである。`rico:hasOrganicProvenance` は「`rico:thingIsSourceOfRelation` → `rico:organicProvenanceRelation_role` → `rico:relationHasTarget` という経路の近道」として定義されており、推論器を有効にした環境では、長い経路から近道が自動的に導かれる。

5.5.4 RiC-O にしかないもの — Proxy

RiC-CM に対応物がない代表的なクラスが `rico:Proxy` である。定義は「ある記録資源を、特定の別の記録資源の中にあるものとして代理するもの」。用途は、文脈ごとに別の情報を持たせることにある。

1 つの記録が複数の記録集合に属せる (多重所属) ことを思い出そう。ある書簡が「山田家文書」の一部であると同時に、研究者が編んだ「震災関係資料コレクション」にも属するとする。このとき「山田家文書の中での整理番号や配列順序」と「コレクションの中での配列順序」は別々の情報である。記録そのものに順序を持たせると、2 つの文脈の情報が衝突する。そこで「文脈ごとの分身」として Proxy を立て、**Proxy** に順序や文脈固有の情報をを持たせる。

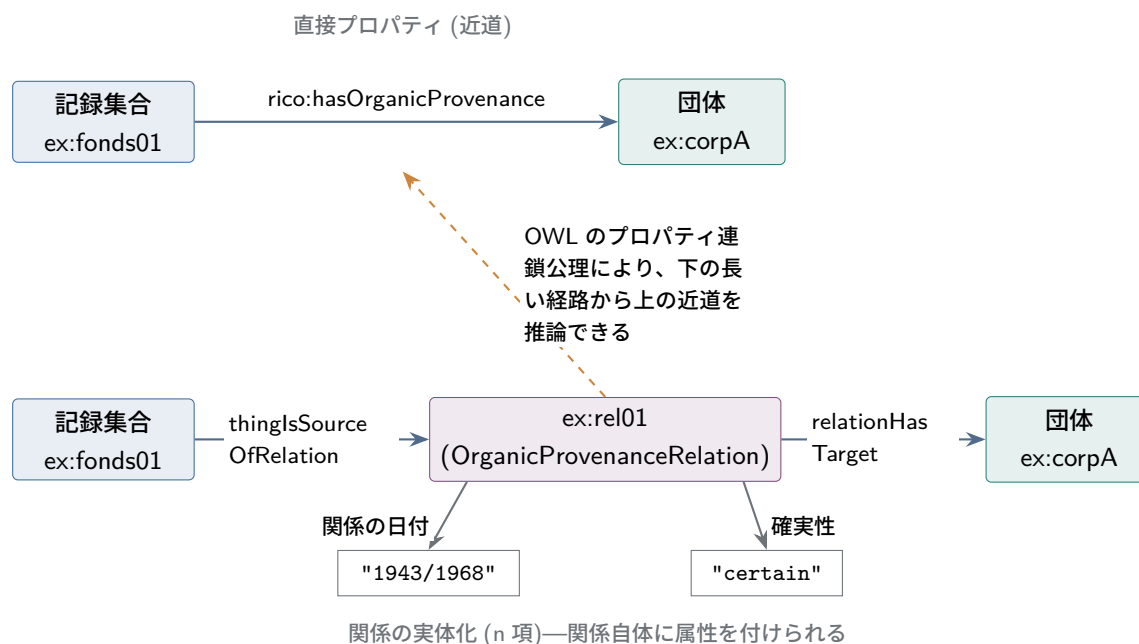


図 5.5 同じ出所関係の 2 つの表現。RiC-O では多くの直接プロパティが「実体化された関係を経由する長い経路の近道」として形式的に定義されており、長い経路を書けば近道は推論で得られる (第 6 章)。

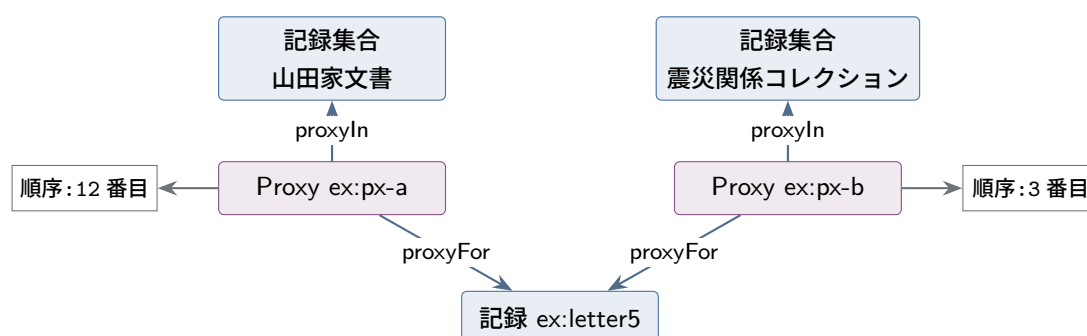


図 5.6 Proxy の仕組み。同じ記録が文脈 (所属する記録集合) ごとに別の「分身」を持ち、配列順序のような文脈固有の情報は分身の側に記録される。原秩序 (第 3 章) を、他の文脈での秩序と衝突させずに保存できる。

順序そのものは `rico:precedesInSequence`/`rico:followsInSequence` 系のプロパティ (Proxy 用の変種を含む) で表す。RiC-O 1.1 では補助的に順位を数値で持つ `rico:rankInSequence` も追加されたが、順序プロパティと併用する場合に限る、と注記されている。EAD 経験者なら、この Proxy は EAD の構成要素の並び (文書順) をグラフの世界に持ち込むための装置だと理解すればよい。なお同名の概念が OAI-ORE という別の標準にもあり、発想はそこから借りられている。

5.6 実例を読む

公式サンプル (スイス docuteam 社のデジタル保存パッケージの例) を簡略化したものを読んでみよう。デジタル保存の実務では PREMIS 語彙と併用されることが分かる。

読む前に、見慣れない名前の正体を説明しておく。以下に現れる `<_20201118115821577>` や `<700>` は、**リテラルではなく IRI** である (文法コラムのとおり、引用符 `"..."` で囲まれていればリテラル、山括弧 `<>` で囲まれていれば IRI である)。ただしこれまでの例と違って接頭辞を使わず、**相対 IRI**—基底となる IRI に対する相対的な名前で、データの読み込み時に完全な IRI に補完される—で書かれている。名前の中身は、システムが発行時刻 (2020 年 11 月 18 日 11 時 58 分 21 秒 577) から機械的に作った機関内の識別子であり、それ自体に意味はない。「一意でありさえすれば、名前は無意味な記号列でよい」という IRI の性格がよく現れた実例である。長いので、以下では `<_2020...577>` のように中略して示す。

```
PREFIX rico:  <https://www.ica.org/standards/RiC/ontology#>
PREFIX premis: <http://www.loc.gov/premis/rdf/v3/>

<_2020...577> a rico:Record ;
    rico:title "Logo docuteam" ;
    rico:scopeAndContent "Enthaelet eine Logo-Datei" ;
    rico:isOrWasRegulatedBy <700> ;                # 閲覧制限などの規則
    rico:thingIsSourceOfRelation [                  # 作成関係を実体化
        a rico:CreationRelation ;
        rico:type "archivist" ;
        rico:relationHasTarget <101> ] ;
    rico:hasOrHadDigitalInstantiation <_2020...159> .

<_2020...159> a rico:Instantiation, premis:Representation .

<700> a rico:Rule ;
    rico:type "accessRestrictionsClosureYear" ;
    rico:endDate "2025" .

<101> a rico:Person ;
    rico:name "Tobias Wildi" .
```

注目してほしい点を挙げる。第一に、1つの個体が `rico:Instantiation` と `premis:Representation` の両方のクラスに属している。RiC-O は具現化の物理管理・保存の詳細を扱わない方針なので、そこは PREMIS に接続する—という設計思想 (RiC-CM §1.2, RiC-O 設計原則) がそのまま実データに現れている。第二に、閲覧制限が `rico:Rule` 実体として表現されている。ISAD(G) では「アクセス条件」というテキスト要素だったものが、終了年を持つ独立の実体になり、機械的な判定 (2025 年を過ぎたか) に使える形になっている。第三に、作成関係が実体化され、関与の種別 (“archivist”) が関係に付与されている。前節までの道具立てが全部使われていることが分かるだろう。

確認問題 (ヒントは付録 A.5)

問 5.1 次の Turtle を日本語で読み下せ。

```
ex:s2 a rico:RecordSet ;
    rico:name "防疫統計系列" ;
    rico:isOrWasIncludedIn ex:fondsP83 .
```

問 5.2 問 5.1 の内容を、セミicolonとコンマを使わず、3つの独立した文 (トリプル) として書き直せ。

問 5.3 `hasCreator-` はどんな関係か。規則の形 ($\rightarrow$ を使った形) で書け。

問 5.4 「クラスが主語になる」文の例を本章から 1 つ挙げ、パニングの考え方でどう読めるか説明せよ。

第 6 章

推論 — グラフだからできること

RiC-O の公式ドキュメントは、RiC-O 導入の効用として「SPARQL による問い合わせ」と並んで「オントロジーの論理による推論 (inference)」を挙げる。本章では推論とは何かを説明し、アーカイブズ記述で推論が実際に役立つ場面を、具体例で 1 つずつ確かめる。

6.1 推論とは何か

推論とは、明示的に記述されたトリプルとオントロジーの公理 (定義) から、書かれていないが論理的に導けるトリプルを機械が導出することである。

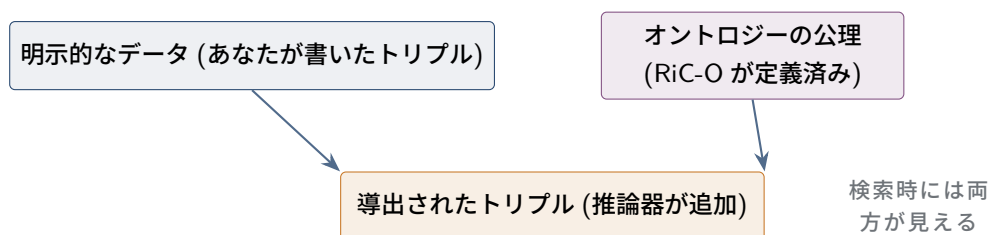


図 6.1 推論の基本的な仕組み。記述者は最小限を書き、残りは推論器が補う。

推論の元手になる公理は、RiC-O にあらかじめ組み込まれている。主なものは 5 種類で、いずれも第 5 章で登場した。下位クラス (subClassOf)、下位プロパティ (subPropertyOf)、逆プロパティ (inverseOf: RiC-O には 419 の逆プロパティ宣言がある)、推移性 (TransitiveProperty)、プロパティ連鎖 (propertyChainAxiom: 243 件) である。つまり RiC-O は、用語のリストであると同時に推論の規則集でもある。この規則集を活用するには、推論機能を持つトリプルストア (RDF 専用のデータベース。GraphDB、Fuseki など。第 7 章) にデータを載せ、推論を有効にするだけでよい。

本章の例では、推論の結果を確かめる道具として SPARQL (RDF の検索言語) の式がたびたび登場する。初見でも次のことだけ知っていれば読める。

コラム: SPARQL を読むための最小限

SPARQL の基本形は `SELECT 変数 WHERE { パターン }` である。読み方は 3 つ覚えればよい。

第一に、`?r` のように `?` で始まる語は **変数** (まだ分かっていない穴) である。SELECT の後に並べた変数が、答えとして返してほしいものを指す。

第二に、WHERE の波括弧の中身は、Turtle とまったく同じ書き方の「主語 述語 目的語」パターンの並びである (ピリオドやセミコロンの規則も Turtle と同じ)。違いは、任意の位置に変数を置けることだけである。たとえば `?r a rico:Record` は「`?r` は記録である」という条件を表す。

第三に、同じ変数が複数のパターンに現れたら、それらは**同じもの**を指す。検索エンジンはグラフの中から、すべてのパターンを同時に満たす変数の割り当てを探し、その一覧を返す。これだけである。なお実際の式には Turtle と同様の PREFIX 宣言が先頭に付くが、本書の例では紙幅の都合で省略している。

この 3 点で本章の例はすべて読める。より進んだ機能 (条件を絞る FILTER、「～が存在しない」ことを調べる FILTER NOT EXISTS、検索結果から新しいトリプルを作る CONSTRUCT など) は、登場する箇所でその都度説明する。

以下、推論の効用を場面別に見る。

6.2 場面 1: 階層をまたぐ検索 — 推移性

フォンド→シリーズ→ファイル→記録という 4 層の編成を、各層のあいだの「直接含む」だけで記述したとする。

```
ex:fonds01    rico:directlyIncludes  ex:series02 .
ex:series02   rico:directlyIncludes  ex:file07 .
ex:file07     rico:directlyIncludes  ex:record31 .
```

利用者が知りたいのは多くの場合「フォンド 01 に (直接だろうと間接だろうと) 含まれる記録の全部」である。RiC-O では `rico:directlyIncludes` が推移的プロパティ `rico:includesTransitive` の下位プロパティとして定義されているため、推論器は図 6.2 の破線のトリプルを自動で導出する。記述論理の記法 (第 5 章 5.2.1 節) で書けば、使われている公理は次の 2 本である。

$$\text{directlyIncludes} \sqsubseteq \text{includesTransitive}$$

$$\text{includesTransitive} \circ \text{includesTransitive} \sqsubseteq \text{includesTransitive}$$

左は「直接含むなら含む」、右は「含むものが含むものは、含む」(推移性) と読む。この 2 本から導かれる「含む」の全体を**推移閉包** (transitive closure) と呼ぶ。直接の関係を出発点に、たどれるかぎりたどってできる関係の集まりのことである。フォンドからシリーズ、シリーズからファイルとたどれるなら、フォンドからファイルへの関係も閉包に入る。右の公理を変数を使った規則の形 (第 5 章 5.2.1 節) で書けば

$$\text{includesTransitive}(x, y) \wedge \text{includesTransitive}(y, z) \rightarrow \text{includesTransitive}(x, z)$$

である。

推論による導出(書かなくてよい)

```
ex:fonds01    rico:includesTransitive  ex:series02 , ex:file07 , ex:record31 .
ex:series02   rico:includesTransitive  ex:file07 , ex:record31 .
```

したがって、階層が何段あっても、SPARQL は 1 行で書ける。

```
SELECT ?r WHERE {
```

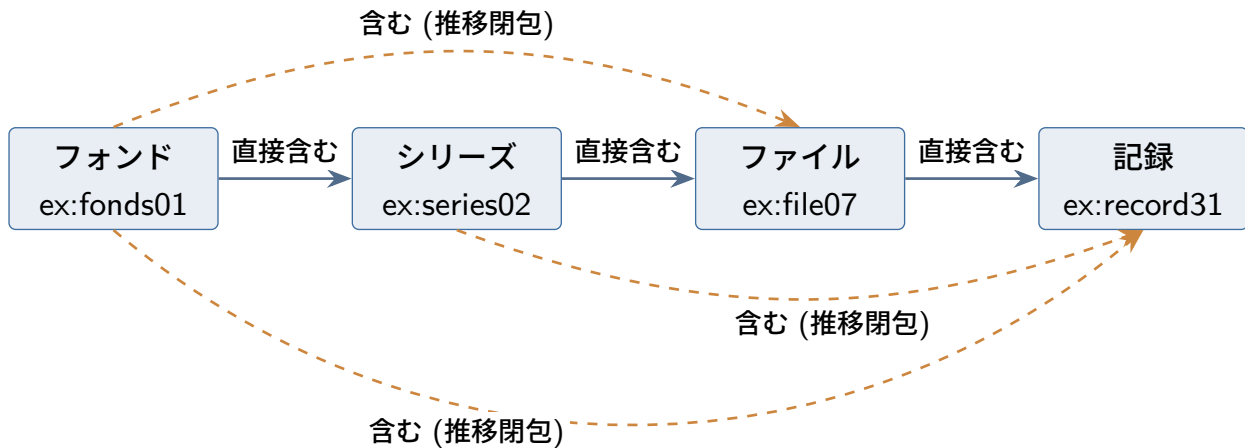


図 6.2 推移性による導出。実線は記述するトリプル (`rico:directlyIncludes`)、破線は推論で得られるトリプル (`rico:includesTransitive`)。書くのは隣り合う階層の 3 本だけで、階層をまたぐ「含む」は推論器が補う。

```
ex:fonds01 rico:includesTransitive ?r .
?r a rico:Record .
}
```

読み下すと「fond 01 が (推移の意味で) 含んでいる ?r であって、かつ記録であるものを、すべて挙げよ」。2 つのパターンに同じ変数 ?r が現れているので、両方の条件を同時に満たすものだけが答えになる。結果は `ex:record31` を含む記録の一覧である。

EAD/XML でこれに相当する処理は、木を再帰的にたどるプログラムを書くことだった。RDB では再帰共通表式 (recursive CTE) という比較的高度な SQL が要る。グラフ + 推論では、モデルの側が「含む関係は連鎖する」と知っているので、問い合わせが単純になる。全体-部分 (`rico:hasPartTransitive`)、イベントの部分 (`hasSubevent...`)、場所の包含、先祖 (`rico:hasAncestor`)、時間的先行 (`rico:precedesInTime`) にも同様の推移的プロパティが用意されている。

コラム: 公理と手続き — 宣言と計算のあいだ

プログラムを書いた経験があると、推移性を次の 2 本の規則で書きたくなるかもしれない。親子関係 `parent` から祖先関係 `ancestor` を作る、おなじみの再帰である。

$$\text{parent}(x, y) \rightarrow \text{ancestor}(x, y)$$

$$\text{parent}(x, y) \wedge \text{ancestor}(y, z) \rightarrow \text{ancestor}(x, z)$$

これは計算の手順 (元データ `parent` に 1 歩ずつ継ぎ足して閉包を作る) が透けて見える書き方である。対して本文の公理は $\text{parent} \sqsubseteq \text{ancestor}$ と $\text{ancestor} \circ \text{ancestor} \sqsubseteq \text{ancestor}$ 、つまり結果の性質だけを述べる。

書き手が手続きを意識する必要は、原則としてない。DL/OWL は宣言的な言語で、公理は「何が帰結するか」だけを定め、どんな順序でどう計算するかは推論器に任される。決定可能性 (第 5 章) は「どう計算しても必ず終わる」ことの保証であり、この分業—意味は書き手、手順は処理系—こそが宣言的言語の眼目である。ちなみに上の 2 本も OWL 2 で書ける (1 本目は下位プロパティ、

2 本目は $\text{parent} \circ \text{ancestor} \sqsubseteq \text{ancestor}$ という連鎖公理)。

ただし、但し書きが 2 つある。第一に、2 つの書き方は**意味が微妙に違う**。parent の事実だけから出発するなら帰結は同じだが、RiC のデータでは「親子は不明だが祖先であることは分かる」という粗い事実を *ancestor* として**直接**主張できる (粒度の自由)。推移公理版は、そうして直接主張された祖先関係同士も合成する ($\text{ancestor}(a, b)$ と $\text{ancestor}(b, c)$ から $\text{ancestor}(a, c)$)。線形再帰版は parent を経由しない合成をしない。RiC-O が *rico:hasAncestor* を推移的プロパティとして定義しているのは、粗い主張の混在を前提にした**意味**の選択であって、手続きの都合ではない。

第二に、意味が同じでも**計算の重さ**は公理の形で変わる。線形再帰は「導出済みの事実と元データの結合」だけで閉包が作れるのに対し、推移公理を素朴に適用すると「導出済み同士の結合」が発生し、重複した導出が増えうる。もっともこれは処理系の側で吸収される話で、トリプルストアは推移閉包を専用の処理で計算するのが普通である。OWL 2 にはさらに、「この範囲の公理なら規則エンジンで実装できる」「この範囲なら多項式時間」といった**計算の保証つき部分集合** (プロファイル。RL・EL・QL) が定義されており、RiC-O の主要な公理 (下位クラス・下位プロパティ・逆・推移・連鎖) はおおむね規則エンジン向きの範囲に収まる。記述者としては意味だけを書けばよく、性能はストアの設定と実体化戦略の問題 (本章末の注意点) として切り分けるのが、この技術圏の作法である。

6.3 場面 2: どちら向きに書いても見つかる — 逆プロパティ

記述の現場では「記録の側に作成者を書く」機関と「作成者の側に作成物を書く」機関がありうる。RiC-O では対になるプロパティが *owl:inverseOf* で宣言されているため^{*1}、どちら向きに書かれていても、推論器が逆向きを補い、両方向から検索できる。機関を越えてデータを集約するとき (LOD の本領) には、書き方の向きの違いを吸収するこの仕組みが実質的な意味を持つ。

6.4 場面 3: 詳しく書いた人の損にならない — 下位プロパティのロールアップ

ある機関は人間関係を細かく「配偶者 (*rico:hasOrHadSpouse*)」「教師 (*rico:hasOrHadTeacher*)」と記述し、別の機関は単に「関係があった」としか記述していないとする。RiC-O ではこれらのプロパティが階層をなしている。記述論理で書けば $\text{hasOrHadSpouse} \sqsubseteq \text{hasFamilyAssociationWith} \sqsubseteq \text{isAgentAssociatedWithAgent}$ である。推論器は下位プロパティの言明から上位プロパティの言明を導出するので、「山田花子と何らかの関係があった人物」を上位プロパティ 1 つで問い合わせれば、細かく書かれたデータも粗く書かれたデータも**同じ網**ですくえる。

これは第 3 章で述べた「粒度の自由」の技術的な裏付けである。詳しく書けば詳しい検索に応えられ、粗くしか書けなくても横断検索からは漏れない。記述の濃淡がそのまま検索の分断になる、という事態をオントロジーの階層が防いでいる。クラス側も同様で、*rico:Person* のインスタンスは自動的

^{*1} たとえば *rico:hasCreator* と *rico:isCreatorOf* が対になる。記述論理で書けば $\text{isCreatorOf} \equiv \text{hasCreator}^-$ である。

に `rico:Agent` のインスタンスでもあるから、「エージェント全部」という問い合わせに個人も団体もメカニズムも応答する。

6.5 場面 4: 近道の自動生成 — プロパティ連鎖

第5章 5.5.3 節で見たとおり、関係に日付や確実性を付けたい場合は関係を実体化して書く。2行目の `a` は「～のインスタンスである」を表す述語である (第5章の Turtle 文法コラム)。

```
ex:fonds01 rico:thingIsSourceOfRelation ex:rel01 .
ex:rel01 a rico:OrganicProvenanceRelation ;
    rico:relationHasTarget ex:corpA ;
    rico:relationHasDate   ex:date1943_1968 ;
    rico:relationCertainty "certain" .
```

この「長い経路」に対して、RiC-O は `rico:hasOrganicProvenance` をプロパティ連鎖の近道として定義している。記述論理で書けば

$$\text{thingIsSourceOfRelation} \circ \text{organicProvenanceRelation_role} \circ \text{relationHasTarget} \\ \sqsubseteq \text{hasOrganicProvenance}$$

であり、変数を使った規則の形で書けば

$$\text{thingIsSourceOfRelation}(x, y) \wedge \text{organicProvenanceRelation_role}(y, z) \\ \wedge \text{relationHasTarget}(z, w) \rightarrow \text{hasOrganicProvenance}(x, w)$$

つまり「関係ノードの入口から出口まで3歩でたどれるなら、直接1本で結んでよい」である。推論器は自動的に次を導出する。

```
# 推論による導出
ex:fonds01 rico:hasOrganicProvenance ex:corpA .
```

つまり、丁寧に書かれたデータと簡便に書かれたデータが、検索時には同じ形に揃う。検索する側は常に簡便な近道プロパティで問い合わせればよく、データがどちらの流儀で書かれたかを知らなくてよい。RiC-O 1.1 では日付についても同様の近道 (`rico:hasCreationDate` など) が連鎖として定義された。この仕組みがなければ、「表現方法が複数ある」という RiC-O の柔軟性は検索の悪夢になっていたはずである。柔軟性と検索可能性を両立させる要が、プロパティ連鎖公理だと言ってよい。

6.6 場面 5: 組織の変遷をたどる — アーカイブズらしい総合問題

RiC-O の公式ドキュメントが挙げる問い合わせ例は、アーカイブズ利用の実感によく合っている。「ある機関の消滅後、その活動を (この活動に関して) 引き継いだ団体はどれか」「この写真にはどんな具現化が存在するか」「この原本の現存する写しは何か」。最後に、組織変遷の例を通して確かめる。

県の衛生行政を例にとる。1948年に衛生部が発足し、1994年に保健福祉部へ改組、2010年に健康福祉部へ再編されたとする。RiC ではこれを、3つの団体 (Agent)、1つの活動 (Activity: 衛生行政)、後継関係と活動遂行関係のトリプルで記述できる。

```

ex:eisei      a rico:CorporateBody ; rico:name "衛生部" ;
               rico:hasSuccessor ex:hokenfukushi .
ex:hokenfukushi a rico:CorporateBody ; rico:name "保健福祉部" ;
               rico:hasSuccessor ex:kenkofukushi .
ex:kenkofukushi a rico:CorporateBody ; rico:name "健康福祉部" .

ex:eiseiGyosei a rico:Activity ; rico:name "衛生行政" .
ex:eisei      rico:performsOrPerformed ex:eiseiGyosei .
ex:hokenfukushi rico:performsOrPerformed ex:eiseiGyosei .
ex:kenkofukushi rico:performsOrPerformed ex:eiseiGyosei .

ex:series1950s a rico:RecordSet ; rico:name "伝染病予防関係綴" ;
               rico:hasOrganicProvenance ex:eisei ;
               rico:documents ex:eiseiGyosei .
               # この活動を文書化する

```

このデータをグラフとして描くと図 6.3 になる。ラベルは日本語に意識してある (対応するプロパティ名は上の Turtle のとおり)。

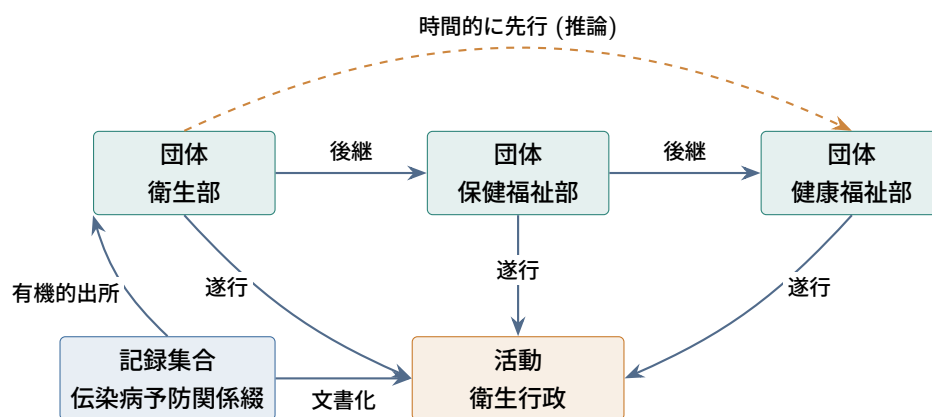


図 6.3 組織変遷の記述のグラフ表現。シリーズから旧機関へ、後継の鎖をたどって現機関へ、また活動を軸に歴代機関の記録集合へと、どの向きにもたどれる。

このデータに対して、たとえば次のことが可能になる。1950 年代のシリーズから出発して、作成組織 (衛生部) → 後継の連鎖 (`rico:hasSuccessor` から導かれる時間的先行関係は推移的) → 現在の担当部局 (健康福祉部) へと機械的にたどれる。逆に、現在の健康福祉部から出発して「歴代の前身組織が同じ活動について残した記録集合の全部」を集めることもできる。

```

# 衛生行政という活動を文書化する記録集合を、作成組織の変遷ごと一覧する
SELECT ?rs ?agent WHERE {
  ?rs rico:documents          ex:eiseiGyosei ;
      rico:hasOrganicProvenance ?agent .
}

```

読み下すと「衛生行政という活動を文書化しており、かつ有機的出所 `?agent` を持つ記録集合 `?rs` について、その組と出所の対をすべて挙げよ」。セミコロンは Turtle と同じく「同じ主語 `?rs` で続ける」の意味である。答えは (伝染病予防関係綴、衛生部) のような対の一覧になり、歴代組織のデータが揃っ

ていれば、活動を軸にした通史的な一覧がこの4行で得られる。

ISAD(G)の世界では、この情報は各フォンドの「管理歴 (administrative history)」に散文で書かれ、照合は人間の仕事だった。RiC では変遷が構造化データになるので、機械が答えられる。行政文書のアーカイブズにとって、これは推論とグラフの効用が最も直接的に現れる場面だろう。

6.7 場面 6: 自分の規則は足せるか — 拡張性と、OWL では書けない規則

ここまでは RiC-O に組み込み済みの公理の話だった。本節では、自分の記述対象に固有の語彙や規則を追加する場面を扱う。「二人の人物の共通の祖先が書いた文書を探したい」という要求を例に、できること・できないことを切り分けてみる。

6.7.1 語彙の追加は明示的に認められている

RiC-O の設計原則の1つは拡張可能性であり、独自の下位クラス・下位プロパティを自分の名前空間で追加することは、公式に想定された使い方である (RiC-O ドキュメント設計原則 5)。RiC-AG も FAQ「How to extend RiC」を設けてこれを正面から扱っており、推奨される作法—独自の名前空間と識別子を用意すること、新しい構成要素を必ず RiC の既存要素の下位として接続すること、拡張を文書化して公開すること、需要が一般的なら EGAD への提案 (GitHub の Issue や利用者メーリングリスト) も検討すること—を、公証文書の「遺言者 (has testator)」を `rico:hasAuthor` の下位プロパティとして定義する事例つきで示している。たとえば村方文書の記述で「名主として作成した」ことを区別したければ、次のように書ける。

```
PREFIX myo: <https://archives.example.jp/ontology#>
myo:hasCreatorAsVillageHead rdfs:subPropertyOf rico:hasCreator .
```

下位プロパティとして宣言しておけば、独自の細かい語彙で記述しても、`rico:hasCreator` での横断検索 (場面3のロールアップ) から漏れない。拡張が「RiC からの逸脱」にならないための要は、この既存階層への接ぎ木である。孤立した独自プロパティを作ると、標準語彙での検索と切れてしまう。

6.7.2 「共通の祖先」の例 — 問い合わせで済むもの

冒頭の要求は、実は新しい規則を追加しなくても実現できる。RiC-O には `rico:hasAncestor` (祖先を持つ) が推移的プロパティとして定義済みなので、親子関係 (`rico:hasChild` など) を書いておけば、世代をまたぐ祖先関係は推論で得られる。あとは SPARQL で「花子の祖先でもあり、太郎の祖先でもある人物が作成した記録」を問い合わせればよい。

```
SELECT ?doc ?x WHERE {
  ex:hanako rico:hasAncestor ?x .    # 花子の祖先(推移閉包)
  ex:taro   rico:hasAncestor ?x .    # 太郎の祖先でもある
  ?x rico:isCreatorOf ?doc .          # その人物が作成した記録
}
```

読み下すと「花子の祖先であり、かつ太郎の祖先でもある人物 ?x と、その ?x が作成した記録 ?doc の対を、すべて挙げよ」。3つのパターンすべてに同じ変数 ?x が現れていることが要で、この変数の共有が「2本の祖先の系譜が同一人物で合流する」という条件そのものを表している。hasAncestor は推移的なので*2、親子関係を書いておくだけで曾祖父母以遠も検索対象に入る。

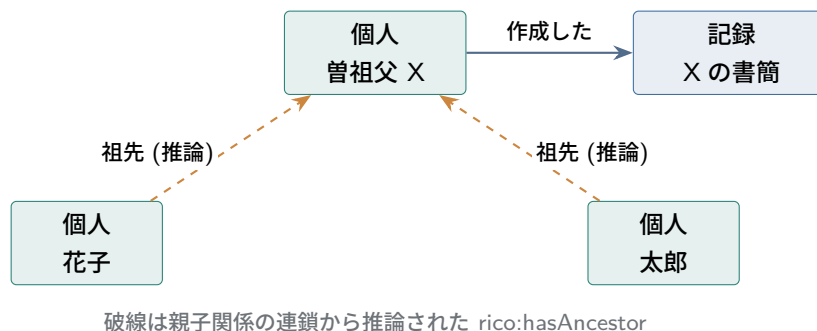


図 6.4 共通祖先の文書を探す。2本の祖先経路が同じ人物 X で合流するかどうかは、SPARQL の変数共有 (2つの三つ組パターンが同じ ?x を指すこと) で表現される。

つまりこの要求は RiC-O の対象範囲内であり、必要な道具は「語彙 (定義済み)+ 推論 (定義済み)+ 問い合わせ (自分で書く)」である。

6.7.3 OWL では書けない規則もある

「共通の祖先を持つ」という関係そのものを、推論で導出される新しいプロパティ (たとえば myo:hasCommonAncestorWith) として定義することは、OWL 2 ではできない。ここに表現力の境界がある。プロパティ連鎖公理 (場面 4) が表現できるのは「A から B へ一本道でたどれる経路」だけであり、「2つの経路が同じ中間ノードで合流する」という条件—論理学で言えば、規則の本体に同じ変数が 2 回現れる結合 (join)—は書けない。この種の規則を導出として実現したければ、OWL の外の道具、すなわちトリプルストアの規則言語 (SWRL、Datalog 系ルール、SHACL ルールなど) か、SPARQL CONSTRUCT による導出トリプルの生成を使うことになる。

整理すると、独自の要求は 3つの層に振り分けて実現する。

1. **語彙の拡張** (下位クラス・下位プロパティの追加)—OWL の範囲内。RiC-O が公式に想定。
2. **答えが欲しいだけの要求**—規則を追加せず SPARQL で問い合わせる。今回の共通祖先はこれで足りる。
3. **OWL で表現できない導出規則**—規則エンジンや CONSTRUCT で実装する。RiC-O の守備範囲の外だが、RiC-O のデータの上で動かすことは通常の技術でできる。

なお「そもそも分野固有の知識を拡張として持つべきか、別の場所に置くべきか」という一段手前の設計判断は、第 7 章 7.4 節で扱う。

*2 記述論理で書けば $\text{hasAncestor} \circ \text{hasAncestor} \sqsubseteq \text{hasAncestor}$ である。

6.8 処理系の中身 — 書いた記述を誰が実行するか

ここまで「推論器」「トリプルストア」とひとまとめに呼んできた処理系の内部を、少しだけ覗いておく。語彙や公理をいくら学んでも、それを実行する機械の姿が見えないままでは、公理・検索・性能の関係が宙に浮いてしまうからである。

6.8.1 トリプルストアの構造

トリプルストアの内部は、リレーショナルデータベースを知っていれば見通しやすい (第 8 章)。おおよそ次の部品からなる。

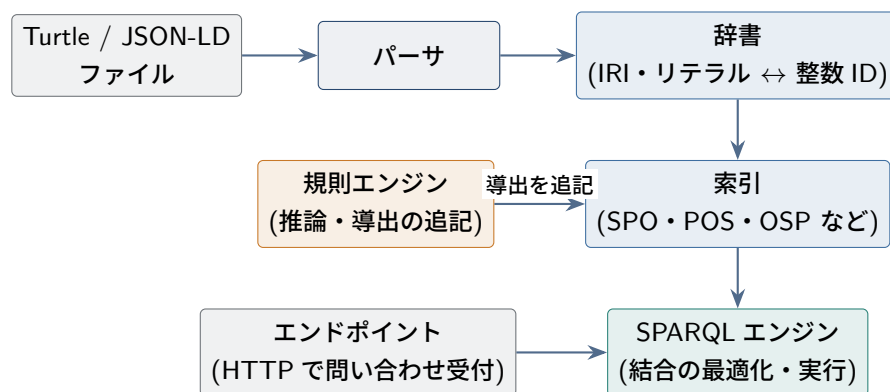


図 6.5 トリプルストアの内部構成のあらまし。RDB でいえば、辞書と索引は B 木索引、SPARQL エンジンの実行計画の最適化器に当たる。

読み込まれたトリプルは、まず辞書で IRI やリテラルが整数の内部 ID に置き換えられる (長い IRI を毎回文字列比較しては遅いからである)。次に索引に格納される。トリプルは主語 (S)・述語 (P)・目的語 (O) の 3 つ組なので、SPO 順・POS 順・OSP 順など複数の並びで冗長に索引を持っておくと、「主語が分かっているとき」「述語だけ分かっているとき」のどの形の問い合わせにも速く答えられる。SPARQL エンジンは、WHERE 節のパターン群をどの順に索引照合すれば結合が小さく済むかを見積もって実行する—RDB の実行計画の最適化と同じ発想である。推論を有効にすると、規則エンジンが公理に対応する規則 (第 5 章のコラムで見た Datalog 型の規則) を繰り返し適用し、導出トリプルを索引に追記していく。

6.8.2 推論の実行方式 — 事前計算か、検索時か

推論の実装には大きく 2 方式ある。マテリアライゼーション (前方連鎖) は、データの投入時に規則を適用し尽くし、導出トリプルをすべて索引に書き込んでおく方式である。検索は速い (すでに書いてあるものを引くだけ) が、格納量が増え、データを削除したときに「その事実依存していた導出」を取り消す再計算が要る。検索時展開 (後方連鎖・クエリ書き換え) は逆に、問い合わせが来た時点で規則をさかのぼって適用する方式で、更新は軽い。検索のたびに計算する。

製品でいえば、GraphDB は前者の代表で、適用する規則の集合 (RDFS のみ、OWL-Horst、OWL

2 RL など) を設定で選ぶ。Apache Jena(Fuseki) は汎用の規則推論器と格納系 (TDB2) の組み合わせで、両方式を構成できる。ほかに Virtuoso、Amazon Neptune、軽量な Oxigraph などがある。いずれも SPARQL エンドポイントという HTTP の窓口を持ち、アプリケーションはそこへ問い合わせを送る。

これらと系統の違う道具として、**タブロー法系**の推論器 (HermiT、Pellet など。Protégé から呼び出せる) がある。これは大量データの導出よりも、**オントロジーそのものの検査**—公理群に矛盾がないか、クラス階層はどう整理されるか—に向く。語彙の検査 (TBox 推論) とデータの導出 (ABox 推論) は同じ「推論」でも性格の違う仕事であり、道具も分かれている。第 6 章の注意点で触れる RiC-O 1.0 の不具合 (48 プロパティの全域反射性が引き起こした矛盾) は、この種の検査によって顕在化し、1.0.2 で修正された。

6.8.3 SPARQL をもう少し — 現場で使う 4 つの機能

基本形 (コラム参照) に加え、実務で頻出する機能を 4 つ、これまでの例データの上で見ておく。

■**OPTIONAL** — **あれば取る、なくても捨てない** 名前があれば添え、なくても一覧から落とさない検索を書く。

```
SELECT ?r ?name WHERE {
  ex:fonds01 rico:includesTransitive ?r .
  OPTIONAL { ?r rico:name ?name }
}
```

OPTIONAL の中は「一致すれば値を返し、なくても行自体は残す」。名前のない記録も空欄つきで一覧に現れる。開世界仮説の世界では「名前が書かれていない」ことは「名前がない」ことを意味しないから、この控えめな挙動が既定であることには意味がある。

■**FILTER** — **値で絞る** 年代で絞り込んでみる。

```
SELECT ?r ?d WHERE {
  ?r a rico:Record ;
    rico:normalizedDateValue ?d .
  FILTER(?d < "1900")
}
```

1900 年より前の記録に絞る。値の比較・文字列照合・言語タグの検査などが書ける (この例は 4 桁の年を文字列のまま比較しており、桁が揃っている場合にだけ正しい。厳密にはデータ型付きリテラルを使う)。

■**プロパティパス** — **推論なしの推移閉包** 述語の繰り返しを、クエリの側で直接書くこともできる。

```
SELECT ?r WHERE {
  ex:fonds01 rico:directlyIncludes+ ?r .
}
```

述語に付いた + は「1 回以上の繰り返し」で、directlyIncludes を何段でもたどる。つまり**推論を使わずに**、問い合わせの側で推移閉包を計算する道もある。使い分けはこうである。推移性を**公理**にすれ

ば「含むは連鎖する」という意味がデータの一部になり、すべての利用者・すべての検索が恩恵を受ける。パスで書けば、その場のその問い合わせ限りの計算で済み、ストアの推論設定に依存しない。公開エンドポイントに推論があるとは限らないので、パスの書き方は覚えておく価値がある。

■CONSTRUCT — 検索結果から新しいトリプルを作る 検索結果を、表ではなくトリプルの形で返させる。

```
CONSTRUCT { ?rs rico:hasOrganicProvenance ?a }
WHERE {
  ?rs rico:thingIsSourceOfRelation ?rel .
  ?rel a rico:OrganicProvenanceRelation ;
       rico:relationHasTarget ?a .
}
```

SELECT が表を返すのに対し、CONSTRUCT は新しいトリプルの集合を返す。この例は、実体化された出所関係から近道プロパティを生成する—つまり場面4の連鎖公理と同じ導出を、推論器なしで手動実行している。データ変換(第7章)や、OWL で書けない規則の実装(場面6)を実際に担うのは、多くの場合この CONSTRUCT である。

6.8.4 OWL 2 プロファイル — 計算の保証つき部分集合

最後に、公理の表現力と処理系の関係を制度として整理したものが OWL 2 のプロファイルである。表現力を上げるほど推論は重くなるので、用途別に「この範囲に収めれば、この方式・この計算量で処理できる」という部分集合が規格化されている。

表 6.1 OWL 2 のプロファイル

名称	性格	主な処理方式・用途
OWL 2 DL	表現力最大 (SROIQ)	タブロー法 (HermiT 等)。語彙の検査向き。最悪計算量は大きい
OWL 2 RL	規則で書ける範囲	規則エンジンでマテリアライズ (GraphDB 等)。大量データの導出向き
OWL 2 EL	大規模な語彙の分類向け	多項式時間。医学用語集 (SNOMED CT) などの巨大オントロジー
OWL 2 QL	RDB の上での問い合わせ向け	クエリ書き換えで SQL に翻訳できる範囲

RiC-O が実際に使っている公理(下位クラス・下位プロパティ・逆・推移・連鎖)は、おおむね OWL 2 RL の範囲に収まる。だから実務では「RL 対応の規則エンジンを持つトリプルストアに載せてマテリアライズする」のが標準の構成になり、本章で見た推論はすべてその上で動く。基数制約 (Proxy の「ちょうど1つ」) のような RL の外の公理は、データ導出にはあまり使われず、語彙検査の場面で意味を持つ。ここでも「表現は控えめに、計算は確実に」という RiC-O の設計方針(第3章のコラム)が確かめられる。

6.9 推論は魔法ではない — 注意点

推論を使うときに知っておくべきこと

開世界仮説に注意。OWL の推論は「書かれていないことは不明」という前提で動く。「作成者が記録されていない記録の一覧」のような欠如の問い合わせは、論理的な否定ではなく「トリプルが存在しない」ことの検査 (SPARQL の `FILTER NOT EXISTS`) として書く必要があり、意味の違いを理解して使い分けなければならない。

推論はデータの誤りも増幅する。誤って張られた 1 本の関係から、多数の誤った導出トリプルが生まれる。特に推移的プロパティは誤りを遠くまで運ぶ。推論の前提は入力データの品質管理である。

計算コストは無料ではない。導出トリプルを事前に全部作る方式 (マテリアライゼーション。§6.8) は検索が速い代わりに格納量と更新コストが増え、検索のたびに導出する方式は逆の性質を持つ。RiC-O 1.0 の初期版では公理の組み合わせが原因で推論器が矛盾を検出する不具合 (48 のロール化プロパティの全域反射性) が見つかり、1.0.2 で修正された、という経緯もある。推論を使う実装は、この種の技術的細部と付き合うことになる (だからこそ、それを引き受けるのは通常システムの側であり、記述者ではない)。

推論が有効なのは公理が意味を捉えているときだけ。RiC-O の公理は EGAD の設計判断の産物であり、あなたの機関の業務ルール (たとえば「同一ファンド内の整理番号は一意」) までは知らない。そうした制約の検証には SHACL のような別の技術を使う (第 8 章)。

確認問題 (ヒントは付録 A.5)

問 6.1 推移性の公理 $R \circ R \sqsubseteq R$ を、変数を使った規則の形で書け。

問 6.2 SPARQL の `OPTIONAL` は、開世界仮説とどうなじみがよいのか。OPTIONAL がない場合に起きることと合わせて説明せよ。

問 6.3 マテリアライゼーション (前方連鎖) と検索時展開の得失を、更新頻度の高いデータを例に比較せよ。

問 6.4 場面 5 の例で、「衛生部文書」から「保健福祉部」へ推論がたどる関係の鎖を、使われる公理とともに書き出せ。

第 7 章

実践の手順

実践の公式の手引きとしては、2025 年 10 月に RiC-AG の草案 0.1 が公開された。ただし AG 自身が「利用場面は多様すぎるため、段階的な手順書の提供は不可能」と明言し、検討しうる行動のリストと事例を示すという方針をとっている。本章では、RiC-AG の実装戦略の章と Getting Started の事例、RiC-CM §1.6.4 の移行論、RiC-O の設計原則と公式サンプル、先行機関の経験を踏まえて、現実的な手順に組み立て直して示す。

7.1 3 つのシナリオ

RiC への関わり方は、機関の状況に応じておおよそ 3 つに分かれる。

シナリオ A: 既存記述の変換・公開 すでに EAD/Excel/データベースにある目録データを RiC-O/RDF に変換し、LOD として公開したり機関横断検索に供したりする。現時点で最も実績が多い。フランス国立公文書館の実例がこの型である。

シナリオ B: RiC ネイティブの記述 新規の記述を最初から実体・関係の形で作成する。対応システムがまだ少ないため先進事例に限られるが、組織変遷の激しい行政文書などでは効果大きい。

シナリオ C: 将来に備えた設計 当面は ISAD(G)/EAD の実務を続けつつ、将来の変換を容易にするようデータの規律を高めておく (作成者の典拠管理、識別子の一意化、日付の正規化など)。ほとんどの機関にとって、当面の現実解はこれである。

RiC-AG の実装戦略の章も、この整理と両立する助言を与えている。目的の特定から始めること (「RiC で何が最も役に立つか」を機関の必要と結びつける)、小さく始めて段階的に育てること (パイロットプロジェクト、失敗するなら最小のコストで)、チーム全体で RiC を学ぶこと (アーキビスト・IT・データ分析者では見る角度が違う)、そして資源の見積もりと業務上の根拠 (ビジネスケース) を作ることである。ソフトウェア導入の順序についても、公開ポータルを先に RiC 化する道、既存データの一对一変換を先行させてから記述の高度化に進む二段構えの道、既存機能と並行して RiC ベースの機能を足していく道、という複数のシナリオを描いている。

シナリオ C が「何もしない」ことと違う点に注意してほしい。RiC への変換で最も苦勞するのは、RiC の複雑さではなく、**既存データの不規律** (同一作成者の表記ゆれ、識別子の重複、日付の自由記述) である。これらは今の実務の中で改善でき、その改善は RiC と無関係にも価値がある。

コラム: 実例の現在地 (2026 年)

「実績が最も多いのはシナリオ A」という本章の見立てには、裏づけがある。先頭を走るのはフランスである。国立公文書館 (AnF) は、国立図書館・文化省との典拠相互運用の実証 PIAAF(2018 年公開) を経て、RiC-O Converter を自ら開発し、パリ公証人記録などから作った約 5,900 万トリプルの知識グラフを、視覚的な検索インタフェース (Sparnatural) 付きで公開している。ポータル FranceArchives は 2023 年に SPARQL リポジトリを公開し、10 万件を超える検索手段から生成した約 5 億トリプル (RiC-O 0.2 準拠) を提供する—現時点で世界最大の RiC 系データセットである。

スイスでは視聴覚遺産ポータル Memobase が RiC-O ベースのデータモデルで本番運用され、docuteam 社の目録ソフトウェア (第 5 章の実例で使った docuteam context) が RiC-O ネイティブの記述を支えている。舞台芸術の SAPA 財団の導入例もスイスである。欧州横断では、ホロコースト関係資料の研究基盤 EHRI が RiC を用いた知識グラフを構築した。このほかポルトガル (科学者書簡)、ギリシャ (裁判所文書)、韓国 (芸術記録)、中国 (配給切符資料のオントロジー拡張) など、各地の適用研究が続いている。ICA EGAD はこれらを集めた「RiC Resource List」を公開しており (巻末の資料リスト参照)、最新の実例はそこで追える。

全体として、本番運用は少数の先行機関に集中し、多くはまだ研究・実証段階—これが 2026 年時点の正直な現在地である。

7.2 シナリオ A の標準的な工程

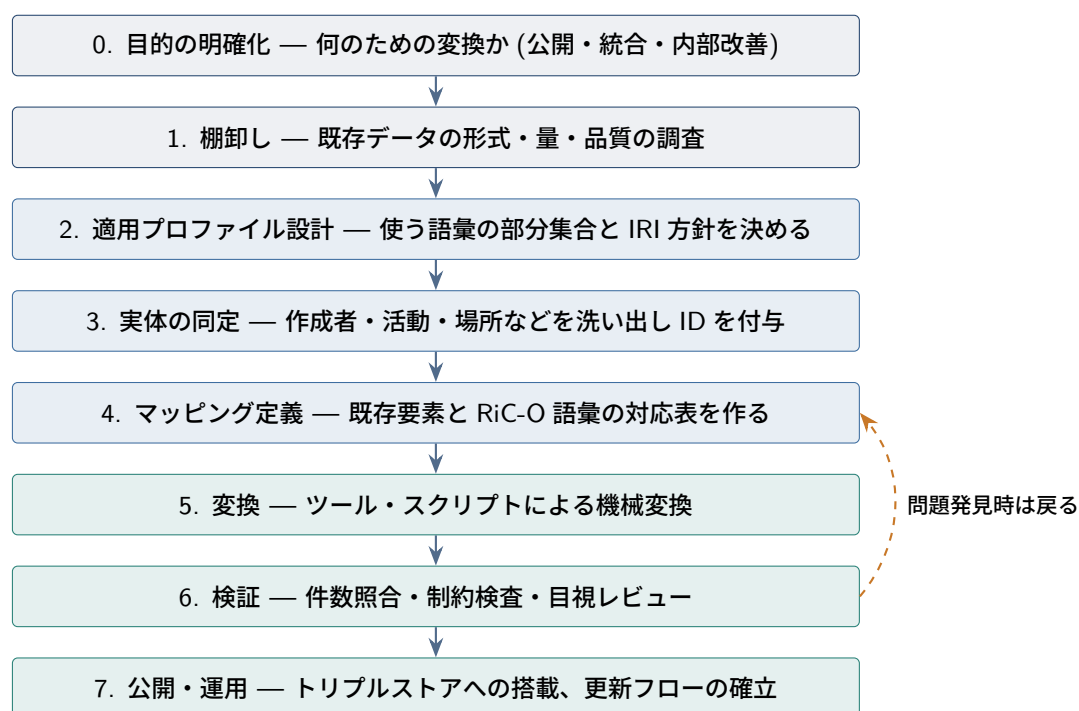


図 7.1 既存記述を RiC-O/RDF へ変換する標準的な工程。2～4 が知的な中心作業であり、アーキビストの判断が最も要る。

7.2.1 工程 2: 適用プロファイルの設計

RiC-O の 480 プロパティを全部使う実装は存在しない。公式ドキュメント自身が「実装機関は必要に応じた部分集合を使えばよい」と明言している。そこで最初に決めるべきは、自機関で使うクラス・プロパティの一覧 (適用プロファイル) である。決めることは主に次の 4 つである。

1. **使う実体の範囲**。最小構成なら Record Set/Record、Agent、Activity 程度から始められる。具現化を分けるか、規則・イベントをどこまで実体化するか。
2. **簡便法と精密法の使い分け** (第 5 章 5.5)。日付・種別・名称をリテラルで書くか実体化するか。推奨は「外部と共有したい概念 (作成者、記録集合種別、場所) は実体化、内部的な注記はリテラル」という切り分けである。
3. **直接プロパティと関係クラスの使い分け** (第 5 章 5.5.3)。関係の日付や典拠を記録する必要がある関係 (出所など) だけ実体化する。
4. **IRI 設計**。ドメイン、パス構造 (/records/、/agents/ など)、変更しないという約束。一度公開した IRI は永続させるのが LOD の倫理である。

このプロファイルは文書化して機関の規則として維持する。RiC の用語で言えば、これは記述という活動を支配する Rule であり、それ自体が RiC で記述できる (RiC-CM 第 6 章)。

7.2.2 工程 4: マッピングの実際

この工程には公式の道具が揃った。RiC-AG 0.1 とともに、既存 4 標準 (ISAD(G)、ISAAR(CPF)、ISDF、ISDIAH) から RiC-CM への包括的なクロスウォーク (対応表) と、EAD 2002 DTD から RiC-O 1.1 へのクロスウォークが、表計算ファイルとして公開された (EAC-CPF 対応も準備中とされる)。AG は同時に、対応が一意でない場合があること (たとえば ISAD(G) の「日付」は複数の解釈がありうる)、機関の符号化慣行に合わせた調整が前提であることも明記している。次の表は、その要点を簡便法での一例として要約したものである。

表 7.1 ISAD(G) 記述要素から RiC-O への対応の例 (簡便法での一例)

ISAD(G) 要素	RiC-O での主な対応
3.1.1 レファレンスコード	rico:identifier(または rico:Identifier 実体)
3.1.2 タイトル	rico:title / rico:name
3.1.3 日付	rico:date 系プロパティ (または rico:Date 実体 + 関係)
3.1.4 記述レベル	rico:hasRecordSetType(Fonds/Series/File 等の個体へ)
3.1.5 数量	rico:RecordResourceExtent(+rico:quantity など)
3.2.1 作成者名	rico:hasOrganicProvenance(または広義の rico:hasOrganicOrFunctionalProvenance) で Agent 実体へ
3.2.2 管理歴・伝記歴	Agent 側の rico:history(散文) または Event 群
3.2.3 記録史	rico:history(または保管・移管の Event 群)
3.3.1 範囲と内容	rico:scopeAndContent
3.4.1 アクセス条件	rico:conditionsOfAccess(または rico:Rule 実体)
3.5.3 関連記述単位	各種の記録資源間関係プロパティ
階層 (上位・下位)	rico:isOrWasIncludedIn / rico:includesOrIncluded など

表を見て気づくとおり、多くの要素には「散文のまま持っていく道」と「実体に分解する道」の両方がある。全部を一度に実体化する必要はない。まず散文のまま変換して器を移し、価値の高いところ(作成者、組織変遷)から段階的に実体化していく、という2段階構えが現実的であり、RiC-Oが簡便法を残しているのはこの進め方を可能にするためである。

7.2.3 工程5～6: 変換と検証

変換の道具としては、EGADが公開しているオープンソースのRiC-O Converter(EAD 2002とEAC-CPFのファイルをRiC-O準拠のRDFに変換する。2020年に第1版公開)が出発点になる。ただし、EADの使い方は機関ごとに癖があるため、既製の変換規則をそのまま使えることは少なく、自機関の符号化慣行に合わせた調整(あるいはXSLT/Pythonによる自作)が普通である。表計算やデータベースからの変換であれば、Python(rdflibライブラリ)によるスクリプトが定番である。

検証は少なくとも3水準で行う。件数の照合(元データの記述単位数・作成者数と、生成された個体数が合うか)、機械的な制約検査(必須プロパティの欠落、IRIの重複。SHACLという制約記述言語が使える)、そして標本の目視(変換結果をSPARQLで引き出し、原本の目録と突き合わせる)である。第6章で述べたとおり、推論は誤りも増幅するので、検証を省いた公開は危うい。

7.3 通し例: 目録と典拠レコードの連携は、RiC でどうなるか

移行の意味を最も実感しやすいのは、多くの機関がすでに実践している記述—ISAD(G)/EADの階層記述と、ISAAR(CPF)/EAC-CPFの典拠レコード—が、RiCでどう書き直されるかを、具体的なデータで見比べることだろう。第6章の衛生行政の例を素材に、従来形とRiC形を並べてみる。順序は、まず階層(記録の側の構造)、次に典拠との連携(作成者の側)である。

7.3.1 従来形:2種類の文書をID文字列でつなぐ

従来の実務では、記録の記述(ISAD(G)に基づく検索手段、日本の実務でいえば目録。符号化はEAD)と、作成者の記述(ISAAR(CPF)に基づく典拠レコード。符号化はEAC-CPF)を別々の文書として維持し、両者を識別子で結びつける。簡略化して示すところである。

```
<!-- 検索手段(EAD)。ISAD(G) 3.1.1, 3.1.2, 3.2.1, 3.2.2 に対応 -->
<archdesc level="fonds"><did>
  <unitid>P-83</unitid>
  <unittitle>衛生部文書</unittitle>
  <origination>
    <corpname authfilenumber="AR-0125">某県衛生部</corpname>
  </origination></did>
  <bioghist><p>衛生部は1948年に設置され、1994年の機構改革で
    保健福祉部に改組された。…</p></bioghist>
</dsc> <!-- 下位の記述。入れ子の深さが階層を表す -->
  <c01 level="series"><did>
    <unitid>P-83/1</unitid><unittitle>庶務系列</unittitle></did>
    <c02 level="file"><did>
      <unitid>P-83/1/12</unitid>
```

```

    <unittitle>予防接種関係綴</unittitle>
    <unitdate>昭和23年度</unitdate></did></c02>
  </c01>
  <c01 level="series"><did>
    <unitid>P-83/2</unitid><unittitle>防疫統計系列</unittitle></did>
  </c01>
</dsc>
</archdesc>

```

多段階記述の階層（フォンド→シリーズ→ファイル）は、c01・c02 という要素の入れ子で表現されている。ここに注目してほしい。上位と下位の間には参照の記載がない。XML 文書の中での位置（どの要素の中にあるか）が、親子関係の表現を兼ねているのである。第2章2.4節の言葉で言えば、階層については「位置が参照を代行している」。これは紙の目録で、字下げや配列が構造を表していたことの直接の継承である。

```

<!-- 典拠レコード(EAC-CPF)。ISAAR(CPF) に対応 -->
<cpfDescription>
  <identity><entityId>AR-0125</entityId>
    <nameEntry><part>某県衛生部</part></nameEntry></identity>
  <description><existDates>1948-1994</existDates></description>
  <relations>
    <cpfRelation cpfRelationType="temporal-later">
      <relationEntry>某県保健福祉部</relationEntry>
    </cpfRelation></relations>
</cpfDescription>

```

この二本立て自体が ISAAR(CPF) の成果であり、作成者情報を複数のフォンドで使い回せるようにした（第2章）。しかし連携の実態を見ると、いくつもの箇所がゆるい。検索手段から典拠への参照は `authfilenumber="AR-0125"` という ID 文字列であり、その ID が指す典拠レコードの存在も一意性も、機械的には保証されない。「作成者である」という関係の性質——作成か蓄積か、いつからいつまでか——は `origination` という要素名と散文 (`bioghist`) に埋まっている。後継組織への関係も、典拠側では `relationEntry` の中の文字列「某県保健福祉部」であって、相手の典拠レコードへの機械可読な参照ですらないことが多い。そして同じ事実（1948 年設置、1994 年改組）が検索手段の散文と典拠レコードの両方に書かれ、二重管理と不同期の温床になる。

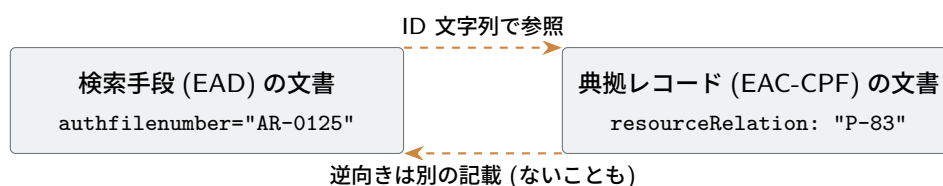
参照の向きが固定されていることも、この構造の帰結として押さえておきたい（図 7.2。参照手段の歴史的経緯は第2章2.4節）。文書ベースの標準では、関係はどちらかの文書のフィールドとして記録されるから、矢印の向きは「どの文書がそのフィールドを持つか」で決まってしまう。EAD の `origination` は検索手段→典拠の向きである。実は逆向きも用意されていて、EAC-CPF の `resourceRelation` を使えば典拠レコードから記録資源を参照できる。しかし両者は別々の文書の中の別々の記載であり、片方だけ書かれたり、内容が食い違ったりしても、それを止める仕組みはない。双方向にしたければ2回書き、整合は人手で保つほかなかった。

この設計は、関係データベースの定石とも比べられる（図 7.2(b)）。RDB では「多数の記述が1つの典拠を指す」関係を、**多の側**のテーブルに外部キー列を置いて表す。向きは意味ではなく設計の定石で固定されるが、文書ベースと違って参照整合性制約があり、存在しない典拠を指す参照はデータベース

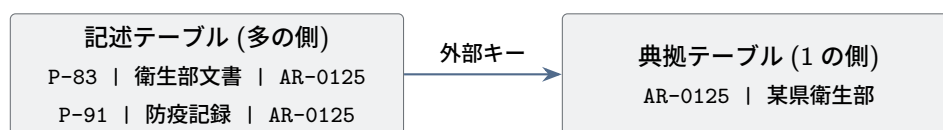
が拒否してくれる。文書ベースの ID 参照は、「外部キーはあるが整合性制約のない」状態にあたる。

RiC ではこの事情がもう一段変わる (図 7.2(c))。関係はどちらの記述の所有物でもない、グラフの独立した辺 (あるいは関係実体) になり、向きは構造の制約ではなく名前の選び方 (`rico:hasCreator` か `rico:isCreatorOf` か) という記法上の問題に退く (第 5 章 5.1.2 節)。参照先は ID 文字列ではなく IRI で、名指しの一意性は世界規模で保証される。ただし公平のために言えば、RDF は開世界仮説をとるため、RDB のような参照整合性の**自動的な強制**はない。指した先の実体が記述されていないくても誤りにはならない (「まだ書かれていないだけ」と解釈される)。存在検査が必要なら SHACL などで明示的に検査する (第 7 章工程 6)。つまり 3 つのアーキテクチャは、参照の**置き場所** (文書の中/多の側の列/独立した辺) と、参照の**保証** (なし/DB が強制/世界規模の名前 + 任意検査) の組み合わせとして整理できる。

(a) 文書ベース: 参照は文書中の記載。2 本は独立で、同期の保証がない



(b) 関係データベース: 外部キーは多の側に置く。向きは定石で固定、整合性は DB が強制



(c) RiC(RDF): 関係は独立した辺。向きは名前の選び方、参照は IRI

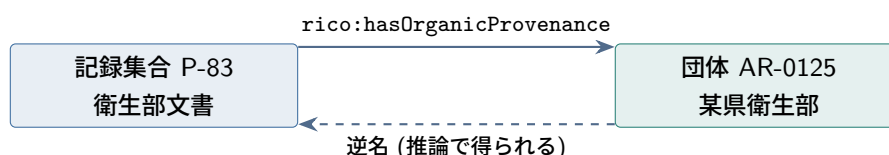


図 7.2 参照アーキテクチャの 3 段階。参照の置き場所は、文書中の記載 (a) から、多の側の外部キー列 (b)、どちらにも属さない独立した辺 (c) へと移り、向きの固定は構造の制約から記法の選択に変わる。

7.3.2 階層の移し方 — 入れ子から明示的な関係へ

まず記録の側の階層を RiC に移そう。原則は 1 つだけである。入れ子で暗黙に表現されていた親子関係を、**1 本ずつ明示的な関係 (トリプル)** にする。各記述単位は独立の実体になり、レベルは `rico:hasRecordSetType` の値 (公式語彙の個体 `rst:Fonds`・`rst:Series`・`rst:File`) で表す。

```
ex:fondsP83 a rico:RecordSet ;
  rico:hasRecordSetType rst:Fonds ;
  rico:identifier "P-83" ;
```

```

rico:name "衛生部文書" ;
rico:directlyIncludes ex:s1 , ex:s2 . # 入れ子だったものを明示

ex:s1 a rico:RecordSet ;
rico:hasRecordSetType rst:Series ;
rico:identifier "P-83/1" ;
rico:name "庶務系列" ;
rico:directlyIncludes ex:f112 .

ex:s2 a rico:RecordSet ;
rico:hasRecordSetType rst:Series ;
rico:identifier "P-83/2" ;
rico:name "防疫統計系列" .

ex:f112 a rico:RecordSet ; # 簿冊も「記録の集合」
rico:hasRecordSetType rst:File ;
rico:identifier "P-83/1/12" ;
rico:name "予防接種関係綴" ;
rico:isAssociatedWithDate [ a rico:Date ;
                           rico:expressedDate "昭和23年度" ] .

```

EAD: 入れ子=階層 (参照の記載はない)

RiC:1 本ずつ明示的な関係

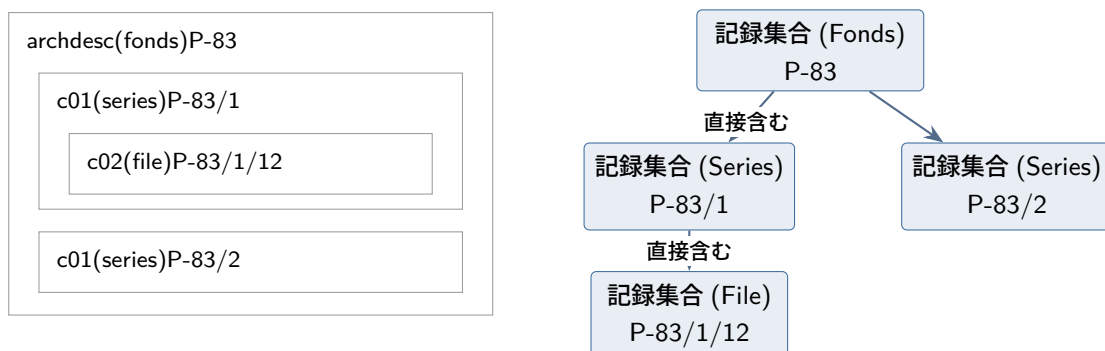


図 7.3 階層記述の移行。EAD では入れ子 (文書内の位置) が階層を表していたが、RiC では各記述単位が独立の実体になり、親子は `rico:directlyIncludes` という明示的な関係で 1 本ずつ主張される。

見た目には同じ木である。しかし性質は 3 つ変わっている。

第一に、**下位の単位が独立の実体になった**。EAD の `c02` 要素はフォンドの文書の一部でしかなく、外から直接指すことができなかった (できても文書内 ID どまりである)。RiC では簿冊 `ex:f112` が自分の IRI を持ち、典拠からも、他機関のデータからも、利用者の引用からも直接指せる。第二に、**階層が検索可能なデータになった**。「このフォンドの下全簿冊」は、EAD では文書構造をたどるプログラムの仕事だったが、RiC では `rico:includesTransitive` の推論 (第 6 章場面 1) または SPARQL のプロパティパス 1 行で得られる。第三に、**単一の木という制約が外れた**。同じ簿冊を複数の集合 (元のシリーズと、たとえば「予防接種関係」という主題別コレクション) に属させることが、関係を 1 本足すだけで表せる (第 3 章の多次元記述)。

一方で、変わらないものもある。ISAD(G) の多段階記述の規則—記述は全体から部分へ、情報は該当するレベルに置き、上位で述べたことを下位で繰り返さない—は、RiC でもそのまま良い設計である。各事実を 1 つの実体にだけ書くという原則は、むしろグラフでこそ徹底できる (上位の情報は関係をたどれば届くので、繰り返す理由が消える)。RiC-CM が「ISAD(G) の記述は RiC に包含される」と述べる (§1.6.4) のは、この意味である。階層は否定されるのではなく、**数ある関係の 1 種類**になる。

7.3.3 典拠との連携:1 つのグラフに実体として置く

次に作成者の側である。同じ内容を RiC で書くと、検索手段と典拠レコードという 2 種類の文書の区別が消え、いま作った階層と同じ 1 つのグラフに、Agent 実体とその関係として合流する。

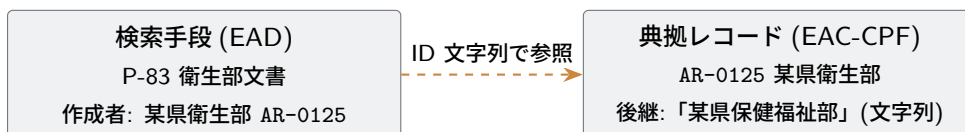
```
ex:fondsP83 a rico:RecordSet ;
  rico:hasRecordSetType rst:Fonds ;
  rico:identifier "P-83" ;
  rico:name "衛生部文書" ;
  rico:thingIsSourceOfRelation ex:prov83 .    # 出所関係を実体化

ex:prov83 a rico:OrganicProvenanceRelation ;
  rico:relationHasTarget ex:eisei ;
  rico:relationHasDate [ a rico:Date ;
                        rico:expressedDate "1948-1994" ] .

ex:eisei a rico:CorporateBody ;
  rico:identifier "AR-0125" ;
  rico:name "某県衛生部" ;
  rico:hasSuccessor ex:hokenfukushi .        # 後継も実体への参照

ex:hokenfukushi a rico:CorporateBody ;
  rico:name "某県保健福祉部" .
```

従来形:2 種類の文書 + ゆるい参照



RiC 形:1 つのグラフ。参照はすべて IRI



図 7.4 目録と典拠の連携の before/after。従来 2 つの文書に分かれ ID 文字列でつながっていたものが、RiC では実体と関係の同じグラフに置かれる。

変わった点を数えあげよう。第一に、参照がすべて **IRI による直接参照**になった。「AR-0125 という ID の典拠がどこかにあるはず」ではなく、`ex:eisei` という実体そのものを指す (従来の典拠 ID は

rico:identifier 属性として実体に残せる)。第二に、関係が**意味と期間を持つ**ようになった。「作成者」欄という書式上の位置ではなく、有機的出所という型の関係で、1948–1994 年という期間付きで主張される。第三に、後継関係が文字列から実体間の関係 (rico:hasSuccessor) になり、第6章で見た変遷の推論・検索がそのまま動く。第四に、二重管理が原理的に解消される。1948 年設置という事実は衛生部という実体に1回だけ書かれ、検索手段風の表示も典拠レコード風の表示も、同じグラフからの出力として生成される (第3章の入力と出力の分離)。

言い換えれば、「典拠レコードの管理」という仕事は RiC では消えるのではなく、**Agent 実体とその関係の管理**へと昇格する。ISAAR(CPF) が定めた典拠の内容 (名称、存在日付、履歴、関連) は Agent の属性と関係にそのまま対応し (公式クロスウォークが対応表を与える)、ISAAR の3種の関連 (階層的・時間的・連想的) は、RiC の豊富な Agent 間関係 (上下、後継、合併・分割、家族、雇用…) へ精密化される。ISAAR の管理情報 (コントロールエリア) だけは対応先が実体でなく「記述の記述」(RiC-CM 第6章) になる—典拠レコードそれ自体のメンテナンス情報は、Agent の属性ではなく記述活動の記録だからである。

7.3.4 混ざって混沌としないか — 論理の統合と、管理の分割

ここで管理の見通しを立てておきたい。従来は、目録は目録、典拠は典拠として**独立**に管理できていた。別々の文書だから、担当も、更新のサイクルも、品質管理も別々に回せた。それが1つのグラフに合流したとき、この独立性がどうなるかは、実務の関心事である。

見通しを与えるのは、**統合されるのは論理 (意味のモデル) であって、管理の単位ではない**という区別である。RiC は「検索手段と典拠レコードは同じ世界の記述であり、意味の上では1つのグラフをなす」と言っているのであって、「1つのファイル・1人の担当・1つの更新フローで管理せよ」とは言っていない。そして RDF の技術スタックには、管理を分割したまま論理を統合するための標準的な道具が、最初から用意されている。

第一の道具が**名前付きグラフ (named graph)** である。トリプルストアは実際には、トリプルを「どのグラフに属するか」というラベル付きの4つ組で格納できる (SPARQL 1.1 で標準化。直列化形式は TriG)。これを使うと、従来の文書の分けを、そのままグラフの分けとして再現できる。

```
GRAPH ex:agents {      # 典拠系:エージェント記述の担当が管理
  ex:eisei a rico:CorporateBody ;
  rico:name "某県衛生部" ;
  rico:hasSuccessor ex:hokenfukushi .
}
GRAPH ex:catalog {     # 目録系:目録担当が管理
  ex:fondsP83 a rico:RecordSet ;
  rico:name "衛生部文書" ;
  rico:hasOrganicProvenance ex:eisei .    # グラフ境界を越えて参照
}
```

典拠系のグラフと目録系のグラフは、別々に更新し、別々にバックアップし、別々に品質検査できる (SHACL の検査もグラフ単位でかけられる)。検索するときだけ、2つのグラフを合成した論理的な1つのグラフとして問い合わせる。つまり従来の「文書の独立性」は失われるのではなく、**物理的な偶然** (たまたま別ファイルだった) から**意識的な設計** (どう分割して管理するか) へと性格を変える。境界を

越える関係トリプル (上の `rico:hasOrganicProvenance`) をどこに置くかは、次に見るとおり、それ自体が設計問題である。

境界を越える**関係の記述**の帰属—目録か、典拠か、それとも独立か—については、論理のレベルでは答えがはっきりしている。**独立**である。関係は RiC の第一級の記述対象であり、実体化すれば両端を IRI で指す第三の実体になる (第 5 章 5.5.3 節)。関係がどちらかの記述の所有物でなくなったことこそ、7.3 節で見た従来形からの変更点だった。管理のレベルでは 3 つの置き方があり、決め手は**その言明を誰が作り、誰が維持するか**である。

1. **目録グラフに置く**。整理の過程で目録担当が確定する関係 (このフォンドの出所は衛生部である、など) は、目録作業の産物だから目録側に置く。「多の側が指す」という外部キーの定石と一致し、EAD からの変換で生まれる関係も自然にここに落ちる。
2. **典拠グラフに置く**。両端とも Agent の関係 (`rico:hasSuccessor`、合併・分割) は典拠作業の産物だから典拠側である。ここは迷う余地がない。
3. **独立の関係グラフに置く**。関係が独自の調査・維持のサイクルを持つ場合である。先例は確立している。RDB では多対多の関係を、どちらのテーブルにも属さない**中間テーブル**として独立させるのが定石であり、LOD には**リンクセット** (linkset, VoID 語彙) という、2 つのデータセットをつなぐリンクの集合を第三の管理単位として公開する作法が普及している。機関横断の同定リンク (`owl:sameAs`、7.5 節) は、ほぼ常にこの形をとる。シリーズシステム (次項) の CRS も、実態は「シリーズと機関の関係の台帳」を中央で独立に管理する仕組みだった。

目録と典拠をまたぐ出所関係について言えば、第一候補は 1 の目録グラフである。多くの機関では出所の確定は整理・目録作成の仕事の一部であり、言明の責任者と置き場所が一致するからである。ただし、関係を実体化して期間・根拠・確実性を持たせ、出所調査として目録更新とは別のサイクルで育てていくなら、3 へ昇格させるのが筋になる。なお直接プロパティ形 (1 本のトリプル) はどこか 1 つのグラフに入れるほかないが、実体化形は関係実体とその属性の一式がまとまって動くので、グラフを分けても壊れない。関係クラス (第 5 章 5.5.3 節) には、グラフ分割と相性が良いという利点もあるわけである。いずれにせよ、どの型の関係をどのグラフに置くかは、適用プロファイルに明記すべき規約事項である。

第二に、そもそも**同じストアに同居させる必要すらない**。IRI による参照は、参照先が同じデータベースにあることを要求しない。典拠系は国レベルの共同典拠サービスが、目録系は各文書館が、それぞれ別のサーバで管理・公開し、IRI で指し合う—これは妥協ではなく、LOD の設計思想そのものである (7.5 節)。SPARQL には複数のエンドポイントをまたいで問い合わせる仕組み (**SERVICE**) もある。

第三に、「どの記述を、誰が、どの規則に基づいて維持しているか」自体を RiC で記録できる (記述の記述、RiC-CM 第 6 章)。ISAAR(CPF) のコントロールエリアが果たしていた管理情報の役割は、ここに引き継がれる。

正直な限界も述べておく。名前付きグラフは格納と検索の仕組みであって、OWL の意味論の一部ではない。推論をどのグラフの範囲で走らせるかは処理系の設定であり (第 6 章 6.8 節)、書き込み権限や承認ワークフローに至っては標準の外で、機関の規程とシステムの実装が受け持つ。第 8 章 8.3.1 節の言葉で言えば、RiC が標準化するのは概念スキーマまでであり、内部スキーマ (格納の分割) には名前付きグラフという標準的な道具があるが、運用統制はどこまでも運用の仕事である。まとめれば、**論理は**

標準が統合し、格納は名前付きグラフで分割でき、統制は運用が受け持つ—混沌は必然ではないが、従来は文書という物理単位が黙って与えてくれた独立性を、今度は自分で設計する必要がある。

7.3.5 シリーズシステムと RiC — 60 年越しの合流

この before/after を見て、オーストラリアのシリーズシステムを思い出した読者もいるだろう。1960 年代にピーター・スコットが提唱した方式で、フォンドという固定した頂点を立てることをやめ、記録はシリーズ単位で、組織は組織で独立に記述し、両者を期間付きの関係で結ぶ。組織改編のたびに記録群の帰属を書き換えるのではなく、関係を足していく。オーストラリア国立公文書館の CRS(Commonwealth Record Series) として半世紀以上運用されてきた。

RiC はこの発想の一般化・国際標準化と読める。シリーズシステムが2種類の記述(シリーズ・機関)と期間付き関係で実現していたことを、RiC は任意の実体と関係に拡張した。逆に言えば、シリーズシステムの運用経験は、そのまま RiC の実践の型になる。整理すると次のとおりで、これは第6章の場面5(衛生行政)で実際に組んだ形でもある。

- シリーズを `rico:RecordSet` として**独立**に記述し、特定のフォンドの「中」に固定しない。
- 作成・蓄積の関係は実体化(`rico:OrganicProvenanceRelation` 等)して**期間**を持たせ、担い手が交代したら関係を**追加**する(過去の関係は書き換えない—`hasOrHad...` 命名の思想)。
- 組織の変遷は `rico:hasSuccessor`、合併・分割は `rico:wasMergedInto`・`rico:wasSplitInto` で機関側のグラフとして持つ。
- フォンドが必要なら(利用者への提示や交換のために)、**二次的な集合**として後から構成する。多重所属が使えるので、構成をめぐる立場の違い(フォンド派とシリーズ派)は排他的でなくなる。

フォンドを基礎とする ISAD(G) と、フォンドを立てないシリーズシステムは、長らく別の学派として扱われてきた。RiC のグラフはどちらも表現できる—これは折衷ではなく、両者が同じ現実の別の切り取り方だったことの、モデルによる確認である。

7.4 ドメイン知識をどこに置くか — 3つの置き場所

適用プロファイルの設計で最も判断に迷うのは、おそらく自分の分野に固有の知識—パフォーマンスアートの上演をめぐる概念、日本近世文書の文書型や村役人の役職、といったドメイン知識—を、RiC の記述のどこに持たせるかという点である。大きく言えば、既存の RiC-O 語彙の範囲で事実だけを記述する道と、RiC-O に自分の分野の語彙を追加してしまう道がある。この2つは見た目以上に性格が違うので、指針を示しておく。

実際には、置き場所は3つある。

置き場所 1: データの中(既存語彙の範囲で書く) 分野固有の情報を、既存プロパティの**値**として書く。種別は `rico:type` に文字列で、説明は `rico:scopeAndContent` や `rico:generalDescription` の散文で、主題は `Thing` への主題関係で表す。RiC-O の規則には一切手を加えない。

置き場所 2: 統制語彙として分離する(SKOS) 分野固有の**概念の体系**—文書型の一覧、役職の一覧、演目の一覧—を、RiC-O とは別の統制語彙(SKOS)として整備し、RiC-O の `*Type` クラスや主題関係から**参照**する。RiC-O の規則にはやはり手を加えない。RiC-AG の FAQ「How to use

RiC with vocabularies」が扱うのはこの道である。

置き場所 3: オントロジーの拡張 (下位クラス・下位プロパティ) 分野固有の**関係の意味**を、RiC-O の既存要素の下位として自分の名前空間に追加する (第 6 章 6.7 節)。RiC の規則体系そのものを (接ぎ木の形で) 広げる道である。

コラム:SKOS — 統制語彙を RDF で書くための標準

ここから SKOS という名前が繰り返し登場するので、まとめて説明しておく。SKOS(Simple Knowledge Organization System) は、シソーラス・件名標目表・分類表といった**統制語彙** (知識組織化体系) を RDF で表すための W3C 標準である (2009 年勧告)。オントロジー (OWL) と同じく RDF の上に載る語彙だが、役割が違う。

道具立ては少ない。語彙の 1 項目を `skos:Concept`(概念) という**個体**として IRI で立て、名前をラベルで付ける。`skos:prefLabel` が優先形 (1 言語につき 1 つ)、`skos:altLabel` が別名・別表記 (いくつでも) である。概念同士は `skos:broader`(上位の概念へ)・`skos:narrower`(下位へ)・`skos:related`(連想) で結び、意味の範囲は注記 `skos:scopeNote` で補う。語彙全体は `skos:ConceptScheme` という単位にまとめる。部品の全体を、小さな語彙の実例で示す (図 7.5 に同じ内容を図示する)。

```
vocab:kinsei a skos:ConceptScheme ;
  skos:prefLabel "近世村方文書型語彙"@ja .

vocab:register a skos:Concept ;
  skos:prefLabel "帳簿"@ja ;
  skos:inScheme vocab:kinsei .

vocab:shumon a skos:Concept ;
  skos:prefLabel "宗門人別改帳"@ja ;
  skos:altLabel "人別帳"@ja , "宗門改帳"@ja ;
  skos:broader vocab:register ;      # 上位概念へ
  skos:scopeNote "近世の戸口・宗旨調査の帳簿"@ja ;
  skos:inScheme vocab:kinsei .

vocab:kenchi a skos:Concept ;
  skos:prefLabel "検地帳"@ja ;
  skos:broader vocab:register ;
  skos:inScheme vocab:kinsei .

# データの側は、種別として概念を参照するだけ
ex:r45 rico:hasDocumentaryFormType vocab:shumon .
```

OWL と混同しやすいのは**階層の意味**である。`skos:broader` は `rdfs:subClassOf` ではない。「宗門人別改帳 `broader` 帳簿」は主題の地図としての上下であって、論理的な包含 (下位のインスタンスはすべて上位のインスタンスでもある) を主張しない。そもそも `skos:Concept` は概念という**個体**であってクラスではないから、インスタンスを持たない (`rst:Fonds` が個体でありながら SKOS の概念でもある、という第 5 章のパニングの例を思い出してほしい)。この意味論の軽

さは手抜きではなく設計方針である。軽いからこそ、図書館の件名標目表のような既存の統制語彙を、重い論理を持ち込まずにそのまま RDF 化して公開できる。

使い分けはこう覚えればよい。**世界の構造** (実体の種類と関係) は OWL で—それが RiC-O であり、**分野の概念の地図** (ものの分類・主題) は SKOS で。公開済みの SKOS 語彙も多く、国立国会図書館の件名標目 (Web NDL Authorities) や Getty の AAT (芸術・建築シソーラス) は IRI で参照できる。自前の語彙を作る前に、既存の公開語彙で足りないかを確認するのが定石である。なお、OWL の語彙と SKOS の語彙を同じデータに混ぜて書ける理由—すべてが RDF のトリプルであり、語彙は同格だから—は、第5章 5.3 節で説明した。

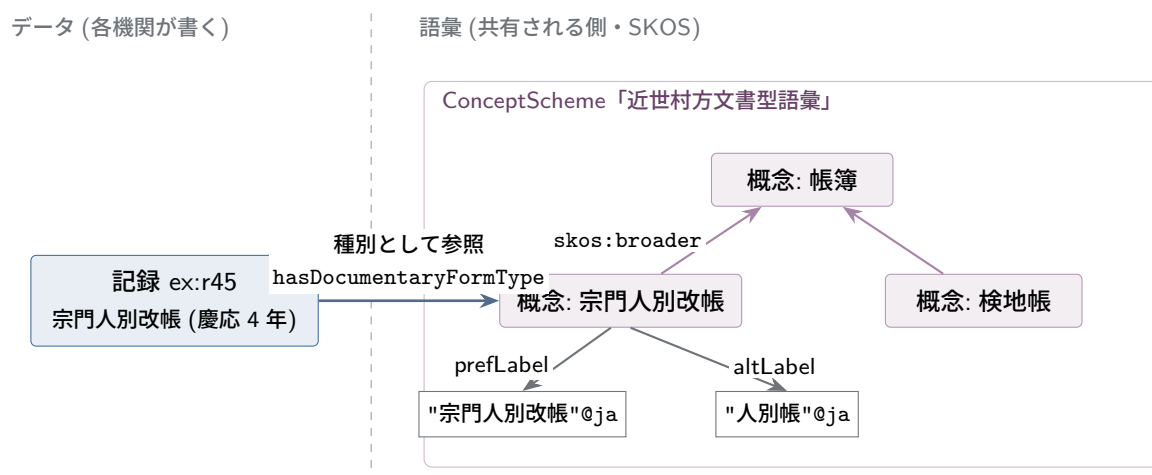


図 7.5 SKOS 語彙の構造。概念は IRI を持つ個体で、ラベル (prefLabel/altLabel) と概念間の上下 (broader) を持ち、語彙全体は ConceptScheme にまとまる。データの側は種別として概念を参照するだけでよく、表記ゆれは語彙側が吸収する。

7.4.1 使い分けの目安

判断の鍵は、その知識が「個別の事実」か「概念の体系」か「関係の種類」かである。

■**個別の事実なら、データに書く** 「この記録は 1868 年に作成された」「この上演は野外で行われた」は、特定のものについての事実であり、既存の属性・関係で書けばよい。和暦・元号はその典型で、`rico:expressedDate`(表記のままの日付「慶応 4 年」)と `rico:normalizedDateValue`(正規化した日付) という既存の仕組みがそのまま使える。新しい規則は要らない。

■**概念の体系なら、統制語彙に分離する** 「宗門人別改帳」「検地帳」「触書」という文書型の一覧、「名主」「組頭」「百姓代」という村役人の体系、能・狂言の演目一覧—これらは個別の記録についての事実ではなく、分野で共有される**概念の地図**である。こうした知識は RiC-O の中に書き込むのではなく、SKOS 語彙として独立に作り、`rico:DocumentaryFormType` や `rico:ActivityType` のインスタンス、あるいは主題として参照するのがよい。分離しておく利点は共有にある。同じ「宗門人別改帳」語彙を複数の機関が参照すれば、機関を越えた横断検索が成立するし、語彙側に多言語ラベルや上位下位関係(「宗門人別改帳は帳簿の一種」)を持たせて、分野の知識を分野の専門家が育てていける。RiC-O

側は「種別がある」ことしか知らなくてよい。ドメイン知識の大半—ものの分類—は、実はこの道で足りる。

■**関係の種類なら、拡張を検討する** 既存の関係では意味が粗すぎて、検索や推論で使いたい区別が失われる場合に、はじめて拡張を考える。RiC-AG の公式例は公証文書の「遺言者 (has testator)」である。遺言状と人物を `rico:hasAuthor`(著者) で結ぶだけでは「遺言者を探す」検索ができないので、`hasTestator` を `rico:hasAuthor` の下位プロパティとして追加する。同型の判断で、「名主として作成した」(村方文書)、「初演した/再演した」(パフォーマンスアート。`rico:performsOrPerformed` の下位) なども拡張の候補になる。ただし拡張には設計・文書化・維持の費用がかかり、RiC の改訂に追従する義務も生じる。第 6 章で述べた作法 (既存要素への接ぎ木) を守れば、拡張を知らない他機関でも `rico:hasAuthor` の水準では検索できるが、拡張の意味そのものは共有した相手にしか伝わらない。だから拡張は「本当に関係の意味が要るとき」に限り、作ったら公開して共有可能にする、が原則である。ただし正直に言えば、この「公開して共有」を支える制度はまだ存在しない。RiC-AG 自身、拡張のレジストリ (登録簿) の維持が「将来は重要になるかもしれない」と述べるにとどまり、当面は RiC-ResourceList に載せてほしいという善意ベースの運用である。この問題は次節で正面から扱う。

7.4.2 通し例: 宗門人別改帳を 3つの書き方で

抽象論では掴みにくいので、同じ事実を 3つの置き場所で書き比べる。素材は「信濃国某村の宗門人別改帳。慶応 4 年、名主の甚兵衛が作成」という近世村方文書である。

■**置き場所 1 で書くと** まず、既存語彙の値としてだけ書いてみる。

```
ex:r45 a rico:Record ;
    rico:name "宗門人別改帳" ;
    rico:type "宗門人別改帳" ;          # 文書型を文字列で
    rico:expressedDate "慶応4年" ;      # 和暦は表記のまま
    rico:normalizedDateValue "1868" ;   # 正規化した年
    rico:hasCreator ex:jinkei .

ex:jinkei a rico:Person ;
    rico:name "甚兵衛" ;
    rico:generalDescription "某村名主(嘉永6年～明治2年)" . # 役職は散文
```

これで事実が残る。和暦が `rico:expressedDate`(表記のまま) と `rico:normalizedDateValue`(正規化した値) という既存の仕組みで扱えているように、この水準で十分な部分も多い。しかし弱点も見える。「名主が作成した文書」は検索できない (その知識は散文の中に埋まっている)。文書型の検索は文字列一致だけなので、他機関が「宗門改帳」「人別帳」と書いていれば別物になる。

■**置き場所 2 で書くと** 文書型と役職を SKOS 語彙として別ファイルに切り出し、データからは参照する。

```
# --- 語彙(共有される側。分野の専門家が育てる) ---
vocab:shumon-aratame a skos:Concept , rico:DocumentaryFormType ;
    skos:prefLabel "宗門人別改帳"@ja ;
```

```

skos:altLabel    "宗門改帳"@ja , "人別帳"@ja ;    # 表記ゆれを吸収
skos:broader     vocab:register .                  # 帳簿の一種

vocab:nanushi a skos:Concept ;
  skos:prefLabel "名主"@ja ;
  skos:altLabel  "庄屋"@ja ;    # 上方での呼称も同じ概念に
  skos:scopeNote "近世村落の代表者。地域により庄屋・肝煎とも"@ja .

# --- データ(各機関が書く側) ---
ex:r45 rico:hasDocumentaryFormType vocab:shumon-aratame .
ex:nanushi45 a rico:Position ; rico:name "某村 名主" .
ex:jinbei rico:occupiesOrOccupied ex:nanushi45 .

```

「名主 (庄屋)」「宗門人別改帳 (人別帳)」という分野の知識は語彙側に 1 回だけ書かれ、どの機関のデータからも参照できる。表記ゆれは語彙が吸収するので、「人別帳」で検索しても同じ概念に着地する。複数の機関が同じ語彙を参照した瞬間に、機関横断の検索が成立する。役職が Position 実体になったので、「甚兵衛が名主だった」ことも構造化された。

■置き場所 3 で書くと それでも書けていないことが 1 つ残っている。「この帳面を、甚兵衛が名主として (村の代表者の職務として) 作成した」という、作成関係そのものの意味である。これを検索や推論で使いたいなら拡張する。

```

# --- 拡張(1回だけ定義し、文書化して公開する) ---
myo:hasCreatorAsVillageHead
  rdfs:subPropertyOf rico:hasCreator ;
  rdfs:label "名主として作成された"@ja .

# --- データ ---
ex:r45 myo:hasCreatorAsVillageHead ex:jinbei .

```

これで「名主として作成された文書の一覧」が 1 つのプロパティで検索でき、しかも下位プロパティのロールアップ (第 6 章) により、この拡張を知らない相手にも rico:hasCreator の水準では見える。逆に言えば、ここまでの必要——「作成した」では粗すぎて困る具体的な検索・推論の要求——がないなら、置き場所 2 で止めるのが正しい。

7.4.3 迷ったときの順序

以上から、指針は次の順序になる。まず置き場所 1 で書けないか (たいてい書ける)。次に、それが分野で共有される概念なら置き場所 2 へ切り出す。関係の意味がどうしても必要なときだけ置き場所 3 へ進む——そして拡張する場合も、いきなり作る前に、RiC-O にすでにないか (CM より語彙は多い)、EGAD に提案できないか (FAQ が推奨する) を確かめる。この順序は「最小の手段を選ぶ」原則であると同時に、「ドメイン知識はできるだけ RiC-O 本体から分離し、共有できる形に置く」原則でもある。RiC はアーカイブズ記述の共通語であって、各分野の百科事典ではない。分野の知識は語彙や拡張として分離して公開すれば他機関の資産になるが、散文やローカルな独自規則に埋め込めば、その機関の中に閉じる。

表 7.2 ドメイン知識の3つの置き場所

	1. データの中	2. 統制語彙 (SKOS)	3. 拡張
向く知識	個別の事実	概念の体系 (分類)	関係の意味
RiC-O への変更	なし	なし (参照のみ)	接ぎ木で追加
例	和暦の日付、上演の場所	文書型・役職・演目の一覧	遺言者、名主として作成
費用	最小	語彙の整備・維持	設計・文書化・改訂追随
共有性	低い (散文は再利用不能)	高い (語彙自体が資産)	公開すれば共有可能

7.5 共有の基盤は誰が用意するのか — 識別子と拡張のインフラ問題

前節の「共有できる形に置く」という指針には、暗黙の前提がある。共有を受け止める**基盤**が存在する、という前提である。この前提は現在ほとんど満たされていない。本節はこの不都合な事実を確認し、その上で、基盤なしで RiC-O が成り立つ範囲を示す。

7.5.1 何が欠けているか

第一に、拡張の共有は制度化されていない。前節で見たとおり、AG が推奨するのは「文書化して公開し、ResourceList に一項目を加える」ことまでであり、拡張の登録・審査・調停・版管理を担う仕組み—プログラミング言語のパッケージレジストリや、図書館界の語彙管理に相当するもの—は存在しない。同じ分野 (たとえば日本近世文書) で2つの機関が別々に拡張を作れば、それらを突き合わせて統合する手続きはどこにもない。

第二に、より深刻なのは識別子の共有基盤である。機関横断の網という RiC の眼目は、「同じものには同じ IRI (または対応付けられた IRI) が使われる」ときに初めて成立する。ところが図書館界に VIAF (各国の著者典拠を束ねる国際基盤) や ISNI があるのに対し、アーカイブズ界にこれへ相当するものは少ない。作成者 (エージェント) については、米国を中心とする共同典拠事業 SNAC、フランスの国立公文書館が公開するエージェント典拠データ、そして事実上のハブとして使われている Wikidata (誰でも編集・参照できる共有の構造化データ基盤) が部分的に機能しているが、網羅性も国際的な調整も VIAF の水準には遠い。さらにアーカイブズ固有の実体—活動、規則、職位、そして記録集合そのもの—に至っては、図書館典拠の守備範囲の外であり、共有識別子の仕組みは構想段階ですらない。各機関は自前のドメインで IRI を鑄造するほかになく (§7.2)、その結果、同一人物・同一組織が機関の数だけ別の IRI を持つことが常態になる。事後的に owl:sameAs や skos:exactMatch で対応付けることは技術的には可能だが、その照合 (同定) 作業こそが高コストで不確実な仕事であり、誰が担うのかは決まっていない。

7.5.2 共有基盤なしで RiC-O はどこまで成り立つか

結論は「半分は成り立ち、半分は成り立たない」である。RiC-O が約束する価値を2つに分けると見通しがよい。

機関の内側で閉じる価値—記述の構造化、多次元の記述、入力と出力の分離、機関内での検索と推論—

は、共有基盤なしで成立する。自機関の IRI だけで閉じたデータセットでも、第6章で見た推論の利益はすべて得られる。現在の先行実装 (フランス国立公文書館を含む) は、実際にはほぼこの水準で動いている。単独の機関にとって RiC-O を採用する理由は、共有基盤の有無とは独立に存在する。

機関を越えてつながる価値—文脈の網が機関境界をまたいで伸びること、LOD としての相互接続—は、共有識別子か事後照合なしには成立しない。そして RiC の一次資料はこの問題を解いていない。IRI の設計と維持は各機関の責任とされ (LOD の一般原則の再確認にとどまる)、AG も「典拠レコードや統制語彙が整っている機関は有利だ」と述べるにとどまる。つまり RiC は**語彙の共通化** (同じ概念に同じ名前) までを提供し、**指示対象の共通化** (同じ人物に同じ識別子) は制度の側に残した。ここは標準の欠陥というより守備範囲の線引きだが、線の外側が空き地のままであることは事実である。

歴史的な先例はある。図書館界でも、目録規則と MARC の標準化が先にあり、VIAF のような国際基盤が実際に動き出すまでには数十年を要した。標準が先行し、インフラが後から (標準があるからこそ) 成立する、というのが通常の順序ではある。アーカイブズ界で同じ順序が再現される保証はないが、RiC という共通語彙の存在は、将来の「アーカイブズ版 VIAF」の前提条件を1つ満たしたと言える。

7.5.3 当面の実務解

インフラの成立を待たずにできることは、方向としては1つに集約される。**自機関の IRI を鑄造しつつ、既存の外部識別子への対応付けを最初から記録しておく**ことである。エージェントには Wikidata・ISNI・VIAF・国内典拠 (国立国会図書館の Web NDL Authorities など) の ID があれば対応付けを張る。場所には既存のジオ語彙や地名典拠を参照する。対応付けは owl:sameAs 等のトリプル数本であり、将来どんな共有基盤が現れても、この対応付けが橋になる。逆に、対応付けのない孤立 IRI の集合は、基盤が現れたときに照合作業を丸ごと負うことになる。前節の表 7.2 の言葉で言えば、これは「共有性への投資を、インフラの成立に先行して行っておく」ことに当たる。

本書はこの識別子・拡張インフラの不在を、第3章で述べた表現の一意性の問題と並ぶ、RiC のもう1つの未解決の構造的リスクとして明記しておく。前者が「書き方が収束するか」の問題だとすれば、こちらは「書かれたものがつながるか」の問題である。どちらも標準文書の内側では解決せず、コミュニティと制度の側の仕事として残されている。

7.6 日本語データの実務 — 和暦・字体・多言語

一次資料の例はヨーロッパの資料が中心なので、日本語データに固有の論点をここでまとめておく。どれも RiC が新しく生んだ問題ではないが、RDF に載せるときの書き方には定石がある。

7.6.1 多言語・多表記の名称

RDF のリテラルには言語タグ (BCP 47) を付けられ、同じ述語に複数並べてよい。

```
ex:eisei rico:name "某県衛生部"@ja ,
                  "Boken Eiseibu"@ja-Latn ,      # ローマ字翻字
                  "ボウケンエイセイブ"@ja-Kana .  # 読み
```

日本の目録実務で必須級の「読み (ヨミ)」は、カタカナ表記に @ja-Kana タグを付けて併記するのが

簡便法である。名称の種別 (本名・筆名・旧姓・翻字…) や使用期間まで管理したい場合は、名称そのものを実体化する精密法がある。RiC-O には `rico:AgentName` クラスが用意されている。

```
ex:saito a rico:Person ;
    rico:hasOrHadAgentName ex:saitoName1 .
ex:saitoName1 a rico:AgentName ;
    rico:textualValue "齋藤茂吉"@ja ;
    rico:type "本名" .
```

簡便法と精密法の使い分けの判断は、日付や種別のとき (第 5 章 5.5 節) と同じである。

7.6.2 旧字体・異体字

「齋藤/斎藤/斉藤」「廣島/広島」のような字体のゆれは、**文字列の統一で解くのではなく、IRI の同一性で解く**のが RiC 流である。実体は 1 つ (`ex:saito`) で、表記は名称の複数形 (上の `rico:AgentName` や SKOS の `altLabel`) として吸収する。識別を文字列一致に頼らずに済むのは、参照を IRI 化したことの直接の見返りである (第 2 章 2.4 節)。そのうえで、データ投入時には Unicode 正規化 (NFC) を統一しておく。見た目が同じで符号位置が違ふ文字は、気づかれないまま検索の取りこぼしを生む。旧字と新字の検索時の同一視 (「広島」で「廣島」も見つける) は、データではなく索引・アプリケーション層の仕事である。

7.6.3 和暦の正規化

日付の二本立て (原表記 + 機械可読値。第 5 章) が最も活きるのが和暦である。`rico:expressedDate` には資料の表記をそのまま、`rico:normalizedDateValue` には機械可読値を書く。RiC-O 自身は後者を「公的な技術標準に基づく機械可読表記」とだけ定めており、実際上の選択肢は ISO 8601 と、その拡張である **EDTF** (Extended Date/Time Format。ISO 8601-2 として標準化) である。EDTF は目録実務で頻出する「あいまいな日付」を書ける。不確実は 1868?、およそは 1868~、年代までなら 186X、期間は 1948-04/1949-03 (昭和 23 年度) である。

```
ex:f112 rico:isAssociatedWithDate [ a rico:Date ;
    rico:expressedDate "昭和23年度" ;
    rico:normalizedDateValue "1948-04/1949-03" ] .

ex:r1 rico:isAssociatedWithDate [ a rico:Date ;
    rico:expressedDate "慶応4年間4月" ;
    rico:normalizedDateValue "1868-05/1868-06" ] .
```

換算には 2 つの注意がある。第一に、明治 5 年 12 月 2 日 (グレゴリオ暦 1872 年 12 月 31 日) 以前の和暦は太陰太陽暦 (天保暦など) なので、月日まで正規化するなら暦の換算が必要である (閏月はその典型で、上の慶応 4 年閏 4 月はグレゴリオ暦の 5 月下旬~6 月中旬にまたがる)。年単位の近似 (慶応 4 年 ≈ 1868) で足りる場面は多いが、旧暦 12 月は翌西暦年にかかることがある。第二に、改元は年の途中で起きる (慶応 4 年 9 月 8 日=明治元年)。原表記を保存してあれば、正規化の方針を後から変えることもできる—二本立ての利点である。

7.6.4 外部典拠との接続

日本語の人物・団体なら、国立国会図書館の Web NDL Authorities が IRI 付きの典拠を提供しており、VIAF にも連携している。自機関の Agent 実体からこれらへ `owl:sameAs` で橋を架けるのが、7.5 節で述べた「当面の実務解」の日本語版である。

7.7 個人情報とアクセス制限 — どこまで公開するか

RiC で記述が精密になるほど、個人に関する構造化データは増える。しかも本書で利点として紹介してきた仕組み—IRI による名寄せ、逆プロパティ、推移閉包—は、そのままプライバシーのリスク要因でもある。散らばっていたために事実上見えなかった情報が、推論と検索で集約されるようになるからである。LOD として公開したデータは複製・収集され、公開の取り消しは事実上効かない。だから「どこまで書くか」と「どこまで公開するか」を、別の問いとして設計する必要がある。

道具立ては4つある。第一に、記録への**閲覧制限の記述**。簡便法は `rico:conditionsOfAccess` に散文で書くこと、精密法は利用規程そのものを `rico:Rule` 実体にして `rico:isOrWasRegulatedBy` で結ぶことである（規程が変われば1箇所の更新で済む）。第二に、**公開の制御はグラフの分割で行う**（7.3.4 節）。内部グラフと公開グラフを分け、公開グラフは内部データからの**生成物**として作る—すべてを1つに書いてから隠すのではなく、公開してよいものを選んで出す設計にする。制限の**記述**（機械可読になった）と制限の**執行**（アクセス制御の実装）は別物であり、後者は名前付きグラフ単位の権限管理などシステムの仕事である。第三に、**存命個人の实体は最小限に**。名称と役割程度にとどめる、あるいは個人を立てず職位（`rico:Position`）までで止める、という選択がある（第4章4.5節の階層がここでも役立つ。なお記述者であるアーキビスト自身の作業履歴も個人データであり、同じ配慮の対象になる）。第四に、削除要請への対応は、IRI 自体は残して内容を消す方式（いわゆる tombstone）が LOD の慣行である。IRI まで消すと、それを参照していた外部データを巻き添えにする。

日本の文脈では、個人情報保護法・情報公開法・公文書管理法の下での利用制限区分を `rico:Rule` として機械可読に記述できることが、RiC の実務的な貢献になり得る。閲覧審査の根拠規程と対象記録の対応がグラフとして残れば、審査の一貫性の検証も、制限期限の到来の検出（第6章の検索）も自動化の対象になる。

7.8 シナリオ B: 新規記述のワークフロー

RiC ネイティブの記述では、作業の単位が「1つの検索手段を書き上げる」ことから「実体を登録し、関係を張る」ことに変わる。受入（アクセスション）から公開までを実体の言葉で言い直すと、次のようになる。

1. **受入の時点**で、移管元の団体（Agent）、移管という活動（Activity/Event）、受け入れた記録群（Record Set）を登録し、関係で結ぶ。この時点では記録集合1個と関係数本の粗い記述でよい。
2. **整理の過程**で、編成（シリーズ、ファイル）を記録集合の入れ子として作り、内容から判明した作成者・活動・場所を実体として追加し、関係を張る。既存の典拠（他機関・国レベルの典拠、Wikidata 等）に同じ実体があれば、自前で作らず参照するか、対応関係を張る。

3. **記述の各時点**で、誰がいつ何を根拠に記述したか (記述の記述。RiC-CM 第 6 章) を、システムが自動記録するのが理想である。
4. **公開**は、蓄積された実体と関係のネットワークからの出力生成である。従来型の検索手段の体裁も、この出力の 1 つとして生成できる。

最初の一步の水準も AG の Getting Started が示している。RiC に必須項目はないが、ほとんどの実体は名称と識別子を持つと役立つこと、ISAD(G) が必須とした 6 要素 (レファレンスコード・タイトル・作成者・日付・数量・記述レベル) が RiC でも出発点として適切なこと、範囲内容や歴史のような散文はそのまま持ち込んだ上で、主題やイベントを構造化した関係として**追加**していけばよいことである。英国国立公文書館の実目録を RiC-CM に写す通し例が、使った実体・属性・関係の一覧つきで掲載されており、雛形として使える。

重要な発想の転換は、**記述が「完成」しなくなる**ことである。検索手段という完成品の代わりに、常に成長し改訂されるネットワークがあり、出力はその時点のスナップショットになる。版管理と改訂履歴 (それ自体が記述の記述である) の重要性が増す。

7.9 道具箱

2026 年時点で実務に使える主な道具を挙げる (RiC コミュニティが共同管理する一覧 RiC-ResourceList も参照)。

- **オントロジーの閲覧**: RiC-O 公式の HTML ドキュメントと CSV 一覧。本格的に読むなら Protégé (スタンフォード大学の無償オントロジーエディタ) で RDF ファイルを開くと、階層や公理を対話的にたどれる。
- **変換**: RiC-O Converter (EAD/EAC-CPF から。フランス国立公文書館の 3 万件超の EAD 検索手段を RiC-O 1.1 データセットに変換した実績を持つ 3.0.0 が公開されている)、rdflib (Python)、XSLT。マッピングの定義には RiC-AG 付属の公式クロスウォークが出発点になる。
- **格納と検索**: トリプルストア。GraphDB (商用、無償版あり、推論に強い)、Apache Jena Fuseki (オープンソース) が定番である。いずれも SPARQL エンドポイントを提供する。
- **検証**: SHACL 処理系 (GraphDB 内蔵、pySHACL など)。
- **可視化**: RiC 専用の可視化ツールはまだなく、RDF 汎用の道具を使い分ける。対象がオントロジー自体なら、WebVOWL (OWL ファイルを読み込むとクラスとプロパティの図を描くウェブツール) や Protégé の可視化プラグインで RiC-O の構造を眺められる。対象がデータなら、GraphDB の Visual graph 機能 (SPARQL 結果や近傍をインタラクティブなグラフとして表示)、LodView・LodLive のような Linked Data ブラウザ (IRI をたどりながら近傍を可視化) が手軽である。検索インタフェースとしては、Sparnatural (SPARQL を書かずに視覚的に問い合わせを組み立てるツール) を RiC-O データに載せた例がフランス国立公文書館関係のデモで公開されている。1 つ注意として、アーカイブズのグラフは巨大になるため「全体を 1 枚に描く」ことは実用にならない。起点を決めて近傍を数ホップ表示する使い方 (本書の図がしているのもこれである) が現実的である。最新のツール状況は RiC-ResourceList で確認できる。
- **記述システム**: docuteam packer/curator (スイス。RiC-O 対応のパッケージングツール) など、対

応製品が少しずつ現れている。既存の主要目録システム (AtoM、ArchivesSpace) の RiC 対応について、RiC-AG の FAQ は率直な見通しを述べている。ソフトウェア開発は EGAD の直接の守備範囲外であること、AtoM の基盤であるリレーショナルデータベースは RDF/OWL 系のデータ形式と相性の課題があること、そして本格的なアプリケーションよりも API(プログラムから機能やデータを呼び出すための窓口) 群の整備のほうが現在のオープンソース生態系では実り多いかもしれないこと、である。導入判断の際には RiC-ResourceList で最新状況の確認が必要である。

コラム: ツールが揃うまで — Excel で RiC を書く

現場の実感として、こう思った読者は多いはずである。ISAD(G) の目録は、専用システムがなくても Excel で書けた。1 行が 1 記述単位、列が記述要素、そして階層は「親 ID」の列に上位のレファレンスコードを書けば表現できた。RiC 対応のツールが揃わない現状でも、同じように手を動かす方法はある。

従来の台帳は、たとえばこういう 1 枚の表である。

参照コード	タイトル	作成年代	記述レベル	親の参照コード
P-83	衛生部文書	1948–1994	フォンド	
P-83/1	庶務系列	1948–1994	シリーズ	P-83
P-83/1/12	予防接種関係綴	昭和 23 年度	ファイル	P-83/1
P-83/2	防疫統計系列	1948–1994	シリーズ	P-83

これで目録が書けた理由を確かめておく。木構造では親が**ちょうど 1 つ**だから、つながりの記述が「親の参照コード」という **1 列**で済んだのである。行が実体、列が属性、参照は ID—第 2 章の言葉で言えば、現場の Excel 台帳はすでに立派な「表形式のグラフ記述」だった。RiC で変わるのは関係の側である。関係が多対多・型付き・期間付きになり、**1 列に収まらなくなる**。

答えは、列を増やすことなく、**シートを増やす**ことである。第一に、実体の種類ごとに 1 シートを立てる (記録集合シート、エージェントシート、場所シート…)。行は実体 1 つ、先頭に必ず ID 列を置く。第二に、関係を独立の**関係シート**に出す。行は関係 1 本で、「源 ID・関係の型・先 ID・開始・終了・根拠」の列を持つ。

記録集合シート

ID	種別	名称	直接含まれる先
P-83	Fonds	衛生部文書	
P-83/1	Series	庶務系列	P-83
P-83/1/12	File	予防接種関係綴	P-83/1

エージェントシート

ID	クラス	名称
AR-0125	CorporateBody	某県衛生部
AR-0230	CorporateBody	某県保健福祉部

関係シート				
源 ID	関係の型	先 ID	開始	終了
P-83	hasOrganicProvenance	AR-0125	1948	1994
AR-0125	hasSuccessor	AR-0230		

多対多は行が増えるだけで書ける。そして関係シートの形をよく見てほしい。源・型・先という3列の**積み上げ型**—第5章のコラムで見た、トリプルの表そのものである。気づいた読者もいるだろうが、これは RDB の中間テーブルであり、RiC の関係実体 (第5章 5.5.3 節) であり、7.3.4 節の「独立の関係グラフ」の表計算版である (親が1つに決まっている素直な階層だけは、上のように実体シートの1列に残す簡便法で足りる)。しかも多くの機関の台帳には、すでに作成者列や関連資料列という形で「アドホックな多重関係」が生えていたはずで、それらを関係シートへ移すのは正規化の作業にすぎない。この形の Excel は、グラフの**編集フロントエンド**として十分に機能する。

機械変換に耐えるための規律は5つある。(1)1行1実体・1行1関係、1セル1値 (複数値はコンマで詰めず行を分ける)。(2)見た目に意味を持たせない (結合セル・色・字下げは変換で消える)。(3)ID を全行に与え、参照は名前でなく ID で書く。(4)列名を固定し、「列名→ RiC-O のプロパティ」の対応表を先頭シートに置く (これがそのままミニ適用プロファイルになる)。(5)日付は Excel の自動変換に注意し、和暦・年度は文字列として持つ (7.6 節)。

ここまで整っていれば、RiC-O への変換は rdfib の短いスクリプトや OpenRefine で機械的にできる (次節の演習がまさにこの形である)。そして将来、記述システムが成熟したときも、この規律で育てた台帳は最小のコストで移行できる—シナリオ C の「将来に備えたデータの規律」を、表計算の上で実践していることになるからである。失われたのは「木ならば1列で書けた」という幸運だけで、行指向の作業様式そのものは、RiC と両立する。

7.10 最初の一步の例 — 演習として

学習目的なら、機関規模の計画は要らない。次の演習を勧める。

1. 手近な記録群 (ゼミ資料、家族文書、サークルの文書箱) を選び、実体を10個前後洗い出す (記録集合 2~3、記録 2~3、人・団体 2~3、活動 1~2、規則や場所があれば加える)。
2. 紙の上でグラフを描く (本書の図の見よう見まねでよい)。どの関係を使うかを RiC-CM §5.6 の一覧から選ぶ。
3. それを Turtle で書き下ろす (第5章の例を雛形に。IRI は ex: で仮置きしてよい)。
4. 無償のトリプルストア (Fuseki) に読み込ませ、SPARQL で「この集合に含まれる記録の一覧」「この人物に関わる実体の一覧」を出してみる。
5. 余力があれば GraphDB で推論を有効にし、書いていないトリプル (逆向き、推移閉包) が検索に現れることを確かめる。

この一巡で、本書の第4~6章の内容が手を動かす形で確認でき、「RiC の記述とは実体と関係のデータを作ることだ」という感覚が身につく。

現時点の限界を正直に

RiC-AG の草案公開で、実践の手引きは大きく前進した。それでもなお、AG は自らを草案かつ「生きた文書」と位置づけてフィードバックを募っている段階であり、意図的に「段階的な手順書」であることを避けてもいる。記録集合の粒度の目安や必須とすべき属性の下限のような「答えの表」は今後も出ない見込みで、AG が与えるのは事例と判断材料である。また現状の事例は英国・北欧・スペインの資料に偏っていることを AG 自身が認めており、日本の実務への当てはめは検証が要る。本章の手順は AG と先行事例から組み立てた現時点の再構成であり、AG の改訂や国内の適用指針が出れば、それに従って更新されるべきものである。学習者としては「確定した正解を暗記する」のではなく、「公式文書がどこまでを定め、どこからが実装の判断かを見分ける」読み方を身につけることが、いちばん息の長い投資になる。

確認問題 (ヒントは付録 A.5)

- 問 7.1** 架空の小さな文書館を想定し、適用プロファイルの最小構成 (使う実体、実体化する関係、IRI 方針) を箇条書きで設計せよ。
- 問 7.2** 本章 7.3 節の EAD 断片のうち、シリーズ「防疫統計系列」だけを Turtle に手で変換せよ (親フォンドとの関係も含めること)。
- 問 7.3** 名前付きグラフを使って典拠系と目録系を分割するとき、グラフ境界を越える関係トリプルをどちらに置くか。理由とともに決めよ。
- 問 7.4** 「昭和 8 年度」という日付を、原表記 (`rico:expressedDate`) と EDTF の機械可読値 (`rico:normalizedDateValue`) の 2 本立てで書け。
- 問 7.5** 存命個人が作成者である記録群を LOD 公開する場合の設計方針を、本章 7.7 節の道具立てから 3 つ選んで述べよ。

第 8 章

どこまで技術を学ぶ必要があるか

「RiC をやるにはプログラミングができないといけないのか」—最もよく聞かれる問いに、本章で答える。結論を先に言えば、**役割による**。全員に必要なのは概念の理解であり、技術の実装は分業できる。ただし分業が成り立つためには、境界を越えて会話できる程度の共通言語が要る。本章はその共通言語の目録である。

8.1 役割別の必要水準

表 8.1 役割別に見た技術素養の目安 (◎=習得すべき、○=読める・使える程度、△=概要を知っていればよい、—=不要)

要素技術	目録担当	データ設計担当	開発者	管理職
RiC-CM の概念 (実体・関係)	◎	◎	○	○
Turtle を読む	○	◎	◎	△
Turtle を書く	△	◎	◎	—
SPARQL(読む・簡単な検索)	○	◎	◎	△
OWL の公理を読む	—	○	◎	—
XML/EAD	○	◎	◎	△
IRI・永続識別子の設計	△	◎	◎	○
トリプルストアの運用	—	△	◎	△
スクリプト (Python 等)	—	○	◎	—
SHACL などの検証技術	—	○	◎	—

「目録担当」の列を見てほしい。書く技術はほとんど要らない。必要なのは、概念モデルを理解すること、システムが出力するデータ (Turtle) や検索式 (SPARQL) を読んで意味が分かること、そして自分の記述判断 (記録か記録部分か、実体化するか) がデータにどう反映されるかを想像できることである。EGAD 自身、複雑さはユーザインタフェースが覆い隠すべきだと述べている (RiC-CM §1.3)。一方「データ設計担当」—機関の適用プロファイルを作り、マッピングを定義する人—には、読み書き両方の素養が要る。アーカイブズ学と情報技術の両方が分かるこの中間人材が、RiC 実装の成否を握る。

8.2 要素技術の地図

第5～7章に登場した技術を、学ぶ順序を意識して整理する。

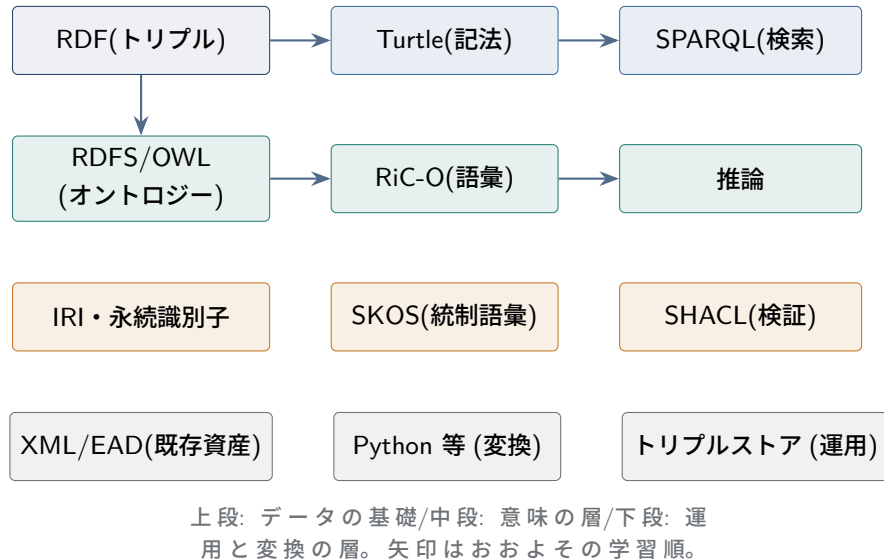


図 8.1 RiC 実装を支える要素技術の地図。すべて W3C 標準または広く普及した技術であり、アーカイブズ専用の特殊技術はない。

各技術の要点と、アーカイブズ実務との接点を短く述べる。

RDF と Turtle 第5章で学んだ。データの原子が「主語・述語・目的語」であること、ものには IRI で名前を付けること。この2点が腹に落ちていれば十分で、構文の細部は都度調べればよい。

SPARQL RDF 用の検索言語。SQL に似た見た目を持つ (SELECT ... WHERE) が、表ではなくグラフのパターン照合である。目録担当も「読める」と、公開エンドポイントでの調べ物や、開発者への要望の伝達が格段に楽になる。

RDFS/OWL 語彙を定義する側の言語。使う (RiC-O を利用する) だけなら深入り不要だが、subClassOf、domain/range、inverseOf、推移性、プロパティ連鎖の5語は第6章の推論を理解する最低限として押さえない。

IRI と永続識別子 LOD の土台。DOI や ARK などの永続識別子の考え方、「一度公開した識別子は変えない・消さない」という運用規律を含む。技術というより制度設計であり、アーキビストの職能と地続きである。

SKOS 統制語彙 (シソーラス) を RDF で表す標準。RiC-O の *Type クラスのインスタンス群は SKOS 語彙として整備することが想定されている (公式の Fonds などの個体も SKOS concept を兼ねる)。件名標目や資料種別の管理経験がそのまま活きる領域である。

SHACL データが機関のルール (必須項目、値の形式) を満たすか検査する制約言語。OWL の推論 (第6章) が「意味を広げる」向きの技術であるのに対し、SHACL は「形を締める」向きの技術で、品質管理の道具である。

XML/EAD と変換スクリプト 既存資産からの移行の道具。EAD の構造理解は変換設計の前提であり、

Python(rdfliib) や XSLT は変換の実装手段である。

トリプルストア RDF 専用のデータベース。SPARQL エンドポイントと推論機能を提供する。運用(バックアップ、性能、アクセス制御)は通常の情報システム運用と同種の仕事である。

8.3 リレーショナルデータベースの経験はどう活かせるか

RDB と SQL に触れた経験のある読者は、その経験の大部分をそのまま活かせる。対応と相違を整理する。

表 8.2 RDB の概念と RDF/RiC-O の概念

RDB	RDF/RiC-O	注意点
テーブル定義	クラス	個体は複数クラスに属せる
行	個体 (資源)	
列	データタイププロパティ	同じプロパティを複数回使える (繰り返し可)
外部キー・結合	オブジェクトプロパティ	向きがあり、逆プロパティが定義される
主キー	IRI	世界規模で一意。公開後は不変が原則
スキーマ (制約)	オントロジー (公理)+SHACL(制約)	OWL の公理は制約ではなく推論規則
NULL(値なし)	トリプルの不在	閉世界と開世界の違い (第 5 章コラム)
SQL	SPARQL	発想は近い。再帰的な探索は SPARQL+ 推論が得意
正規化 (実体の分離)	実体化 (作成者・種別のクラス化)	発想はほぼ同じ。RDB 設計の勘がそのまま使える

とりわけ「正規化」の経験は貴重である。「作成者名を各行に文字列で繰り返し書くのではなく、作成者テーブルに分離して ID で参照する」という RDB 設計の定石は、RiC が属性を実体化する動機 (RiC-CM §3.3) と同じものである (手順の具体例は第 2 章 2.4 節のコラム)。多対多の関係を結合表で表し、その表に「役割」のような列を足す設計も、RiC の関係の実体化とそのまま対応する。ER 図を描いた経験があれば、RiC-CM の実体・属性・関係の章は既視感をもって読めるはずである。逆に、RDB 経験者が戸惑うのは開世界仮説と「スキーマがデータを拒否しない」ことの 2 点であり、そこだけ意識して学び直せばよい。

8.3.1 従属性・独立性の概念はどう対応するか

データベース理論の従属性 (dependency) をめぐる概念群も、RiC の読解にそのまま使える。

■**存在従属と弱実体** ER モデルでは、他の実体が存在しなければ存在できず、同一性 (キー) も親実体に依存する実体を弱実体 (weak entity)、その親子を結ぶ関係を識別関係 (identifying relationship) と呼ぶ。RiC-CM が記録資源の 3 区分を説明する言い回し—「記録の同一性は記録自体に由来する。記録部分の同一性は、属する記録に依存して派生する。記録集合の同一性は、メンバーに依存して派生する」 (§2.2.2)—は、この概念の言い換えとして読める。記録は強実体、記録部分は記録を親とする弱実体に相当し、記録集合はメンバーに存在従属する (ただし親が複数という点で古典的な弱実体より緩い)。具現

化も「記録は少なくとも1つの具現化を持たなければ存在しない」という存在条件で記録と結ばれている。RiC-Oではこの種の従属がOWLの基数制約として書かれる場合がある。たとえば `rico:Proxy` は「ちょうど1つの記録資源を代理し (`rico:proxyFor`)、ちょうど1つの記録資源の中にある (`rico:proxyIn`)」と公理化されており、代理する相手なしには意味をなさない存在従属実体である。ER設計で弱実体を見分ける勘は、RiCのどの実体が単独で立ち、どの実体が他に寄りかかるかを見分ける勘として、そのまま通用する。

■**関数従属と一意性** 正規化理論の関数従属 (functional dependency: X の値が決まれば Y の値が一意に決まる) に相当する宣言は、RiC-Oにはほとんどない。OWLには関数的プロパティ (値を高々1つに限る) という道具があるが、RiC-O 1.1はこれを使っていない。名称も日付も作成者も原則として繰り返し可能であり、「高々1つ」を強制しない。これは設計の欠落ではなく、アーカイブズ記述の性質 (別名・改名・複数の作成者・不確かな日付の併記が常態であること) と、開世界仮説の帰結 (同じものが別のIRIで二度登場するかもしれず、値の一意性を論理で強制すると意図しない同一視の推論を招くこと) を踏まえた選択と読める。したがって「識別子はレコードごとに一意」のような業務上の関数従属は、オントロジーではなくSHACLなどの検査の層で守ることになる。RDBでスキーマが担っていた整合性維持の仕事が、RiCの世界では公理 (推論) と制約 (検査) という2つの層に分離される—この対応を押さえておくと、両者の役割分担が見通しよくなる。

■**データ独立性** ANSI/SPARC以来の「データ独立性」(物理的な格納方式や論理スキーマの変更から応用を隔離する) という発想も、RiCの設計に対応物を持つ。入力と出力の分離 (第3章) は提示独立性の獲得であり、記録 (知的内容) と具現化 (担体上の記録) の分離は、内容の記述を担体・フォーマットの変転から隔離する仕組みである。フォーマット移行が起きても記録の記述は書き換えずに済み、新しい具現化を追加するだけでよい。RDBの教科書で学ぶ「変わりやすいものから変わりにくいものを切り離す」という設計原理が、アーカイブズの時間スケール (数百年) に引き伸ばされて適用されている、と見ることができる。

8.4 学習ロードマップ

アーカイブズ学を学び始めた読者を想定した、無理のない順序を示す。

1. **概念の習得** (本書第1~4章 + RiC-CM 原文)。技術は一切不要。RiC-CMの実体とスコープノート、自分の知っている記録群に当てはめて読む。
2. **グラフに慣れる** (第5章 + 紙とペン)。身近な記録群を実体と関係の図に描く練習。この段階も技術不要で、しかしRiC的な思考の核はここで身につく。
3. **Turtle と SPARQL の読み書き** (第5~6章 + RiC-O 公式サンプル)。テキストエディタと無償ツールがあればよい。第7章末尾の演習がこの段階に当たる。
4. **小さな実データで一巡** (第7章)。手元データ数十件の変換・格納・検索・推論。Pythonの初歩 (他分野でも汎用的に役立つ) を並行して学ぶと選択肢が広がる。
5. **必要に応じた掘り下げ**。実装の道に進むならOWL・SHACL・システム運用へ。目録実務の道に進むなら、ここから先は概念の維持と、開発者と話せる語彙の維持で足りる。

最後に強調したい。RiC が要求する最も重要な素養は、プログラミングではなく**モデリング**—世界を実体と関係に切り分け、その切り分けの根拠を説明できる能力—である。そしてそれは、出所を見極め、編成を設計し、記述の判断を積み重ねてきたアーカイブズ学の伝統的な訓練と、本質的に同じ種類の能力である。RiC はアーキビストに新しい種類の仕事を課すというより、アーキビストが暗黙に行ってきた知的作業を、明示的で機械と共有可能な形式に書き出すことを求めている。

確認問題 (ヒントは付録 A.5)

- 問 8.1** RDB の外部キーと RDF のオブジェクトプロパティの違いを 3 点挙げよ (参照整合性・向き・スキーマの扱いに着目せよ)。
- 問 8.2** 自分の役割 (アーキビスト/IT 担当/研究者) を 1 つ選び、本章の技術地図から「次に学ぶ 2 つ」を選んで理由を述べよ。
- 問 8.3** データの形の検査という仕事において、SHACL と OWL の役割はどう違うか。開世界仮説という言葉を使って説明せよ。

付録 A

付録

A.1 RiC-CM 1.0 実体一覧

ID	名称	定義 (要約)
E01	もの Thing	人間の経験と言説の領域内のあらゆる観念・物・出来事
E02	記録資源 Record Resource	エージェントが生活・業務の中で作成・取得し保持した情報
E03	記録集合 Record Set	共有する属性・関係に基づきエージェントがまとめた 1 つ以上の記録
E04	記録 Record	担体に持続的・復元可能な形で記録された、ひとまとまりの情報内容
E05	記録部分 Record Part	記録の知的完結性に寄与する、独立の情報内容を持つ構成部分
E06	具現化 Instantiation	担体上への情報の記録。記録の物理的またはデジタルな現れ
E07	エージェント Agent	世界の中で活動を遂行するもの
E08	個人 Person	人間個人
E09	集団 Group	エージェントとして共に行動する 2 以上のエージェント
E10	家族 Family	出生・婚姻・養子縁組その他の社会的紐帯で結ばれた 2 人以上
E11	団体 Corporate Body	法的・社会的に認知された組織された集団
E12	職位 Position	集団の中での個人の機能的役割
E13	メカニズム Mechanism	個人・集団が作成した、活動を遂行するプロセスやシステム
E14	イベント Event	時間と空間の中で起こる出来事
E15	活動 Activity	エージェントが目的をもって設計・遂行するイベント
E16	規則 Rule	エージェントの存在・権限や活動の遂行などを支配する条件
E17	マンドート Mandate	あるエージェントから別のエージェントへの責任・権限の委任
E18	日付 Date	実体の識別と文脈化に寄与する年代情報
E22	場所 Place	境界を持つ、名前のついた地理的領域

E19～E21 は欠番である (0.2 版の日付下位実体が 1.0 で廃止された)。

A.2 関係の 13 の型 (RiC-CM §5.2)

型の名称の原語と、各型でもっとも広い関係を挙げる。定義域・値域つきの表は第 4 章の表 4.2 にある。

型 (本書の訳)	原文	各型の最上位の関係
全体-部分	whole/part relations	R002 has or had part
順序	sequential relations	R008 precedes or preceded
主題	subject relations	R019 has or had subject
記録資源間	record resource to record resource relations	R022 is record resource associated with record resource
記録資源-具現化	record resource to instantiation relations	R025 has or had instantiation
出所	provenance relations	R026 has provenance R033 documents
具現化間	instantiation to instantiation relations	R034 is instantiation associated with instantiation
管理	management relations	R036 has or had authority over
エージェント間	agent to agent relations	R044 is agent associated with agent
イベント	event relations	R057 is event associated with
規則	rule relations	R062 is rule associated with
日付	date relations	R068 is date associated with
空間	spatial relations	R074 is place associated with

すべての関係は最上位の R001 is related to(ものはものに関係する) の下位に位置づけられ、ポリヒエラルキーをなす。1 つの関係が複数の型に属することもある。関係自体の属性として、確実性 (RA01 Certainty of Relation)、日付 (RA02 Date of Relation)、記述 (RA03 Description of Relation)、識別子 (RA04 Identifier of Relation)、出典 (RA05 Source of Relation)、場所 (RA06 Place of Relation) が定義されている。

A.3 RiC-CM 1.0 関係一覧 — 原文との対照

本書の図表で用いた日本語の関係名と、RiC-CM 1.0 の原文名との対照表である。RiC-CM §5.4 の各関係の定義 (ID、Name、Domain/Range) から起こした。ID は原典では RiC-R001 から RiC-R086 まで振られている (本書では接頭辞を省いて R001 と書く)。R043 は欠番で、実際に定義されている関係は 85 である。各関係がどの型に属するかは A.2 節と第 4 章の表 4.2 にある。

読むときの注意を 3 つ。第一に、原文の has or had、is or was という言い回しは「現在そうである」場合と「過去にそうだった」場合の両方を含むという含意で、RiC が時間の中で変化する関係を扱うための工夫である。訳では煩雑になるため省いた。第二に、各関係には逆向きの関係 (ID の末尾に i が付く) が対で定義されている。この表は順方向だけを挙げている。第三に、定義域 (出発点になれる実体) と値域 (到達点になれる実体) は、A.1 節の実体一覧の名称で示した。

ID	原文の関係名	本書の訳	定義域 → 値域
R001	is related to	に関係する	もの → もの
R002	has or had part	部分を持つ	もの → もの
R003	has or had constituent	構成部分を持つ	記録・記録部分 → 記録・記録部分
R004	has or had component	構成要素を持つ	具現化 → 具現化
R005	has or had subdivision	下部組織を持つ	集団 → 集団
R006	has or had subevent	下位イベントを持つ	イベント → イベント
R007	contains or contained	含む (場所として)	場所 → 場所
R008	precedes or preceded	先行する	もの → もの
R009	precedes in time	時間的に先行する	もの → もの
R010	is original of	の原本である	記録・記録部分 → 記録・記録部分
R011	is draft of	の草稿である	記録 → 記録
R012	has copy	複本を持つ	記録資源 → 記録資源
R013	has reply	返信を持つ	記録資源 → 記録資源
R014	has or had derived instantiation	派生した具現化を持つ	具現化 → 具現化
R015	migrated into	へ移行した	具現化 → 具現化
R016	has successor	後継を持つ	エージェント → エージェント
R017	has descendant	子孫を持つ	個人 → 個人
R018	has child	子を持つ	個人 → 個人
R019	has or had subject	主題を持つ	記録資源 → もの
R020	has or had main subject	主要な主題を持つ	記録資源 → もの
R021	describes or described	を記述している	記録資源 → もの
R022	is record resource associated with record resource	記録資源として関連する	記録資源 → 記録資源
R023	has genetic link to record resource	記録資源と生成上のつながりを持つ	記録資源 → 記録資源
R024	includes or included	含む (記録集合として)	記録集合 → 記録・記録集合
R025	has or had instantiation	具現化を持つ	記録資源 → 具現化
R026	has provenance	出所を持つ	記録資源・具現化 → エージェント
R027	has creator	作成者を持つ	記録資源・具現化 → エージェント
R028	has accumulator	集積者を持つ	記録資源・具現化 → エージェント
R029	has receiver	受領者を持つ	記録資源・具現化 → エージェント
R030	has collector	収集者を持つ	記録資源・具現化 → エージェント
R031	has sender	送信者を持つ	記録資源・具現化 → エージェント
R032	has addressee	受信者を持つ	記録資源・具現化 → エージェント
R033	documents	文書化している	記録資源・具現化 → 活動
R034	is instantiation associated with instantiation	具現化として関連する	具現化 → 具現化
R035	is functionally equivalent to	と機能的に等価である	具現化 → 具現化
R036	has or had authority over	に権限を持つ	エージェント → もの
R037	is or was owner of	の所有者である	集団・個人・職位 → もの
R038	is or was manager of	の管理者である	エージェント → 記録資源・具現化
R039	is or was holder of	の所持者である	エージェント → 記録資源・具現化
R040	is or was holder of intellectual property rights of	の知的財産権者である	集団・個人・職位 → 記録資源・具現化
R041	is or was controller of	を統制している	エージェント → エージェント
R042	is or was leader of	の指導者である	個人 → 集団
R043	欠番 (<i>RiC-CM §5.4</i> に定義がない)		
R044	is agent associated with agent	エージェントとして関連する	エージェント → エージェント
R045	has or had subordinate	部下を持つ	エージェント → エージェント
R046	has or had work relation with	と業務上の関係を持つ	エージェント → エージェント
R047	has family association with	と家族的なつながりを持つ	個人 → 個人
R048	has sibling	きょうだいを持つ	個人 → 個人
R049	has or had spouse	配偶者を持つ	個人 → 個人
R050	knows of	の存在を知っている	個人 → 個人
R051	knows	と面識がある	個人 → 個人

(前ページからの続き)

ID	原文の関係名	本書の訳	定義域 → 値域
R052	has or had correspondent	文通相手を持つ	個人 → 個人
R053	has or had teacher	師を持つ	個人 → 個人
R054	occupies or occupied	に就いている	個人 → 職位
R055	has or had member	構成員を持つ	集団 → 個人
R056	exists or existed in	の中に置かれている	職位 → 集団
R057	is event associated with	イベントとして関連する	イベント → もの
R058	has or had participant	参加者を持つ	イベント → もの
R059	affects or affected	に影響を与えている	イベント → もの
R060	is or was performed by	によって遂行された	活動 → エージェント
R061	results or resulted in	をもたらした	イベント → もの
R062	is rule associated with	規則として関連する	規則 → もの
R063	regulates or regulated	を規律している	規則 → もの
R064	is or was expressed by	によって表現されている	規則 → 記録資源
R065	issued by	によって発行された	規則 → エージェント
R066	is or was enforced by	によって執行されている	規則 → エージェント
R067	authorizes	に権限を与える	マンドート → エージェント
R068	is date associated with	日付として関連する	日付 → もの
R069	is beginning date of	の開始日である	日付 → もの
R070	is birth date of	の生年月日である	日付 → 個人
R071	is end date of	の終了日である	日付 → もの
R072	is death date of	の没年月日である	日付 → 個人
R073	is modification date of	の改変日である	日付 → もの
R074	is place associated with	場所として関連する	場所 → もの
R075	is or was location of	の所在地である	場所 → もの
R076	is or was jurisdiction of	の管轄区域である	場所 → エージェント
R077	is or was adjacent to	と隣接している	場所 → 場所
R078	overlaps or overlapped	と重なっている	場所 → 場所
R079	has author	著者を持つ	記録 → 個人・集団・職位
R080	is creation date of	の作成日である	日付 → 記録資源・具現化
R081	is or was creation date of all members of	の全メンバーの作成日である	日付 → 記録集合
R082	is or was creation date of some members of	の一部メンバーの作成日である	日付 → 記録集合
R083	is or was creation date of most members of	の大半のメンバーの作成日である	日付 → 記録集合
R084	is date of occurrence of	の発生日である	日付 → イベント
R085	is within	の期間内にある	日付 → 日付
R086	intersects	と期間が重なる	日付 → 日付

A.4 よく使う RiC-O 語彙の抜粋

語彙	用途
<code>rico:RecordSet</code> / <code>rico:Record</code> / <code>rico:RecordPart</code>	記録資源の 3 区分
<code>rico:Instantiation</code>	具現化
<code>rico:Person</code> / <code>rico:Group</code> / <code>rico:CorporateBody</code> / <code>rico:Family</code> / <code>rico:Position</code> / <code>rico:Mechanism</code>	エージェント
<code>rico:Activity</code> / <code>rico:Event</code> / <code>rico:Rule</code> / <code>rico:Mandate</code> / <code>rico:Date</code> / <code>rico:Place</code> / <code>rico:Proxy</code>	文脈の実体
<code>rico:name</code> / <code>rico:title</code> / <code>rico:identifier</code> / <code>rico:generalDescription</code> / <code>rico:scopeAndContent</code> / <code>rico:history</code>	記録資源の「特定の集合の中での分身」(順序・文脈情報の担い手) 基本のリテラル属性 (実体化しない簡便法) 散文系の属性
<code>rico:includesOrIncluded</code> / <code>rico:isOrWasIncludedIn</code>	記録集合と記録 (集合) の所属
<code>rico:directlyIncludes</code> / <code>rico:includesTransitive</code>	直接の包含/推移閉包 (推論用)
<code>rico:hasCreator</code> / <code>rico:hasOrganicProvenance</code> / <code>rico:hasAccumulator</code>	出所・作成
<code>rico:performsOrPerformed</code> / <code>rico:documents</code> / <code>rico:hasSuccessor</code> / <code>rico:isSuccessorOf</code>	活動の遂行/活動の文書化 エージェントの変遷
<code>rico:hasOrHadInstantiation(analogue/digital 変種あり)</code>	記録資源と具現化
<code>rico:thingIsSourceOfRelation</code> / <code>rico:relationHasTarget</code>	実体化された関係の始点・終点
<code>rico:isAssociatedWithDate</code> / <code>rico:normalizedDateValue</code>	日付の関係づけと正規化値

A.5 確認問題のヒント

■第 1 章 問 1.1:1.2 節の 4 部構成の図と表。問 1.2:CM は概念 (安定)、O は実装 (修正が入る)—1.2 節と 5.3 節。問 1.3:1.3 節の一次資料表と、序文の読者案内。

■第 2 章 問 2.1:2.4.2 項。識別子を持つこと=参照の受け手になれること。問 2.2:7.3.1 項の EAD 例が再利用できる (`authfilenumber` と `relationEntry`)。問 2.3:2.4.4 項第 4 期と 7.3.1 項。問 2.4:2.6 節「図書館」の段落と 7.5 節。

■第 3 章 問 3.1: 表 3.1(自由と負担の対応)。問 3.2:3.6 節の (6)。同じ作成者関係を直接プロパティで書く場合と関係実体で書く場合など。問 3.3:3.1 節のコラム。

■第 4 章 問 4.1:4.2 節のコラム (文集のたとえ)。問 4.2: 図 2.2 右側や図 7.3 が形の参考になる。問 4.3:4.5 節。検証可能性と改訂履歴。問 4.4:4.5 節末尾の段落。

■第 5 章 問 5.1: 文法コラムと 5.4 節。`rico:isOrWasIncludedIn` は「～に含まれる (た)」。問 5.2: 文法コラムの 3 通りの等価な書き方。問 5.3:5.2.1 項の R^- の定義。問 5.4:5.2 節のコラム「クラスが主語になる書き方」。

■第6章 問 6.1:6.1 節と 5.2.1 項の「連鎖」。問 6.2:6.8.3 項。問 6.3:6.8.2 項。削除・更新時の再計算に注目。問 6.4: 場面 5 の図とリスティング。出所関係→団体→後継の順。

■第7章 問 7.1:7.2 節の工程 2。問 7.2:7.3.2 項の Turtle が手本。問 7.3:7.3.4 項末尾。多の側の定石。問 7.4:7.6.3 項。年度は期間 1933-04/1934-03。問 7.5:7.7 節の 4 つの道具立て。

■第8章 問 8.1:8.3 節の対応表と 7.3.1 項の図 7.2。問 8.2:8.1 節の役割別水準表。問 8.3:8.3 節と 6.9 節。OWL の公理は「制約」ではなく推論の根拠。

A.6 用語集 — 原語との対照、分野で意味がずれる言葉

前半はアーカイブズ学の基本用語で、分野の外から来た読者のために原語を添えた。後半は、隣接分野のあいだで意味がずれるために注意が要る言葉である。

■アーカイブズ学の基本用語

出所 provenance 記録がどの主体の、どんな営みの産物であるかという**生成の文脈**。RiC-FAD §5 の言い方では「ある個人・集団が生活や業務の中で作成・蓄積・使用した」という関係を指す。日常語の「出どころ」に引かれて「どこを経てきたか」まで含めたくなるが、記録が作成者の手を離れたあとの移り変わりは**伝来**として区別するのが標準の用法である。RiC は出所を単一の蓄積者に限らず、作成・集積・送受信に関わった複数の主体と、記録を生んだ活動にまで広げた (第3章 3.3 節)。

伝来 記録が作成者の手を離れたあとの、所有・責任・管理の移り変わり。ISAD(G) の記述要素 3.2.3 がこれにあたり (国立公文書館訳の見出しが「伝来」)、編成の履歴や検索手段の作成、ソフトウェアのマイグレーションなど、現在の構造をかたちづくった行為も含む。英語名は標準によって異なり、ISAD(G) は archival history、米国の DACS(5.1) は custodial history と呼ぶが、日本語ではどちらも伝来である。目的は真正性・完全性・解釈にとって重要な履歴を示すこと。RiC では散文としては History 属性 (RiC-A21) に収まり、構造化して書くなら管理関係 (所持者 R039、管理者 R038) で表す。出所関係とは型が別である (第3章 3.3 節のコラム)。

出所原則 principle of provenance 出所を記述と管理の基礎に置く原則。2 つの下位原則からなる。**フォンド尊重 (respect des fonds)**: ある主体が作成・蓄積した記録はひとまとまりに保ち、他の主体の記録と混ぜない。**原秩序尊重 (respect for original order)**: 蓄積・使用の過程で形成された記録どうしの秩序を保存する。図書館の分類が主題によって資料を並べ替えるのに対し、アーカイブズは出所によるまとまりを崩さない。第2章参照。

フォンド fonds 1 つの主体が生活または業務の中で作成・蓄積・使用した記録の全体。フランス語由来で単複同形。従来の記述はフォンドを頂点とする階層で組み立てられた。RiC では記録集合の種類の1つになる (第3章 3.4 節)。

記述単位 unit of description ISAD(G) で、1 件の記述の対象となるまとまり。フォンドでも 1 通の書簡でも、同じ 26 要素で記述できるという抽象。RiC はこれを記録・記録集合・記録部分の 3 実体に解体した。

多段階記述 multilevel description フォンドから下位のまとまりへと段階的に降りていく ISAD(G) の

記述方式。上位に書いたことを下位で繰り返さない (非反復の原則) などの規則を伴う。

検索手段 finding aid / **目録** 検索手段は、利用者が記録にたどり着くための道具の総称 (finding aid の定着訳)。目録・ガイド・索引などはその種類で、日本の実務の代表格が目録。ISAD(G) は検索手段の中身となる記述の標準、EAD はその符号化形式。第 2 章参照。

典拠レコード authority record 作成者などの情報を記録の記述から切り出して独立させたレコード (ISAAR(CPF))。RiC では Agent 実体とその関係に対応する (第 7 章 7.3 節)。

アクセスポイント access point 記述の中に埋め込まれる、人物・団体・場所・主題の統制された名前 (文字列)。それ自体は記述されない。

具現化 instantiation 記録という知的な内容が、担体 (紙、フィルム、デジタルファイル) の上に記録された現れ。同じ記録が複数の具現化を持ちうる。RiC が新たに立てた実体である (第 3 章 3.5 節)。

GLAM 美術館・図書館・文書館・博物館 (Galleries, Libraries, Archives, Museums) の総称。文化資源の収集・保存・記述という共通の仕事を持つ。第 2 章 2.6 節参照。

■分野で意味がずれる言葉

管理メタデータ administrative metadata 記述それ自体についてのメタデータ (誰がいつどの規則で作成・改訂したか)。RiC では記述レコードという一段上の記録への記述メタデータとして、同じ実体・関係で表す。第 4 章 4.5 節参照。

参照 reference ある記述の中から別のものを名指すこと。名前の文字列→レコード ID → IRI と強化されてきた (第 2 章)。矢印の向きの読み方は第 5 章 5.1.2 節。

実体 entity ERM で、関心対象の「もの」の種類 (実体型) またはその実例。RiC-CM の 19 実体は実体型である。第 4 章 4.1 節参照。

クラス class RDF/OWL における実体型に相当する概念。RiC-O のクラスは RiC-CM の実体 + 実装用の追加概念。

個体 individual / **資源** resource RDF/OWL におけるインスタンス。IRI で識別される。

属性 attribute / **プロパティ** property ERM の属性は実体の特性。RDF ではもの-値を結ぶデータタイププロパティと、もの-ものを結ぶオブジェクトプロパティに分かれ、後者は ERM では「関係」に当たる。同じ語が層によって別物を指すので注意。

関係 relation ERM で実体間のつながり。RDF 実装ではオブジェクトプロパティまたは関係クラス (実体化) として現れる。

「一種である」と「属する」実体階層の「一種である」(記録は記録資源の一種、という分類) と、記録が記録集合に「属する」(通常の関係、R024 など) は別物である。記録と記録集合の間に「一種である」の関係はない。第 4 章のコラム参照。

定義域 domain / **値域** range 関係 (プロパティ) の出発点/到達点になれる実体 (クラス) の指定。

トリプル triple RDF のデータ単位。主語・述語・目的語の 3 つ組。

a(Turtle の述語) 「～というクラスのインスタンスである」を表す述語 rdf:type の略記。第 5 章の文法コラム参照。

IRI 国際化資源識別子。ウェブのドメイン登録が保証する一意性を借用した、世界規模で一意な名前。

ページが開けることは条件ではない。第 5 章参照。

名前空間 namespace IRI の共通する前半部分。「どの機関の、どの語彙集の名前か」を表す。Turtle では PREFIX 宣言で短い接頭辞 (rico: など) を与えて略記する。

リテラル literal 文字列・数値・日付などの生の値。

オントロジー ontology ある領域のクラスとプロパティを機械可読に定義した語彙体系。哲学の存在論とは別物 (ただし語源は同じ)。

推論 inference 明示されたデータと公理から、書かれていないトリプルを機械的に導くこと。

実体化 reification 関係や属性値を、それ自体 1 個のノード (個体) に昇格させる技法。関係に属性を付けたときに使う。

グラフ graph 点 (ノード) と、点をつなぐ線 (アーク) からなる構造。棒グラフや折れ線グラフとは別物である。路線図や家系図と同じ形で、木 (階層) はその特殊な場合。第 2 章 2.3 節参照。

推移閉包 transitive closure 直接の関係からたどれるかぎりたどって得られる関係の全体。「直接含む」の推移閉包が「(間接も含めて) 含む」である。第 6 章 6.2 節参照。

開世界仮説 open world assumption 「書かれていないことは偽ではなく不明」とする解釈の立場。OWL はこの立場をとる。

閉世界仮説 closed world assumption 「書かれていないことは偽」とする立場。リレーショナルデータベースや第二次 AI ブームの知識表現はこちらに立ち、常識推論 (例外を後から足せる規則) はこの仮説の上ではじめて書ける。データが完全でない領域では誤った否定を導くため、OWL は開世界仮説を採った。第 3 章 3.1 節のコラム参照。

LOD (Linked Open Data) IRI で名指され相互リンクされた、公開 RDF データの総称ないし公開の作法。

SPARQL RDF グラフの検索言語。

SHACL RDF データの形を検査する制約記述言語。

SKOS 統制語彙 (シソーラス・件名標目表・分類表) を RDF で表す W3C 標準。概念 (skos:Concept) を個体として立て、prefLabel/altLabel と broader/narrower で結ぶ。broader は subclassOf ではない。第 7 章のコラム参照。

名前付きグラフ named graph トリプルに「所属グラフ」のラベルを付けて格納・検索する仕組み (SPARQL 1.1)。典拠系と目録系を分割管理したまま論理的に統合できる。第 7 章 7.3 節参照。

トリプルストア triplestore RDF を格納し SPARQL 検索 (しばしば推論も) を提供するデータベース。

A.7 一次資料と参考リソース

本書のソースリポジトリの `sources/` には、以下の一次資料のうち RiC-FAD 1.0・RiC-CM 1.0・RiC-O 1.1(RDF 本体と構成要素の CSV 一覧)・RiC-AG を実物のまま収めてある。取得元・取得日と、原典を読むときの注意は同ディレクトリの README.md にある。

- RiC Resource List(実例・文献の集約)— ica-egad.github.io/RiC-ResourceList
- AnF Sparnatural デモ (公証人記録の知識グラフ検索)—

`sparna-git.github.io/sparnatural-demonstrateur-an`

- RiC-FAD 1.0 (2023) — github.com/ICA-EGAD/RiC-FAD
- RiC-CM 1.0 (2023) — github.com/ICA-EGAD/RiC-CM(ICA サイトからも入手可)
- RiC-O 1.1 (2025) — github.com/ICA-EGAD/RiC-O
- RiC-AG 0.1 草案 (2025 年 10 月 29 日、ICA バルセロナ大会で公開) — github.com/ICA-EGAD/RiC-AG(本文、既存 4 標準→ RiC-CM および EAD 2002 → RiC-O 1.1 の公式クロスウォーク、ダイアグラム)
- RiC-O 解説サイト — ica-egad.github.io/RiC-O/(About、Why use RiC-O?、Diagrams、Examples ほか)
- RiC-O Converter — EGAD 公開の EAD/EAC-CPF 変換ツール (2020 年初版)
- RiC-ResourceList — ica-egad.github.io/RiC-ResourceList(コミュニティが共同管理するプロジェクト・文献一覧、2025 年 4 月開設)
- Records in Contexts users — 利用者コミュニティ (Google Group)
- 従来標準:ISAD(G) 2nd ed. (2000)、ISAAR(CPF) 2nd ed. (2004)、ISDF (2007)、ISDIAH (2008)、いずれも ICA サイトで公開
- 関連標準:ISO 23081(記録管理メタデータ)、PREMIS(保存メタデータ)、EAD/EAC-CPF(符号化)、W3C RDF/OWL/SPARQL/SKOS/SHACL 各仕様

索引

A

ABox, **55**, **73**
 authority record, → 典拠レコード

B

BIBFRAME, **15**

C

CIDOC CRM, **15**

D

Datalog, **55**

E

EAC-CPF, **6**, **79**
 EAD, **6**, **60**, **79**
 EDTF, **92**
 entity, → 実体
 ERM, **27**

F

finding aid, → 検索手段
 FranceArchives, **77**

G

GLAM, **15**
 GraphDB, **72**

I

identifier, → 識別子
 IIIF, **15**
 inference, → 推論
 instantiation, → 具現化
 IRI, **14**, **42**, **72**, **91**
 ISAAR(CPF), **6**
 ISAD(G), **6**
 ISDF, **6**
 ISDIAH, **6**
 ISO 23081, **14**

J

JSON, **47**
 JSON-LD, **47**

L

LOD, **10**, **100**

M

materialization, → マテリアライゼーション
 Memobase, **77**

N

N-Triples, **47**
 named graph, → 名前付きグラフ

O

OWL, **49**, **88**
 OWL 2 プロファイル, **72**

P

PIAAF, **77**
 PREMIS, **15**
 provenance, → 出所
 Proxy, **29**, **60**

R

RDF, **42**
 RDF/XML, **47**
 RDFS, **49**
 reference, → 参照
 reification, → 実体化
 RiC-AG, **1**
 RiC-CM, **1**
 RiC-FAD, **1**
 RiC-O, **1**
 RiC-O Converter, **79**

S

SHACL, **58**, **79**
 SKOS, **57**, **86**
 SPARQL, **58**, **64**, **72**

T

TBox, **55**, **73**
 TriG, **84**
 Turtle, **47**, **64**

V

VIAF, **91**

W

Wikidata, **91**

X

XML, **8**, **47**

あ

アクセス制限, **94**
 アクセスポイント (access point), **7**
 異体字, **92**
 一階述語論理 (first-order logic), **51**
 エージェント (agent), **29**, **53**, **91**
 オントロジー (ontology), **2**, **49**

か

開世界仮説 (open world assumption), **18**, **75**
 外部キー (foreign key), **101**
 関係 (relation), **2**, **11**, **22**, **34**, **60**, **67**, **97**, **101**
 管理メタデータ (administrative metadata), **38**
 記述の記述, **38**

記述論理 (description logic), **50**
逆プロパティ (inverse property), **48**, 67
記録 (record), **29**
記録資源 (record resource), 22, **31**, 60, 80
記録集合 (record set), 22, **29**, 101
記録部分 (record part), 22, **31**, 101
空白ノード (blank node), **44**
具現化 (instantiation), 16, **23**, 32, 102
クラス (class), 2, 34, **49**
グラフ (graph), **9**, 40, 94
クロスウォーク (crosswalk), **78**
結合表, **11**
決定可能性 (decidability), **52**
検索手段 (finding aid), **6**, 17, 49, 80
語彙 (vocabulary), 3, **57**, 70, 90
個人情報, **94**

さ

参照 (reference), **10**, 48
識別子 (identifier), **10**, 91
実体 (entity), 2, 14, 25, **27**
実体化 (reification), 37, **60**, 85, 101
出所 (provenance), **19**, 38, 69, 85
出所原則 (principle of provenance), **7**
シリーズシステム (series system), **86**
推移性 (transitivity), **65**
推移閉包 (transitive closure), **65**
推論 (inference), 18, 49, **64**, 102
正規化 (normalization), **11**, 93
層構造, **57**
属性 (attribute), 19, **33**, 59, 84
属性 (attribute、XML), **8**

た

対称プロパティ (symmetric property), **48**
多言語ラベル, **92**
多次元記述 (multidimensional description), **19**
多段階記述 (multilevel description), **7**
縦持ち・横持ち, 43
積み上げ型 (stacked), **43**
データ独立性 (data independence), **101**
適用プロファイル (application profile), **78**
典拠レコード (authority record), **10**, 79
伝来, **20**
ドメイン知識, **86**
トリプル (triple), **42**, 84
トリプルストア (triplestore), **72**

な

名前空間 (namespace), **44**, 70
名前付きグラフ (named graph), **84**

は

パニング (punning), **55**
非単調推論 (non-monotonic reasoning), **18**
表計算での記述, 96
表現の一意性, **24**
フォンド (fonds), **7**, 22, 65, 86
プロパティ (property), 2, 25, 38, **49**, 90
プロパティ連鎖 (property chain), **68**
閉世界仮説 (closed world assumption), **18**

ま

マッピング (mapping), **78**
マテリアライゼーション (materialization), **72**
名称統制, **10**

目録, **6**

や

矢印の向き, **48**
要素 (element、XML), **8**
読み (ヨミ), 92

ら

リテラル (literal), 24, **42**, 78
リンクセット (linkset), 85
レファレンスコード (reference code), **10**

わ

和暦, **92**

Records in Contexts 入門

—アーカイブズ記述の新しい国際標準を理解する—

著者 久保山哲二
(学習院大学大学院人文科学研究科アーカイブズ学専攻/計算機センター)
発行 2026 年 8 月 (第 1.3 版)

本書の執筆には大規模言語モデル Claude Fable 5 と Claude Opus 5(Anthropic) を用いた。本書の構成と内容に関する文責は久保山哲二にある。本文組版は LuaLaTeX(luatexja、原ノ味フォント) による。

本書はクリエイティブ・コモンズ表示 4.0 国際ライセンス (CC BY 4.0) で提供する。出典を表示すれば、複製・再配布・改変・翻訳・授業や出版での利用を自由に行ってよい。一次資料である ICA EGAD の RiC 各文書も同じ CC BY 4.0 で公開されている。
